HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I

BÀI GIẢNG
LẬP TRÌNH WEB



PTIT

HÀ NỘI - 2014

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN I

BÀI GIẢNG

LẬP TRÌNH WEB

PTIT

NGƯỜI VIẾT: THS.TRỊNH THỊ VÂN ANH

HÀ NỘI - 2014

LỜI MỞ ĐẦU

Trong những năm gần đây, cùng với sự phát triển của Công nghệ thông tin, mạng máy tính đang được phát triển rộng rãi, một trong những ứng dụng phổ biến trên mạng máy tính là Internet và các dịch vụ trở nên không thể thiếu trong cuộc sống hiện đại. Để phục vụ việc học tập nghiên cứu của sinh viên ngành Công Nghệ Thông Tin, chúng tôi biên soạn **bài giảng lập trình Web**. Sách bao gồm 9 chương, mỗi chương đều có phần kiến thức lý thuyết và các ví dụ cụ thể cho phần lý thuyết đó.

Chương 1: Ngôn ngữ HTML

Trình bày khái niệm ngôn ngữ đánh dấu siêu văn bản, các thành phần cơ bản và cấu trúc của một tập tin HTML. Giới thiệu các vấn đề liên quan đến việc sử dụng cú pháp và hiệu ứng của ngôn ngữ đánh dấu siêu văn bản, để lập trình Web.

Chương 2: CSS (Cascading Style Sheets)

Đây là là một ngôn ngữ quy định cách trình bày cho các tài liệu viết bằng HTML, XHTML hay XML,... Nó cung cấp cho chúng ta hàng trăm thuộc tính trình bày dành cho các đối tượng với sự sáng tạo cao. Hiện tại CSS đã được hỗ trợ bởi tất cả các trình duyệt.

Chương 3: JavaScript

Ngôn ngữ kịch bản trong lập trình Web Giới thiệu ngôn ngữ kịch bản phổ biến hiện nay là JavaScript. Hướng dẫn các bước tiến hành khai báo, lập trình và sử dụng ngôn ngữ kịch bản trong HTML.

Chương 4: XML (eXtensible Markup Language)

Chương này sẽ lần lượt giới thiệu một số khái niệm cơ bản trong XML, cách tạo 1 XML trong Netbean và những quy ước cơ bản cần biết khi xây dựng một tài liệu XML. Đưa môt số ví dụ và công cụ hỗ trợ phân tích các trang web ở dạng xml hoặc html.

Chương 5: Using JSP tags

Giới thiệu ngôn ngữ lập trình jsp phía server gồm: các thẻ, cú pháp ngôn ngữ. Bên cạnh giới thiệu cách kết nối với csdl, các ví dụ thao tác với csdl bằng jsp. Sau đó nêu chi tiết 1 số đối tượng và action của jsp.

Chương 6: Servlet

Đây cũng là ngôn ngữ viết phía server. Trong chương này chúng tôi giới thiệu tạo một dự án với servlet, giới thiệu 2 đối tượng quan trọng của nó là request và response. Giới thiệu một số module phía người dùng và admin khi xử lý bằng ngôn ngữ servlet.

PTIT

Chương 7: Session

Trong chương này giới thiệu về session, sau đó áp dụng nó trong module đăng nhập - đăng ký và giỏ hàng.

Chương 8: JSP custom tags

Giúp người học hiểu công dụng của việc dùng các thẻ mở rộng (custom tags), nêu các bước tạo thẻ mở rộng. Hướng dẫn cách dùng 2 thẻ mở rộng có sẵn đó là CKeditor và CKfinder trong module admin. Cuối cùng hướng dẫn các bạn cách tạo thẻ mở rộng trong phần module View của mô hình MVC.

Chương 9: Các nền tảng hỗ trợ phát triển ứng dụng web trên J2EE

Trong chương này giới thiệu và cài đặt 2 frame work Spring MVC và Strust 2.

Bài giảng lập trình Web hướng dẫn cách xây dựng một ứng dụng viết trên internet từ cơ bản đến nâng cao bằng công nghệ HTML, servlet và JSP. Được biên soạn với phương châm đảm bảo tính logic, khoa học, thiết thực, dễ hiểu nhằm trang bị cho sinh viên những kiến thức cơ bản, phục vụ cho nghiên cứu, thiết kế, lập trình một ứng dụng Web hoàn chỉnh. Tài liệu này được tham khảo từ một số tài liệu của các tác giả trong nước, tổng hợp và lược dịch từ một số tài liệu chuyên ngành của nước ngoài. Tuy đã rất cố gắng trong công tác biên soạn, nhưng vẫn không tránh khỏi thiếu sót. Trong phạm vi hạn hẹp của cuốn bài giảng này, chúng tôi không thể đề cập được tất cả những vấn đề đang nóng hổi trong lĩnh vực thiết kế và lập trình Web đòi hỏi. Rất mong sự đóng góp phê bình từ các bạn đồng nghiệp và các em học sinh khi khi đọc cuốn bài giảng này.

PTIT

Tác giả - Giáo viên: Ths. Trịnh Thị Vân Anh

MỤC LỤC

LỜI MỞ ĐẦU 3
MỤC LỤC 5
CHƯƠNG 1: NGÔN NGỮ HTML 9
1. GIỚI THIỆU HTML 9
1.1 Cấu trúc tài liệu HTML 9
1.2 Cách đưa ảnh vào một trang web 11
1.3 Sử dụng các siêu liên kết 12
2. CÁC THẺ HTML CƠ BẢN 15
2.1 Tiêu đề (Heading) 15
2.2 Thẻ khối <SPAN>, <DIV> 16
2.3 Kẻ đường ngang 16
2.4 Thẻ font 17
2.5 Bảng (table) 17
2.6 Danh sách (list) 19
2.7 Chèn đa phương tiện (Multimedia) 19
2.8 Biểu mẫu (form) 20
CHƯƠNG 2: BẢNG ĐỊNH DẠNG THEO TẦNG 23
1. KHÁI NIỆM 23
2. CÚ PHÁP 24
3. CLASS VÀ ID 29
CASE STUDY 1 (HTML & CSS) 34
CHƯƠNG 3: NGÔN NGỮ KỊCH BẢN JAVASCRIPT 35
1. GIỚI THIỆU 35
1.1 Giới thiệu về JavaScript 35
1.2 Chèn JavaScript vào trang web 35
2. NỀN TẢNG VÀ CÚ PHÁP 37
2.1 Biến và dữ liệu 37
2.2 Các toán tử 38

PTIT

2.3 Biểu thức qui tắc 39
2.4 Tập lệnh 42
3. CÁC ĐỐI TƯỢNG CƠ BẢN TRONG JAVASCRIPT 44
3.1 Các đối tượng cơ bản 44
3.2 Đối tượng event 48
3.3 Các sự kiện 49
CHƯƠNG 4: NGÔN NGỮ XML 52
1. CẤU TRÚC VÀ CÚ PHÁP CỦA XML 52
1.1 XML là gì 52
1.2 Một số qui tắc của XML 52
1.3 Chỉ thị xử lý (Processing Instructions) và lời chú thích (Comments) 54
1.4 Không gian tên (namespace) 54
2. DTD (DOCUMENT TYPE DEFINITION) 56
2.1 DTD là gì? 56
2.2 Định nghĩa một tài liệu DTD 56
3. CÁC ỨNG DỤNG CỦA PHÂN TÍCH (PARSER) XML 58
3.1 Các cách phân tích (parser) xâu XML trong Java 58
3.2 Dùng DOM 59
3.3 Dùng SAX 63
3.4 Dùng StAX 66
3.5 Phân tích HTML trong Java dùng Jsoup 70
3.6 Dùng Json 73
CASE STUDY 2 (JAVASCRIPT & XML) 78
CHƯƠNG 5: SỬ DỤNG CÁC THẺ JSP 80
1. GIỚI THIỆU JSP (JAVA SERVER PAGES) 80
2. CÚ PHÁP JSP 83
2.1 Các toán tử trong ngôn ngữ jsp 83
2.2 Các câu lệnh trong ngôn ngữ jsp 83
3. ĐỐI TƯỢNG REQUEST VÀ RESPONSE 84

PTIT

3.1 Đối tượng Request 84
3.2 Đối tượng Response 85
4. CÁC ACTIONS CHUẨN CỦA JSP 86
4.1 <jsp:include > 86
4.2 JavaBean 87
4.3 <jsp:element>, <jsp:attribute> và <jsp:body> 89
5. KẾT NỐI CSDL 90
5.1 Các bước làm việc với CSDL 90
5.2 Một số ví dụ cho cơ sở dữ liệu (CSDL) 92
CASE STUDY 3 + 4 (DÙNG JSP) 95
CHƯƠNG 6: GIỚI THIỆU SERVLET 96
1. GIỚI THIỆU SERVLET 96
1.1 Servlet là gì 96
1.2 Tạo dự án Web với servlet 97
2. CÁC ĐỐI TƯỢNG REQUEST VÀ RESPONSE 99
2.1 Đối tượng request 99
2.2 Đối tượng response 100
3. Ví dụ cho một số module phía server 100
CASE STUDY 3 (DÙNG JSP + SERVLET) 111
CHƯƠNG 7: ĐỐI TƯỢNG SESSION 112
1. ĐỐI TƯỢNG SESSION 112
2. ỨNG DỤNG ĐỐI TƯỢNG SESSION KHI ĐĂNG NHẬP 112
3. GIỎ HÀNG 115
CASE STUDY 3 + 4 118
CHƯƠNG 8: CÁC THẺ MỞ RỘNG (JSP CUSTOM TAGS) 119
1. GIỚI THIỆU CUSTOM TAG 119
1.1 Giới thiệu custom tag 119
1.2 Ưu điểm của Custom tag 119
2. CÁC BƯỚC ĐỂ CÀI ĐẶT CUSTOM TAG DÙNG JSP 121

PTIT

2.1 Từng bước tạo 1 Custom tag 121
2.2 CKEditor trong jsp 125
2.3 CKFinder trong jsp 125
3. VÍ DỤ MINH HỌA 127
BÀI CUỐI KỲ (TRANG WEB HOÀN CHỈNH) 132
CHƯƠNG 9: CÁC NỀN TÀNG HỖ TRỢ PHÁT TRIỂN ỨNG DỤNG WEB TRÊN J2EE 133
1. KHÁI QUÁT VỀ CÁC NỀN TẢNG HỖ TRỢ PHÁT TRIỂN WEB 133
1.1 Hệ nền J2EE 133
1.2 Các ứng dụng phân tán đa tầng 133
1.3 Hai framework hỗ trợ phát triển web 134
2. SPRING 135
2.1 Giới thiệu 135
2.2 Spring Web MVC 135
2.3 Cơ chế xử lý Request-Respone 139
2.4 Tạo ứng dụng login với spring 139
2.5 Kết hợp Spring Framework với Hibernate Framework 145
2.6 Ví dụ thêm, sửa, xóa (CRUD) trong Spring Framework 149
3. STRUTS 156
3.1 Khái niệm 156
3.2 Cấu trúc của Strust 157
3.3 Tạo Login với Strust 2 158
TÀI LIỆU THAM KHẢO 161

PTIT

CHƯƠNG 1: NGÔN NGỮ HTML

1. GIỚI THIỆU HTML

1.1 Cấu trúc tài liệu HTML

"Internet","World Wide Web", và "Web page" không chỉ còn là các thuật ngữ. Giờ đây, các thuật ngữ này đã trở thành hiện thực. Internet là mạng máy tính lớn nhất trên thế giới, được xem như là mạng của các mạng. World Wide Web là một tập con của Internet. World Wide Web gồm các Web Servers có mặt khắp mọi nơi trên thế giới. Các Web server chứa thông tin mà bất kỳ người dùng nào trên thế giới cũng có thể truy cập được. Các thông tin đó được lưu trữ dưới dạng các trang Web. Trong phần này, chúng ta sẽ học về Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language - HTML), đây là một phần quan trọng trong lĩnh vực thiết kế và phát triển thế giới Web. Ngôn ngữ đánh dấu siêu văn bản chỉ rõ một trang Web được hiển thị như thế nào trong một trình duyệt. Sử dụng các thẻ và các phần tử HTML, ta có thể:

- Điều khiển hình thức và nội dung của trang
- Xuất bản các tài liệu trực tuyến và truy xuất thông tin trực tuyến bằng cách sử dụng các liên kết được chèn vào tài liệu HTML
- Tạo các biểu mẫu trực tuyến để thu thập thông tin về người dùng, quản lý các giao dịch .....
- Chèn các đối tượng như audio clip, video clip, các thành phần ActiveX và các Java Applet vào tài liệu HTML

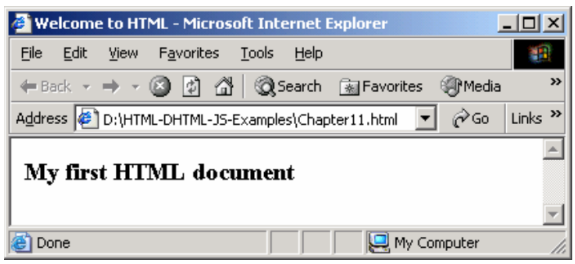
Tài liệu HTML, tạo thành mã nguồn của trang Web. Khi được xem trên trình soạn thảo, tài liệu này là một chuỗi các thẻ và các phần tử, mà chúng xác định trang web hiển thị như thế nào. Trình duyệt đọc các file có đuôi .htm hay .html và hiển thị trang web đó theo các lệnh có trong đó.

Cấu trúc cơ bản:

Một document HTML luôn bắt đầu bằng <html> và kết thúc bằng </html> (trong đó ta cũng có thể viết nhỏ hoặc viết hoa <html> và </html>). Nhờ có cặp thẻ này mà Browser bắt được đó là HTML - document để trình duyệt. Những chữ đó chỉ để dành riêng cho Browser, người đọc chỉ nhận được những gì viết giữa cặp thẻ <body> và </body>. Trong một document html, chú thích được dùng như sau: <!-- Đây là dòng chú thích, chỉ dành riêng cho bạn, browser bỏ qua dòng này --> Ví dụ, theo cú pháp HTML dưới đây sẽ hiển thị thông điệp "My first HTML document".

PTIT

```
<HTML>
<HEAD>
<TITLE>Welcome to HTML </TITLE>
</HEAD>
<BODY>
  <H3>My first HTML document</H3>
</BODY>
</HTML>
```

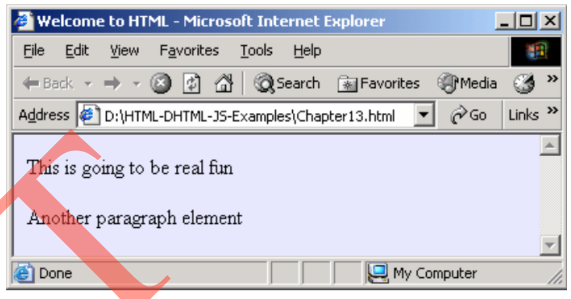


Hình 1.1: Ví dụ cú pháp HTML

Thẻ đoạn <p>:

Thẻ <P> để trình bày một đoạn văn bản. Trong một tài liệu HTML, nội dung có thể được nhóm thành các đoạn. Thẻ đoạn <P> được sử dụng để đánh dấu sự bắt đầu của một đoạn mới.

```
<HTML>
<HEAD>
<TITLE>Welcome to HTML</TITLE>
</HEAD>
<BODY BGCOLOR = lavender>
  <P>This is going to be real fun
  <P> Another paragraph element
</BODY> </HTML>
```



Hình 1.2 Ví dụ thẻ đoạn

Thẻ đóng </P> là không bắt buộc. Thẻ <P> kế tiếp sẽ tự động bắt đầu một đoạn mới.

**Thẻ ngắt
:**

Phần tử
 được sử dụng để ngắt dòng trong tài liệu HTML. Thẻ
 bổ sung một ký tự xuống dòng tại vị trí của thẻ.

Chọn canh lề:

Thuộc tính align được sử dụng để canh lề cho các phần tử HTML trong trang Web. Chúng ta có thể canh lề văn bản, các đối tượng, hình ảnh, các đoạn, các phân đoạn,…

Sau đây là các cách canh lề văn bản:

Bảng 1.1: Thuộc tính align

Value	**Description**
Left	Văn bản được căn lề trái
Center	Văn bản được căn lề giữa
Right	Văn bản được căn lề phải

PTIT

Justify	Văn bản được căn lề hai bên

Ví dụ:

```
<HTML>  <HEAD>
<TITLE> Learning HTML</TITLE> </HEAD>
<BODY BGCOLOR=lavender>
  <P align = right>This is good fun</P>
</BODY> </HTML>
```

Thẻ <META>:

Phần tiêu đề cũng chứa phần tử META. Phần tử này cung cấp thông tin về trang web mà chúng ta xây dựng. Nó gồm tên tác giả, tên phần mềm dùng để viết trang đó, tên công ty, thông tin liên lạc… Phần tử META sử dụng sự kết hợp giữa thuộc tính và giá trị.

Ví dụ, để chỉ Trịnh Vân Anh là tác giả, người ta sử dụng phần tử META như sau:

```
<META name=”Author” content=” Trịnh Vân Anh”>
```

Tác giả của tài liệu là “Trịnh Vân Anh”

Thuộc tính http-equiv có thể được sử dụng để thay thế thuộc tính name. Máy chủ HTTP sử dụng thuộc tính này để tạo ra một đầu đáp ứng HTTP (HTTP response header). Đầu đáp ứng được truyền đến trình duyệt để nhận dạng dữ liệu. Nếu trình duyệt hiểu được đầu đáp ứng này, nó sẽ tiến hành các hành động đặc biệt đối với đầu đáp ứng đó. Ví dụ:

```
<META http-equiv=”Expires” content=”Mon, 15 Sep 2003 14:25:27 GMT”>
```

Hiển thị ra như sau: Expires: Mon, 15 Sep 2003 14:25:27 GMT

Do vậy, nếu tài liệu đã lưu lại, HTTP sẽ biết khi nào truy xuất một bản sao của tài liệu tương ứng.

Bảng 1.2: Các ký tự đặc biệt trong tài liệu HTML

Ký tự đặc biệt	**Mã định dạng**	**Ví dụ**
Lớn hơn (>)	>	`<CODE>If A > B Then   A = A + 1 </CODE>`
Nhỏ hơn (<)	<	
Trích dẫn (““)	"	`<BODY> " To be or not to be ? " That is the question </BODY>`
Ký tự “&”	&	`<P> William & Graham went to the fair`
một khoảng trắng		

1.2 Cách đưa ảnh vào một trang web

PTIT

Để làm cho trang web sinh động hơn, hấp dẫn hơn, ta có thể đưa thêm hình ảnh vào. Có những trang web liên quan đến số liệu, đồ thị và đồ thị cũng là một dạng ảnh (images). Sau đây là cách đưa một hình ảnh vào trong trang web.

- Ảnh cho vào trang web nên save dưới dạng .GIF hoặc .JPG
- Không nên to quá, đây không nói đến kích thước (cm) mà nói đến kính cỡ file (số KB).
- Khi chuyển quét, chúng ta nên chọn 72 dpi (dots per inch) bởi vì đa phần các màn hình máy tính làm việc với độ phân giải này.
- Khi upload lên trang web, cần upload cả ảnh.
- Tên của các tệp nên sử dụng chữ thường, không nên sử dụng chữ hoa. Ví dụ: myphoto.jpg chứ không Myphoto.jpg.

Sau đây là mã HTML để đưa hình ảnh vào trang web:

```
<html> <body bgcolor="#ffffff">
<center>
   <img src="myphoto.jpg" width="200" height="400"
       border="0" alt="Here is my first photo">
</center>
</body> </html>
```

Ví dụ: `<IMG SRC=Flowers.jpg ALT="Beautiful Flowers">`

Đưa ảnh làm nền:

```
<body bgcolor="#màu mà bạn thích" background="back.jpg">
```

Nếu ảnh nhỏ hơn phạm vi hiển thị của nền thì ảnh được xếp kề nhau để lấp đầy toàn bộ vùng hiển thị.

PTIT

1.3 Sử dụng các siêu liên kết

Siêu liên kết là một phần tử bên trong tài liệu mà liên kết đến một vị trí khác trong cùng tài liệu đó hoặc đến một tài liệu hoàn toàn khác. Ví dụ, khi ta kích vào siêu liên kết sẽ nhảy đến liên kết cần đến. Các siêu liên kết là thành phần quan trọng nhất của hệ thống siêu văn bản.

Khả năng chính của HTML là hỗ trợ siêu liên kết. Một siêu liên kết, hay nói ngắn gọn là một liên kết, là sự kết nối đến tài liệu hay tệp khác (đồ họa, âm thanh, video) hoặc ngay cả đến một phần khác trong cùng tài liệu đó. Khi kích vào siêu liên kết, người dùng được đưa đến địa chỉ URL mà chúng ta chỉ rõ trong liên kết.

- Một phần khác trong cùng tài liệu
- Một tài liệu khác
- Một phần trong tài liệu khác
- Các tệp khác – hình ảnh, âm thanh, trích đoạn video
- Vị trí hoặc máy chủ khác

Các liên kết có thể là liên kết trong hoặc liên kết ngoài

- Liên kết trong là liên kết kết nối đến các phần trong cùng tài liệu hoặc cùng một web site
- Liên kết ngoài là liên kết kết nối đến các trang trên các web site khác hoặc máy chủ khác.

Để tạo siêu liên kết, chúng ta cần phải xác định hai thành phần:

- Địa chỉ đầy đủ hoặc URL của tệp được kết nối
- Điểm nóng cung cấp cho liên kết. Điểm nóng này có thể là một dòng văn bản, thậm chí là một ảnh.

Khi người dùng kích vào điểm nóng, trình duyệt đọc địa chỉ được chỉ ra trong URL và "nhảy" đến vị trí mới. Mỗi nguồn tài nguyên trên Web có một địa chỉ duy nhất. Ví dụ, 207.46.130.149 là địa chỉ web site của Micorsoft. Giờ đây, để nhớ các con số này rất khó và dễ nhầm lẫn. Vì vậy, người ta sử dụng các URL. URL là một chuỗi cung cấp địa chỉ Internet của web site hay tài nguyên trên World Wide Web.

Ví dụ, 216.239.33.101 có thể được biểu diễn bằng URL là www.google.com

URL cũng có thể tham chiếu đến một vị trí trong một tài nguyên. Ví dụ, bạn có thể tạo liên kết đến một chủ đề trong cùng một tài liệu. Trong trường hợp đó, định danh đoạn được sử dụng ở phần cuối của URL.

Định dạng là: giao thức: tên của site / tài liệu chính #định danh đoạn

Có hai dạng URL:

- URL tuyệt đối: là địa chỉ Internet đầy đủ của trang hoặc tệp, bao gồm giao thức, vị trí mạng, đường dẫn tùy chọn và tên tệp. Ví dụ, http://www.microsoft.com là một địa chỉ URL tuyệt đối.
- URL tương đối: là một URL có một hoặc nhiều phần bị thiếu. Trình duyệt lấy thông tin bị thiếu từ trang chứa URL đó. Ví dụ, nếu giao thức bị thiếu, trình duyệt sử dụng giao thức của trang hiện thời.

Thẻ <A> được sử dụng để xác định văn bản hay ảnh nào sẽ dùng làm siêu liên kết trong tài liệu HTML. Thuộc tính HREF (tham chiếu siêu văn bản) được dùng để chỉ địa chỉ hay URL của tài liệu hoặc tệp được liên kết.

Cú pháp của HREF là:

```
<A HREF = "protocol://host.domain:port/path/filename">Hypertext</A>
```

Trong đó:

- Giao thức: Đây là loại giao thức. Một số giao thức thường dùng:
  - http: giao thức truyền siêu văn bản
  - telnet: mở một phiên telnet

PTIT

- Gopher: tìm kiếm file
- ftp: giao thức truyền file
- mailto: gửi thư điện tử

- Host.domain: Đây là địa chỉ Internet của máy chủ
- Port: Cổng phục vụ của máy chủ đích
- HyperText: Đây là văn bản hay hình ảnh mà người dùng cần nhấp vào để kích hoạt liên kết.

  Ví dụ:

```
<A HREF= "Doc1.html">Back Home</A>  (đường dẫn tương đối)
<A HREF= "C:\mydirectory\html examples\ Doc1.htm">Back Home</A>
(đường dẫn tuyệt đối)
```

Liên kết đến các phần trong cùng một tài liệu

Thẻ neo <A> (anchor) được sử dụng để người dùng có thể "nhảy" đến những phần khác nhau của một tài liệu. Ví dụ, bạn có thể hiển thị nội dung của trang Web như một loạt các liên kết. Khi người dùng kích vào một đề tài nào đó thì các chi tiết nằm ở một phần khác của tài liệu được hiển thị. Kiểu liên kết này gọi là "named anchor" bởi vì thuộc tính NAME được sử dụng để tạo các liên kết này:

```
<A NAME = "marker"> Topic name </A>
```

Chúng ta không phải sử dụng bất kỳ văn bản nào để đánh dấu điểm neo. Để dùng, ta sử dụng vạch dấu (marker) trong thông số HREF như sau:

```
<A HREF= "#marker"> Topic name </A>
```

Dấu # ở trước tên của siêu liên kết để báo cho trình duyệt biết rằng liên kết này liên kết đến một điểm được đặt tên trong tài liệu. Khi không có tài liệu nào được chỉ ra trước ký tự #, trình duyệt hiểu rằng liên kết này nằm trong cùng tài liệu.

Liên kết đến một điểm xác định ở một tài liệu khác

Bây giờ chúng ta đã biết cách sử dụng các vạch dấu trong cùng một tài liệu, hãy thử "nhảy" đến một vị trí trên một tài liệu khác. Để "nhảy" đến một điểm trên tài liệu khác, chúng ta cần phải chỉ ra tên của tài liệu khi chúng ta tạo vạch dấu. Trước tiến trình duyệt sẽ đọc tên tài liệu và mở tài liệu đó. Sau đó nó sẽ đọc vạch dấu và di chuyển đến điểm được đánh dấu.

Ví dụ: `<A HREF=Chapter18.html#Internet>Internet</A>`

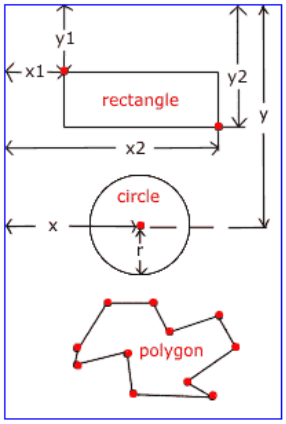
ImageMap

Trên một số trang web, bạn thấy một bức tranh mà khi gõ chuột vào mỗi góc có thể tới một trang mới. Ở đó người ta sử dụng một bức tranh, một tấm ảnh để làm liên kết. Để định vị từng khu vực trên bức ảnh, người ta sử dụng tọa độ của từng khu vực

PTIT

đó. Bạn thử Click vào trán, miệng, mũi, cổ của Hillary Clinton để xem ví dụ. Cái đó gọi là ImageMap, sử dụng mã sau:

```
<img src="h_clinton.gif" width="117" height="158"
border="0" usemap="#CongPhuong">
 <map name="CongPhuong">
    <area href="link_1.htm" alt="Trang web 1"
      shape="rect" coords="29,24,82,52">
    <area href="link_2.htm" alt="Trang web 2"
      shape="rect" coords="25,58,82,84">
    <area href="link_3.htm" alt="Trang web 3"
      shape="rect" coords="25,82,81,103">
    <area href="link_4.htm" alt="Trang web 4"
      shape="rect" coords="33,106,84,140">
 </map>
```



Trong đó bạn đặt một bức ảnh là MAP bằng USERMAP: usemap = "#CongPhuong", chia phần trên map bằng AREA, quy định tọa độ của từng phần trên map bằng COORDS: coords="29,24,82,52", quy định liên kết của từng phần bằng thẻ quen thuộc HREF, định hình cho từng phần bằng SHAPE: shape="rect". SHAP còn có thể mang giá trị circle (tròn), poly (polygon = đa giác). Bạn có thể sử dụng COORDS như sau:

- Nếu shape = "rect": coords="x_1,y_1,x_2,y_2", trong đó x_1,y_1 là tọa độ góc trên bên trái và x_2,y_2 là tọa độ của góc dưới bên phải của hình chữ nhật tính từ góc trên bên trái của bức ảnh.
- Nếu shape = "circle": coords="x, y, r", trong đó x, y là tọa độ của tâm hình tròn tính từ góc trên bên trái của bức ảnh và r là bán kính hình tròn.
- Nếu shape = "poly": cords = "x_1, y_1, x_2, y_2, ..., x_n, y_n", trong đó x_n, y_n là tọa độ của từng góc của đa giác tính từ góc trên bên trái của bức ảnh.

Để tính được tọa độ của từng miền, bạn dùng một phần mềm xử lý ảnh như Corel PhotoPaint 10, PaintShop Pro 7... hoặc phải đoán và thử dựa vào kích cỡ của bức ảnh.

Sử dụng e-mail: Nếu muốn người sử dụng gửi được e-mail, chúng ta có thể đưa một đặc tính vào trong trang Web và cho phép họ gửi e-mail từ trình duyệt. Tất cả những gì chúng ta cần làm là chèn giá trị mailto vào trong thẻ liên kết.

Ví dụ: `<A HREF="mailto:thisperson@mymail.com">`
`<A HREF="#"> (Về đầu trang)`

2. CÁC THẺ HTML CƠ BẢN

2.1 Tiêu đề (Heading)

Phần tử này được sử dụng để cung cấp tiêu đề cho phần nội dung hiển thị trên trang web. Tất cả những phần tử tiêu đề phải có thẻ kết thúc. Nó bắt đầu bằng thẻ

PTIT

<H1> và kết thúc bằng </H1>. Những phần tiêu đề được hiển thị to và in đậm hơn để phân biệt chúng với các phần còn lại của văn bản. Chúng ta cũng có thể hiển thị phần tiêu đề một trong sáu kích thước từ H1 đến H6. Tất cả những gì chúng ta làm là định rõ kích thước H1, H2 …Trình duyệt chú trọng đến cách hiển thị.

Ví dụ:

```
<HTML>
<HEAD>
<TITLE>Introduction to HTML</TITLE>
</HEAD>
      <H1>Introduction to HTML</H1>
      <H2>Introduction to HTML</H2>
      <H3>Introduction to HTML</H3>
      <H4>Introduction to HTML</H4>
      <H5>Introduction to HTML</H5>
      <H6>Introduction to HTML</H6>
</HTML>
```



Hình 1.3: Các thẻ heading

2.2 Thẻ khối <SPAN>, <DIV>

Có những trường hợp chúng ta muốn chia văn bản trong một trang web thành những khối thông tin logic. Chúng ta cũng có thể áp dụng những thuộc tính thông thường cho toàn bộ khối. Phần tử DIV và SPAN được sử dụng để nhóm nội dung lại với nhau. Phần tử DIV dùng để chia tài liệu thành các thành phần có liên quan với nhau. Phần tử SPAN dùng để chỉ một khoảng các ký tự. Phần tử SPAN dùng để định nghĩa nội dung trên dòng (in-line) còn phần tử DIV dùng để định nghĩa nội dung mức khối (block-level). Ví dụ:

```
<DIV>
Division 1
    <P> The DIV element is used to group elements
    <P>Typically, DIV is used for block level elements
</DIV>
<DIV align = right>
    Division 2
    <P>This is a second division  <BR>
    <H2>Are you having fun?</H2>
</DIV>
    <P>The second division is right aligned.
<SPAN STYLE = font-size:25; Color:blue>Common formatting
</SPAN>is applied to all the elements in the division
```

2.3 Kẻ đường ngang

Thẻ <HR> (horizontal rule) được dùng để kẻ một đường ngang trên trang. Những thuộc tính sau giúp điều khiển các đường nằm ngang. Nó chỉ có thẻ bắt đầu, không có thẻ kết thúc và không có nội dung.

PTIT

Bảng 1.3: Các thuộc tính của thẻ <HR>

Thuộc tính	Mô tả
Align	Chỉ định vị trí của đường nằm ngang. Chúng ta có thể canh lề center (giữa) |right (phải) |left (trái). Ví dụ align=”center”
Width	Chỉ độ dài của đường thẳng. Nó có thể xác định bằng các pixel hoặc tính theo phần trăm. Mặc định là 100%, nghĩa là toàn bộ bề ngang của tài liệu.
Size	Chỉ độ dày của đường thẳng và được xác định bằng các pixel.
Noshade	Chỉ đường được hiển thị bằng màu đặc thay vì có bóng.

Ví dụ: `<HR noshade size = “5” align = “center” width = “50%”>`

2.4 Thẻ font

Thẻ <FONT> dùng để điều khiển sự hiển thị văn bản trên trang web. Chúng ta cũng có thể chỉ định các thuộc tính như kích thước, màu sắc, kiểu… Chúng ta có thể đặt các thuộc tính <FONT> cho cả tài liệu bằng cách đặt phần tử vào bên trong thẻ <BODY>. Ngoài ra, thuộc tính FONT có thể đặt cho từng từ, từng phần và từng phần tử trong trang.

Bảng 1.4: Các thuộc tính của thẻ font

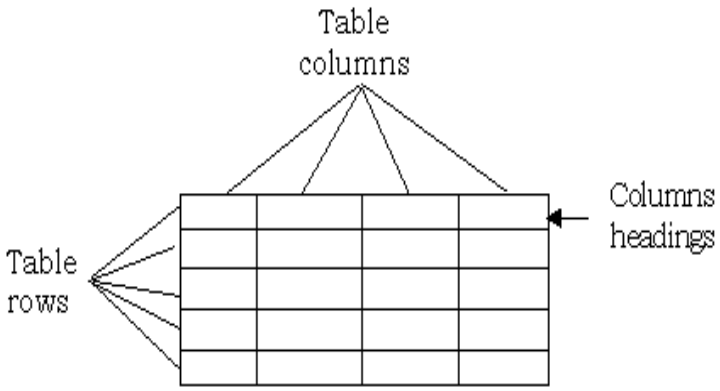
Thuộc tính	Mô tả
Color	Được dùng để chỉ màu của font. Chúng ta có thể dùng tên màu hoặc giá trị thập phân để xác định màu.
Size	Được dùng để chỉ kích thước của font. Chúng ta có thể xác định các kích thước FONT từ 1 cho đến 7. Kích thước lớn nhất là 7 và nhỏ nhất là 1. Chúng ta có thể dùng một kích thước chuẩn và chỉ ra những kích thước tiếp theo liên quan đến kích thước chuẩn. Ví dụ, nếu kích thước chính là 3, thì SIZE=+4 sẽ tăng lên 7, SIZE=-1 sẽ giảm xuống 2
Face	Được dùng để chỉ định kiểu font (phông chữ)

Ví dụ: `<H3><FONT SIZE = “3” COLOR = “HOTPINK” FACE = “Arial”>My first HTML document</FONT></H3>`

2.5 Bảng (table)

Chúng ta có thể sử dụng bảng để hiển thị dữ liệu dưới dạng các hàng và các cột. Bảng giúp cho chúng ta điều khiển, xác định và sắp xếp vị trí của văn bản và hình ảnh trên trang web, thay vì giao tất cả các việc đó cho trình duyệt.

BTIT



Hình 1.4: Mô hình của bảng

Thẻ <TABLE> được dùng để tạo bảng trong tài liệu HTML. Các thuộc tính của phần tử <TABLE> được áp dụng cho bảng, nhưng không cho dữ liệu hiển thị trên bảng. Đơn vị cơ bản của bảng là một ô và được định nghĩa bằng thẻ <TD>.

Các ô tạo thành một hàng. Các hàng tạo thành bảng. Điều này được nói đến trong cú pháp của HTML được sử dụng để tạo bảng. Thẻ <TD> được lồng trong thẻ <TR>. Thẻ <TR> được nằm trong cặp thẻ đóng và mở <TABLE>. Thuộc tính BORDER có thể được sử dụng để định nghĩa các ô và cấu trúc của bảng. Thuộc tính này chỉ độ rộng của đường viền. Nếu giá trị được đặt là "0" thì đường viền sẽ không hiển thị.

Mỗi cột trong bảng, phần tiêu đề có thể định nghĩa. Thẻ <TH> định nghĩa tiêu đề cho cột. Ví dụ: <TH> Employee Name </TH>

Phần tử <CAPTION> được dùng để thêm vào một chú thích cho bảng, ở đây ta sử dụng nó để mô tả bảng.

Ví dụ: `<CAPTION> Creating a Table </CAPTION>`

<CAPTION> nằm ngay sau thẻ mở <TABLE>. Những thẻ <TD> và <TR> có thể được dùng để chèn theo thứ tự các hàng và các cột vào trong bảng.

Đôi khi chúng ta muốn nối các dòng và các cột vào trong một ô. Như vậy, chúng ta tạo một cột để kéo rộng ra cho hơn một dòng, hay tạo ra một dòng để kéo rộng ra cho hơn một cột. Thuộc tính COLSPAN và ROWSPAN được sử dụng để tạo ra những ô mà chúng có thể kéo rộng ra cho hơn một dòng hay cột. Thuộc tính COLSPAN được sử dụng với thẻ <TH>, trong khi đó thuộc tính ROWSPAN được sử dụng với thẻ <TD>.

Ví dụ:

```
<TABLE BORDER = 2 BGCOLOR = lavender CELLSPACING = 2
CELLPADDING = 6>
  <CAPTION>Creating a Table</CAPTION>
  <TR><TH></TH><TH></TH><TH>NUTS</TH>
     <TH>BOLTS</TH><TH>Hammers</TH></TR>
  <TR><TD ROWSPAN=3>Quarter 1</TD> <TD>Jan</TD><TD>2500</TD>
```

PTIT

```
        <TD>1000</TD> <TD>1240</TD></TR>
  <TR><TD>Feb</TD><TD>3000</TD><TD>2500</TD><TD>4000</TD></TR>
  <TR><TD>March</TD> <TD>3200</TD> <TD>1000</TD> <TD>2400</TD></TR>
</TABLE>
```

2.6 Danh sách (list)

Nếu đang quen sử dụng Word, chắc chúng ta sẽ hỏi, làm thế nào để tạo được một danh sách có thứ tự (numbering: 1, 2, 3, 4 ....) hoặc có các dấu chấm tròn (bullets) ở đầu từng dòng.

Các loại danh sách mà ta có thể chèn vào tài liệu HTML là:

- Danh sách không thứ tự
- Danh sách có thứ tự
- Danh sách định nghĩa

Một số thẻ sau: (ordered list) (unordered list) (list) và các thuộc tính của các thẻ đó: type="disc", type="square", type="circle". Hai danh sách trên được viết bằng HTML như sau:

```
<ol>
   <li>Mua rau tươi nhà bà Tám béo</li>
   <li>Qua chợ Mơ mua chả ở quầy chị Hảo vẩu cho rẻ</li>
   <li>4 lạng thịt băm trên chợ Cầu Giấy</li>
   <li>1 con gà làm sẵn ở chợ Vĩnh Hồ</li>
</ol>
<ul type="circle">
   <li>Ra vẻ thích ăn đồ đắt tiền</li>
   <li>Nhưng thực ra lại rất ki bo</li>
   <li type="disc">Không thương chồng</li>
   <li type="square">Lười như hủi ;-)</li>
</ul>
```

PTIT

2.7 Chèn đa phương tiện (Multimedia)

Một trong những nhân tố lớn nhất trong sự phát triển của web đó là sự tích hợp của đa phương tiện (multimedia) bên trong những tài liệu HTML.

Chèn ảnh động (.GIF): `<IMG SRC=”coffeecup.gif” ALT="Steaming Coffee">`

Chèn âm thanh: `<BGSOUND SRC=”path\sound filename” loop=”positive number/infinite”>`

Chèn video: `<EMBED SRC=”path\video file name” WIDTH=”width in pixels or percentage” HEIGHT=”height in pixels or percentage”>`

- Thẻ Marquee, một số các thuộc tính thường dùng của lệnh marquee: BGCOLOR = "color", tạo màu nền cho vùng hiện thị lệnh marquee (có thể dùng hình nền...).

- DIRECTION = "left/right/up/down": chọn hình thức chạy chữ (trái, phải, lên, xuống...)
- BEHAVIOR = "scroll/slide/alternate": thiết lập kiểu chạy chữ:
    - Scroll: di chuyển tuần tự.
    - Slide: di chuyện 1 thời gian, sau đó dừng lại.
    - Alternate: di chuyển từ trái sang phải sau đó dừng lại.
  - TITLE = "message": Một đoạn tin nhắn nhỏ được hiển thị khi người dùng rê chuột vào.
  - SCROLLMOUNT = ”number”: điều chỉnh tốc độ chạy
  - SCROLLDELAY = "number": chỉnh thời gian hiển thị (*tính bằng giây*)
  - LOOP = "number|-1|infinite": quản lý số lần lặp.
  - WIDTH = "number": chỉnh độ rộng của vùng chạy chữ. Có thể dùng đơn vị pixel (px) hoặc phần trăm (%).

  Ví dụ:

```
<marquee onmouseover="this.stop()" onmouseout="this.start()"
scrollamount="2" direction="down" width="100%" height="200" align="center">
Di chuyển xuống dưới,
dừng lại khi ta rê chuột vào,
chạy tiếp khi rê chuột ra
</marquee>
```

2.8 Biểu mẫu (form)

Form HTML là một phần của tài liệu, nó chứa các phần tử đặc biệt gọi là các điều khiển. Các điều khiển được sử dụng để nhập thông tin từ người dùng và cung cấp một số tương tác. Dữ liệu do người dùng nhập vào có thể được xác nhận hợp lệ nhờ các kịch bản phía máy khách (client-side scripts) và được chuyển đến máy chủ để xử lý thêm.

Việc sử dụng biểu mẫu trên World Wide Web là khá nhiều và liên tục tăng lên. Sau đây là một số cách sử dụng thông thường:

- Thu thập tên, địa chỉ, số điện thoại, địa chỉ e-mail và các thông tin khác để người dùng đăng ký cho một dịch vụ hay một sự kiện nào đó
- Thu thập thông tin dùng để đăng ký mua một mặt hàng nào đó, ví dụ, khi muốn mua một cuốn sách trên Internet, ta phải điền tên, địa chỉ gửi thư, phương thức thanh toán và các thông tin liên quan khác.
- Thu thập thông tin phản hồi về một web site. Hầu hết các site cung cấp một dịch vụ nào đó đều khuyến khích khách hàng gửi thông tin phản hồi. Ngoài việc xây dựng mối quan hệ với khách hàng, đây còn là một nguồn thông tin để trao đổi hoặc cải tiến dịch vụ.

PTIT

- Cung cấp công cụ tìm kiếm cho web site. Các trang cung cấp nhiều thông tin khác nhau thường cung cấp cho người dùng hộp tìm kiếm để cho phép họ tìm kiếm thông tin nhanh hơn.

Phần tử <FORM> được sử dụng để tạo một vùng trên trang như một biểu mẫu. Nó chỉ ra cách bố trí của biểu mẫu. Các thuộc tính bao gồm:

Bảng 1.5: Các thuộc tính của FORM

Thuộc tính	**Mô tả**
Accept	Thuộc tính này xác định danh sách các kiểu MIME được máy chủ nhận ra, trong đó có chứa kịch bản (script) để xử lý biểu mẫu. Cú pháp là ACCEPT = "Internet media type"
Action	Thuộc tính này xác định vị trí của script sẽ xử lý biểu mẫu hoàn chỉnh và đã được gửi đi Cú pháp là: ACTION = "URL"
Method	Thuộc tính này xác định phương thức dữ liệu được gửi đến máy chủ. Nó cũng xác định giao thức được sử dụng khi máy khách gửi dữ liệu lên cho máy chủ. Nếu giá trị là GET thì trình duyệt sẽ tạo một câu hỏi có chứa địa chỉ URL của trang, một dấu chấm hỏi và các giá trị do biểu mẫu tạo ra. Trình duyệt sẽ trả lại câu hỏi cho kịch bản được xác định trong URL để xử lý. Nếu giá trị là POST, thì dữ liệu trên biểu mẫu được gửi đến kịch bản xử lý như một khối dữ liệu. Người ta không sử dụng chuỗi câu hỏi. Cú pháp là METHOD = (GET | POST).

Khi tạo một biểu mẫu, ta có thể đặt các điều khiển lên biểu mẫu để nhận dữ liệu nhập vào từ người dùng. Các điều khiển này được sử dụng với phần tử <FORM>. Tuy nhiên, ta cũng có thể sử dụng chúng ở bên ngoài biểu mẫu để tạo các giao diện người dùng.

Phần tử <INPUT> xác định loại và sự xuất hiện của điều khiển trên biểu mẫu. Các thuộc tính của phần tử này là: TYPE, NAME, VALUE, SIZE, MAXLENGTH, CHECKED, SRC.

Trong đó thuộc tính TYPE gồm: TEXT [Default], CHECKBOX, RADIO, SUBMIT, RESET, IMAGE, BUTTON.

Ví dụ: `<INPUT TYPE=button VALUE="click" NAME="b1">`

```
<INPUT TYPE=text VALUE="" NAME="textbox" SIZE=20>
```

Phần tử TextArea (vùng văn bản):

```
<TEXTAREA NAME="text1" COLS=20 ROWS=5> </TEXTAREA>
```

Phần tử lựa chọn (Select):

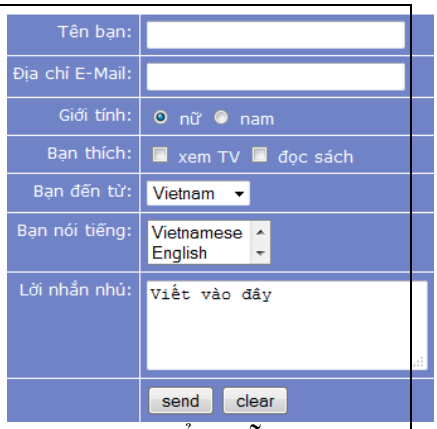
```
<P><B>How do you buy your stocks?</B>
<SELECT NAME="RESULT_RadioButton-5">
   <OPTION></OPTION>
   <OPTION>1) On-Line</OPTION>
   <OPTION>2) Touch Tone Trading</OPTION>
   <OPTION>3) Broker Assisted</OPTION>
```

Print

```
    <OPTION>4) Other</OPTION>
</SELECT></P>
```

Ví dụ 1 biểu mẫu liên hệ (form contact):

```
<form
method="post"action="mailto:webmaster@tridung
.de">
  Tên bạn:
  <input type="text" size="30" name="ten">
  E-Mail:
  <input type="text" size="30" name="e-mail">
  Giới tính:
  <input type="radio" value="f" name="sex"
checked> nữ
  <input type="radio" value="m" name="sex">
nam
  Bạn thích:
  <input type="checkbox" value="Xem TV"
name="TV"> xem TV <input type="checkbox"
value="Doc sach" name="Book"> đọc sách
  Bạn đến từ:
  <select name="land">
   <option value="VN">Vietnam
   <option value="USA"> USA
   <option value="De"> Germany</select>
  Bạn nói tiếng:
  <select name="language" size="2">
    <option value="Viet">Viet
    <option value="Engl"> Anh
    <option value="Deutsch"> Duc</select>
  Lời nhắn nhủ:
  <textarea cols="10" rows="3"
name="tin">Viết vào đây</textarea>
  <input type="submit" value="send">
  <input type="reset" value="clear">
</form>
```



Hình 1.5: Biểu mẫu liên hệ

PTIT

CHƯƠNG 2: BẢNG ĐỊNH DẠNG THEO TẦNG

1. KHÁI NIỆM

Bảng định dạng theo tầng (Cascading Style Sheets - CSS) là một ngôn ngữ quy định cách trình bày cho các tài liệu viết bằng HTML, XHTML, XML,… CSS được tạo nên từ các qui tắc kiểu cách, mà nó báo cho trình duyệt biết được cách trình bày một tài liệu. Nó là một kỹ thuật thêm vào các kiểu (font, màu, khoảng cách) cho những trang web. Thứ tự ưu tiên của CSS (ở đây theo độ ưu tiên giảm dần):

- Mặc định của trình duyệt
- CSS đặt ngoài trong 1 file
- CSS bên trong (trong thẻ <head>)
- CSS trên thẻ

Một kiểu trên thẻ là kiểu được gán trực tiếp cho một phần tử nào đó. Kiểu này không áp dụng vào tất cả các phần tử thuộc một loại nào hay một lớp nào đó. Kiểu trên thẻ được định nghĩa bằng thuộc tính STYLE đối với phần tử của thẻ đó. Ví dụ, nếu muốn đặt màu cho phần tử <H1> màu đỏ, chúng ta phải đặt thuộc tính STYLE bên trong thẻ H1 như sau: <H1 STYLE = ”color:red” >.

Lợi ích của các CSS:

Nạp chồng trình duyệt: Mỗi trình duyệt đều có thể hiển thị các trang web theo cách riêng của nó. Trước đây các nhà phát triển không kiểm soát được các trang web hiển thị trên trình duyệt. Suy cho cùng bạn không biết trình duyệt nào mà người dùng cách nửa vòng trái đất sử dụng. Nhờ có các css bạn có thể nạp chồng các qui ước của trình duyệt và đặt theo cách riêng của chúng ta. Chẳng hạn, bạn có thể xác định kiểu mà trong đó một phần tử <H1> cần hiển thị:

```
<H1><FONT SIZE=3 COLOR=AQUA> <B> Overriding the browser </B> </FONT></H1>
```

- Bố cục trang (Page layout): Những css có thể dùng để hiển thị font thay đổi màu mà không làm thay đổi cấu trúc của trang web. Điều này có nghĩa là với tư cách là một nhà thiết kế bây giờ ta có thể tách biệt những yêu cầu về thiết kế hình ảnh trực quan từ cấu trúc logic của trang web và địa chỉ là hai chuyện hoàn toàn khác nhau. Khi sử dụng các biện pháp liên quan trong css của bạn, bạn có thể thể hiện các tài liệu sao cho đẹp mắt trên bất kỳ màn hình nào và theo bất kỳ độ phân giải nào.
- Sử dụng lại các css: Một khi đã định nghĩa kiểu thông tin, chúng ta có thể nhúng css bên trong tài liệu HTML. Lần lượt thay thế, chúng ta có thể kết nối tất cả các trang trên website đến css. Điều này chắc chắn rằng các trang web của chúng ta đều có cùng diện mạo thông tin khi được hiển thị. Vì vậy, bạn có thể có được nền

PTIT

chung của trang ví dụ như logo của trang và một số thông tin chuẩn (cho các trang) trong một css. Điều này sẽ đảm bảo được cách nhìn và cảm nhận thông dụng về trang website. Cứ thử hình dung xem có vài trăm trang web và bạn phải xác định kiểu của mỗi trang một cách độc lập.

- Chỉ cần làm một lần thật tốt: Chúng ta có thể tạo một css và được liên kết đến nhiều tài liệu. Tất cả những tài liệu sẽ có diện mạo giống nhau. Tuy nhiên, quan trọng nhất là khi bạn thực hiện thay đổi thì tất cả các tài liệu được kết nối vào css sẽ bị thay đổi theo.

2. CÚ PHÁP

Cú pháp CSS cơ bản: Selector { property:value; }

Trong đó:

+ Selector: Các đối tượng mà chúng ta sẽ áp dụng các thuộc tính trình bày. Nó là các thẻ HTML, class hay id.

Ví dụ: body, h2, p, img, #title, #content, .username, …

Trong CSS ngoài viết tên selector theo tên thẻ, class, id. Chúng ta còn có thể viết tên selector theo phân cấp như để chỉ các ảnh ở trong #entry, chúng ta viết selector là #entry img, như vậy thì các thuộc tính chỉ định sẽ chỉ áp dụng riêng cho các ảnh nằm trong #entry. Khi viết tên cho class, đôi khi sẽ có nhiều thành phần có cùng class đó, ví dụ như thẻ img và thẻ a cùng có class tên vistors nhưng đây lại là hai đối tượng khác nhau, 1 cái là ảnh của người thăm, 1 cái là liên kết tới trang người thăm. Nên nếu khi viết CSS ta ghi là .visitors {width:50 } thì sẽ ảnh hưởng tới cả hai thành phần. Nên trong trường hợp này, nếu bạn có ý dùng CSS đó chỉ riêng phần ảnh thì chỉ nên ghi img .visitors thôi.

Một cách viết tên selector nữa đó là dựa trên tên các thuộc tính có trong HTML. Ví dụ trong HTML ta có đoạn mã :

```
<input name=”Search” type=”Text” value = ”Key  Word”>
```

Để áp dụng thuộc tính CSS cho riêng ô tìm kiếm này chúng ta sẽ dùng selector input [name = ”Search”]. Ngoài việc viết tên selector cụ thể, chúng ta cũng có thể dùng một selector đại diện như { color:red } sẽ tác động đến tất cả các thành phần có trên trang web làm cho chúng có text màu đỏ.

+ Property: Chính là các thuộc tính quy định cách trình bày. Ví dụ: backgroundcolor, font-family, color, padding, margin, … Mỗi thuộc tính CSS phải được gán một giá trị. Nếu có nhiều hơn một thuộc tính cho một selector thì chúng ta phải dùng một dấu; (chấm phẩy) để phân cách các thuộc tính. Tất cả các thuộc tính trong một selector sẽ được đặt trong một cặp ngoặc nhọn sau selector. Ví dụ:

PTIT

```
body { background:#FFF; color:#FF0000; font-size:14pt }
```

Để dễ đọc hơn, bạn nên viết mỗi thuộc tính CSS ở một dòng. Tuy nhiên, nó sẽ làm tăng dung lượng lưu trữ CSS. Ví dụ:

```
body {background:#FFF;
      bolor:#FF0000;
      font-size:14pt}
```

Đối với một trang web có nhiều thành phần có cùng một số thuộc tính, chúng ta có thể thực hiện gom gọn lại như sau:

```
h1 { color:#0000FF;
     text-transform:uppercase }
h2 {
     color:#0000FF;
     text-transform:uppercase; }
```

```
h3 {
  color:#0000FF;
  text-transform:uppercase; }
h1, h2, h3 {
  color:#0000FF;
  text-transform:uppercase; }
```

+ Value: Giá trị của thuộc tính. Ví dụ, trên value chính là #FFF dùng để định màu trắng cho nền trang. Đối với một giá trị có khoảng trắng, ta nên đặt tất cả trong một dấu ngoặc kép. Ví dụ: font-family: ”Times New Roman” Đối với các giá trị là đơn vị đo, không nên đặt một khoảng cách giữa số đo với đơn vị của nó.

Chú thích trong CSS:

Cũng như nhiều ngôn ngữ web khác. Trong CSS, chúng ta cũng có thể viết chú thích cho các đoạn code để dễ dàng tìm, sửa chữa trong những lần cập nhật sau. Chú thích trong CSS được viết như sau /* Nội dung chú thích */

```
/* Màu chữ cho trang web*/
body {color:red }
```

Vị trí đặt CSS:

Chúng ta có ba cách khác nhau để nhúng CSS vào trong một tài liệu HTML

+ Cách 1: Nội tuyến - trong thẻ (kiểu thuộc tính)

Đây là một phương pháp nguyên thủy nhất để nhúng CSS vào một tài liệu HTML bằng cách nhúng vào từng thẻ HTML muốn áp dụng. Trong trường hợp này chúng ta sẽ không cần selector trong cú pháp.

Lưu ý: Nếu chúng ta muốn áp dụng nhiều thuộc tính cho nhiều thẻ HTML khác nhau thì không nên dùng cách này.

Ở ví dụ sau chúng ta sẽ tiến hành định nền màu trắng cho trang và màu chữ xanh lá cho đoạn văn bản như sau:

```
<html> <head>
<title>Ví dụ</title> </head>
<body style=”background-color=#FFF;”>
   <p style=”color:green”>^_^ Welcome To WallPearl’s Blog ^_^</p>
</body> </html>
```

PTIT

+ Cách 2: Bên trong (thẻ style)

Tất cả các thuộc tính CSS vào trong thẻ style (để tiện cho công tác bảo trì, sửa chữa). Ví dụ làm trang web có màu nền trắng, đoạn văn bản chữ xanh lá:

```
<html> <head>
<title>Ví dụ</title>
<style type=”text/css”>
    body { background-color:#FFF }
    p { color:#00FF00 }
</style> </head>
<body>
    <p>^_^ Welcome To WallPearl’s Blog ^_^</p>
</body> </html>
```

Lưu ý: Thẻ style nên đặt trong thẻ head.

+ Cách 3: Bên ngoài (liên kết với một file CSS bên ngoài)

Tương tự như cách 2 nhưng thay vì đặt tất cả các mã CSS trong thẻ style chúng ta sẽ đưa chúng vào trong một file CSS (có phần mở rộng .css) bên ngoài và liên kết nó vào trang web bằng thuộc tính href trong thẻ link. Đây là cách làm được khuyến cáo, nó đặc biệt hữu ích cho việc đồng bộ hay bảo trì một website lớn sử dụng cùng một kiểu mẫu.

Ví dụ: `<link rel=”stylesheet” type=”text/css” href=”style.css” />`

Trong CSS chúng ta còn có thể sử dụng thuộc tính @import để nhập một file CSS vào CSS hiện hành. Cú pháp: @import url (liên kết).

```
<style type="text/css">
  @import url("style.css");
</style>
```

Background: định màu nền/ảnh nền cho một trang web cũng như các kỹ thuật định vị, điều chỉnh ảnh nền.

Thuộc tính background-color giúp định màu nền cho một thành phần trên trang web. Các giá trị mã màu của background-color cũng giống như color nhưng có thêm giá trị transparent để tạo nền trong suốt.

```
body { background-color:cyan}
h1 { background-color:red}
h2 { background-color:orange}
```

Việc sử dụng ảnh nền giúp trang web trông sinh động và bắt mắt hơn. Để chèn ảnh nền vào một thành phần trên trang web chúng ta sử dụng thuộc tính background-image.

```
body { background-image:url(logo.png)}
h1 { background-color:red}
h2 { background-color:orange}
p {background-color: FDC689}
```

PTIT

Theo mặc định ảnh nền khi được chèn sẽ nằm ở góc trên, bên trái màn hình. Tuy nhiên với thuộc tính background-position bạn sẽ có thể đặt ảnh nền ở bất cứ vị trí nào (trong không gian của thành phần mà nó làm nền). Có nhiều kiểu giá trị cho thuộc tính position. Như đơn vị chính xác như centimeters, pixels, inches, … hay các đơn vị qui đổi như %, hoặc các vị trí đặt biệt như top, bottom, left, right.

Khi sử dụng quá nhiều thuộc tính CSS sẽ gây khó khăn cho người đọc, công tác chỉnh sửa cũng như tốn nhiều dung lượng ổ cứng cho nên CSS đưa ra một cấu trúc rút gọn cho các thuộc tính cùng nhóm.

```
background-color:transparent;
background-image: url(logo.png);
background-repeat: no-repeat;
background-attachment: fixed;
background-position: right bottom;
thành một dòng ngắn gọn:
background:transparent url(logo.png) no-repeat fixed right bottom;
Khái quát cấu trúc rút gọn cho nhóm background:
background:<background-color> | <background-image> |
<background-repeat> |
<background-attachment> | <background-position>
```

Font Chữ:

Thuộc tính font-family có công dụng định nghĩa một danh sách ưu tiên các font sẽ được dùng để hiển thị một thành phần trang web. Theo đó, thì font đầu tiên được liệt kê trong danh sách sẽ được dùng để hiển thị trang web. Nếu như trên máy tính truy cập chưa cài đặt font này thì font thứ hai trong danh sách sẽ được ưu tiên…cho đến khi có một font phù hợp.

Có hai loại tên font được dùng để chỉ định trong font-family: family-names và generic families.

- Family-names: Tên cụ thể của một font. Ví dụ: Arial, Verdana, Tohama,…
- Generic families: Tên của một họ gồm nhiều font. Ví dụ: sans-serif, serif,…

Ví dụ chúng ta sẽ viết CSS để quy định font chữ dùng cho cả trang web là Times New Roman, Tohama, sans-serif, và font chữ dùng để hiển thị các tiêu đề h1, h2, h3 sẽ là Arial, Verdana và các font họ serif.

```
body { font-family:"Times New Roman",Tohama,sans-serif }
h1, h2, h3 { font-family:arial,verdana,serif }
```

Chú ý: Đối với các font có khoảng trắng trong tên như Times New Roman cần được đặt trong dấu ngoặc kép.

Thuộc tính font-style định nghĩa việc áp dụng các kiểu in thường (normal), in nghiêng (italic) hay xiên (oblique) lên các thành phần trang web.

PTIT

```
h1 { font-style:italic; }
h2 { font-style:oblique; }
```

Thuộc tính font-weight mô tả cách thức thể hiện của font chữ là ở dạng bình thường (normal) hay in đậm (bold). Ngoài ra, một số trình duyệt cũng hỗ trợ mô tả độ in đậm bằng các con số từ 100 – 900. Kích thước của một font được định bởi thuộc tính font-size. Tương tự như các thuộc tính background, chúng ta cũng có thể rút gọn các thuộc tính font lại thành một thuộc tính đơn như ví dụ sau:

```
h1 {
   font-style: italic;
   font-variant:small-caps;
   font-weight: bold;
   font-size: 35px;
   font-family: arial,verdana,sans-serif;}
Thành
h1 {font: italic bold 35px arial,verdana,sans-serif;}
Cấu trúc rút gọn cho các thuộc tính nhóm font:
Font :<font-style> | < font-variant> | <font-weight> | <font-
size> |< font-family>
```

Text:

Để định màu chữ cho một thành phần nào đó trên trang web chúng ta sử dụng thuộc tính color.

```
body { color:#000}
h1 { color:#0000FF }
h2 { color:#00FF00}
```

Thuộc tính text-indent cung cấp khả năng tạo ra khoảng thụt đầu dòng cho dòng đầu tiên trong đoạn văn bản. Giá trị thuộc tính này là các đơn vị đo cơ bản dùng trong CSS.

Ví dụ: `p {text-indent:30px}`

Thuộc tính text-align giúp bạn thêm các canh chỉnh văn bản cho các thành phần trong trang web. Thuộc tính này có tất cả 4 giá trị: left (canh trái – mặc định), right (canh phải), center (canh giữa) và justify (canh đều).

Ví dụ: `h1, h2 {text-align:right}`

Thuộc tính letter-spacing được dùng để định khoảng cách giữa các ký tự trong một đoạn văn bản. Ví dụ định khoảng cách giữa các ký tự trong thành phần h1, h2 là 7px:

`h1, h2 {letter-spacing:7px}`

Thuộc tính text-decoration giúp ta thêm các hiệu ứng gạch chân (underline), gạch xiên (line-through), gạch đầu (overline), và một hiệu ứng đặc biệt là văn bản nhấp nháy (blink).

Ví dụ : `h1 { text-decoration:underline}` (gạch chân).

PTIT

Text-transform là thuộc tính qui định chế độ in hoa hay in thường của văn bản mà không phụ thuộc vào văn bản gốc trên HTML. Thuộc tính này có tất cả 4 giá trị: uppercase (in hoa), lowercase (in thường), capitalize (in hoa ở ký tự đầu tiên trong mỗi từ) và none (không áp dụng hiệu ứng – mặc định).

Pseudo-classes cho liên kết:

Một thành phần rất quan trọng trong mọi website chính là liên kết. CSS cung cấp một điều khiển đặc biệt được gọi là pseudo-classes. Pseudo-classes cho phép bạn xác định các hiệu ứng định dạng cho một đối tượng liên kết ở một trạng thái xác định như khi liên kết chưa được thăm (a:link), khi rê chuột lên liên kết (a:hover), khi liên kết được thăm (a:visited) hay khi liên kết đang được kích hoạt – đang giữ nhấn chuột (a:active).

Ví dụ, chúng ta sẽ áp dụng 4 màu sắc khác nhau cho từng trạng thái liên kết, các liên kết chưa thăm có màu xanh lá, kích cỡ font 14px, có khung viền màu đen; các liên kết mouse over sẽ có màu đỏ tươi, kích cỡ font 1.2em, hiệu ứng nhấp nháy, nền light cyan; các liên kết đã thăm sẽ có màu đỏ, không có đường gạch chân và các liên kết đang kích hoạt có màu tím, font dạng small-caps, nền light yellow.

```
a {         border:1px solid #000;
            font-size:14px }
a:link {    color:#00FF00; }
a:hover{    background-color:#00BFF3;
            color:#FF00FF;
            font-size:1.2em;
            text-decoration:blink}
a:visited{background-color:#FFF568;
            color:#FF0000;
            text-decoration:none}
a:active{ color:#662D91;
            font-variant:small-caps }
```

DTIT

3. CLASS VÀ ID

Chúng ta học về cách sử dụng class và id để áp dụng các đặc tính đặc biệt cho một thành phần web. Trong đó:

- Class dùng để nhóm các đối tượng có cùng thuộc tính, do tính chất đó nó có thể được sử dụng nhiều lần.
- Id dùng để nhận dạng một đối tượng đặc trưng, id có tính duy nhất.

Class bắt đầu bằng ký tự '.', ví dụ class:

```
li .tp { color:FF0000 } //tp và tinh là tên của class
li .tinh { color:0000FF}
```

Lưu ý: Không nên đặt tên class với ký tự đầu là chữ số, nó sẽ không làm việc cho Firefox.

ID bắt đầu với ký tự '#', ví dụ:

```
#hanoi { color:# 790000 }
#hcmc { color:#FF0000 }
#danang { color:#FF00FF }
.tinh { color:#0000FF }
```

Lưu ý: Không nên đặt tên id với ký tự đầu là chữ số, nó sẽ không làm việc cho Firefox.

Thẻ <Span> và <Div>:

Thẻ trong HTML thật ra là một thẻ trung hòa, nó không thêm hay bớt bất cứ một thứ gì vào một tài liệu HTML cả. Nhưng chính nhờ tính chất trung hòa này mà nó lại là một công cụ đánh dấu tuyệt vời để qua đó chúng ta có thể viết CSS định dạng cho các phần tử mong muốn. Ví dụ:

```
.nhanmanh { font-weight:bold }
<p>Không có gì quý hơn
<span class="nhanmanh">độc lập</span>,
<span class="nhanmanh">tự do</span>.
```

Thẻ cũng là một thẻ trung hòa và được thêm vào tài liệu HTML với mục đính nhóm các phần tử lại cho mục đích định dạng bằng CSS. Điểm khác biệt là dùng để nhóm một khối phần tử trong khi đó có thể nhóm một hoặc nhiều khối phần tử.

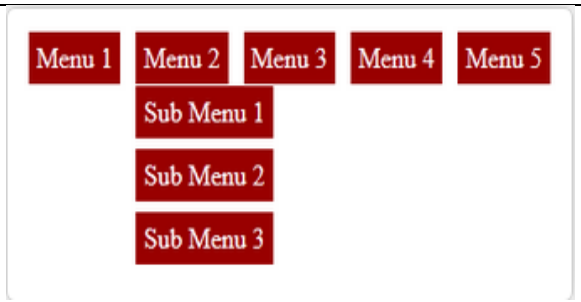
Ví dụ làm menu:

```
ul#menu li {
//bỏ các dấu chấm tròn
  list-style-type: none;
//menu dàn hàng ngang
  float: left;
//tạo lề cho các menu
  margin-right: 10px; }
ul#menu li a {
  text-decoration:none;
  background-color:#900;
  padding: 5px;
  color:white;}
ul#menu li ul#sub {
  display: none;}
ul#menu li:hover ul#sub {
  display: block;}
ul#menu li {
  positon: relative;}
ul#menu li ul#sub {
  display: none;
```

```
<ul id="menu">
  <li><a href="#">Menu 1  </a></li>
  <li><a href="#">Menu 2</a>
<ul id="sub">
  <li><a href="#">Sub Menu 1</a></li>
  <li><a href="#">Sub Menu 2</a></li>
  <li><a href="#">Sub Menu 3</a></li>
</ul>
</li>
  <li><a href="#">Menu 3</a></li>
  <li><a href="#">Menu 4</a></li>
  <li><a href="#">Menu 5</a></li>
</ul>
```

PTIT

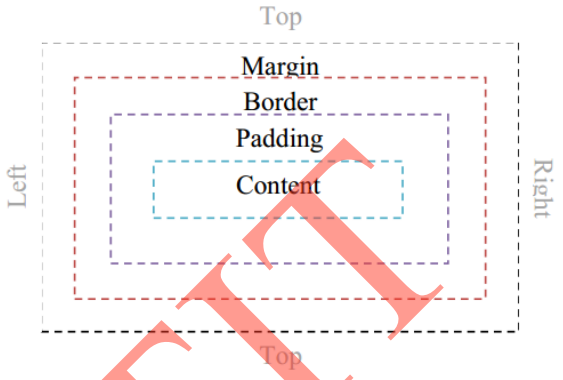
```
  position: absolute;
  width: 100px;}
ul#menu, ul#menu ul#sub {
  padding:0;
  margin: 0;}
```



Hình 2.1: Ví dụ menu

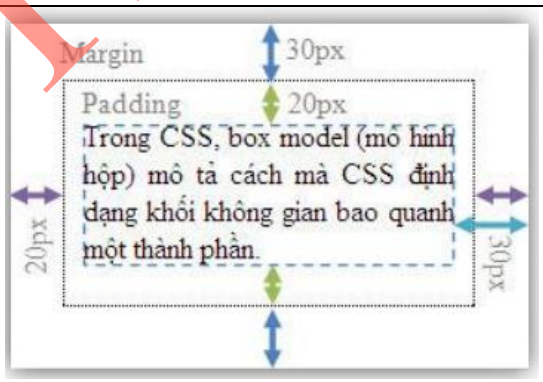
Mô hình hộp (Box Model):

Mô hình hộp mô tả cách mà CSS định dạng khối không gian bao quanh một thành phần. Nó bao gồm padding (vùng đệm), border (viền) và margin (canh lề) và các tùy chọn. Hình bên dưới mô tả cấu trúc minh họa mô hình hộp cho một thành phần web.



Hình 2.2: Mô hình hộp

```
p {
  width:200px;
  margin:30px 20px;
  padding:20px 10px;
  border:1px solid #000;
  text-align:justify
}
```



Hình 2.3: Căn chỉnh lề

Margin và padding của mô hình hộp:

```
body {
  margin-top:80px;
  margin-bottom:40px;
  margin-left:50px;
  margin-right:30px;
  border:1px dotted
     #FF0000}
//gọn hơn viết như sau:
body {
  margin:80px 30px 40px
```

```
Cú pháp :
margin:<margin-top> | <margin-right> |
<margin-bottom> | <margin-left>
Hoặc:
margin:<value1>|<  value2>
value 1 là giá trị  margin-top và margin-
bottom
value 2 là giá trị margin-left và margin-
right.
```

PTIT

50px; border:1px dotted #FF0000}	

Padding có thể hiểu như là một thuộc tính đệm. Padding không ảnh hưởng tới khoảng các giữa các các đối tượng như margin mà nó chỉ quy định khoảng cách giữa phần nội dung và viền của một đối tượng.

Cú pháp Padding: `<padding-top> | <padding-right> | <padding-bottom> | <padding-left>`

Border:

Nó thường được dùng trong trang trí, đóng khung cho một đối tượng cần nhấn mạnh, phân cách các đối tượng giúp trang web trông dễ nhìn hơn,…Border-width là một thuộc tính quy định độ rộng cho viền của một đối tượng web. Thuộc tính này có các giá trị: thin (mảnh), medium (vừa), thick (dày), hay là một giá trị đo cụ thể như pixels. Border-color là thuộc tính quy định màu viền cho một đối tượng web. Border-style là thuộc tính quy định kiểu viền thể hiện của một đối tượng web. CSS cung cấp tất cả 8 kiểu viền tương ứng với 8 giá trị: dotted, dashed, solid, double, groove, ridge, inset và outset. Ngoài ra, hai giá trị none hay hidden dùng để ẩn đường viền.

Cấu trúc rút gọn: `Border:<border-width> |<border-color> |<border-style>`

Float & Clear:

Float (theo như nghĩa tiếng Việt là thả trôi) là một thuộc tính CSS dùng để cố định một thành phần web về bên trái hay bên phải không gian bao quanh nó. Thuộc tính float có 3 giá trị:

- Left: Cố định phần tử về bên trái.
- Right: Cố định phần tử về bên phải.
- None: Bình thường.

Thuộc tính clear là một thuộc tính thường được gán vào các phần tử liên quan tới phần tử đã được float để quyết định hướng của phần tử này. Có tất cả 4 thuộc tính: left (tràn bên trái), right (tràn bên phải), both (không tràn) và none.

Định vị tuyệt đối (Absolute position) là sự định vị mà trong đó các thành phần được định vị không để lại bất cứ một khoảng trống nào trong tài liệu. Một thành phần được định vị tuyệt đối sẽ nhận giá trị position là absolute. Các đối tượng đã định vị tuyệt đối sẽ dùng kết hợp với các thuộc tính top, left, right, bottom để xác định tọa độ.

Định vị tương đối (Relative position) cho một thành phần là sự định vị được tính từ vị trí gốc trong tài liệu. Các thành phần đã được định vị tương đối sẽ để lại khoảng không trong tài liệu. Các thành phần được định vị tương đối sẽ nhận giá trị position là relative.

PTIT

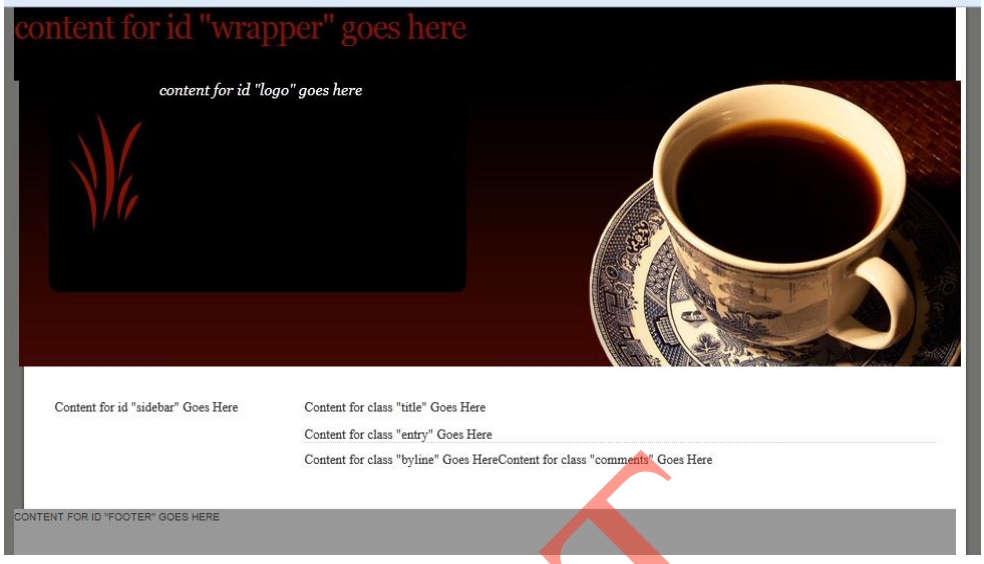
Layers:

Cách đặt các thành phần web ở các lớp khác nhau với thuộc tính z-index. Gán cho mỗi phần tử một con số, phần tử có số cao hơn sẽ nằm trên, phần tử có số thấp hơn sẽ nằm dưới. Ví dụ xây dựng template:

```
#wrapper {
      width: 960px;
      margin: 0 auto;
      overflow:hidden;
      background: url(images/img01.gif);}
#menu {
      float: left;
      width: 939px;
      height: 70px;
      background:  #000000;}
#logo {
      float: left;
      width: 939px;
      height: 273px;
      margin-left: 5px;
      background: url(images/img02.jpg) no-repeat left top;}
#page {
      width: 939px;
      margin: 0 auto;
      background:#FFF;}
#sidebar {
      float: left;
      width: 215px;
      padding: 30px 0 0 30px;}
#content {
      float: right;
      width: 630px;
      padding: 30px 30px 0 0;}
#footer {
      margin: 0 auto;
      float:left;
      width:939px;
      height: 80px;
      min-height: 64px;
      padding: 0;
      background:#999;
      text-transform: uppercase;
      font-family: Arial, Helvetica, sans-serif;
      font-size: 10px;}
<div id="wrapper">
  <div id="menu"><h1>Content for  id "wrapper" Goes Here</h1></div>
  <div id="logo"><h2>Content for  id "logo" Goes Here</h2></div>
  <div id="page">
    <div id="sidebar">Content for  id "sidebar" Goes Here</div>
    <div id="content">
      <div class="post">
```

PTIT

```
      <div class="title">Content for  class "title" Goes Here</div>
      <div class="entry">Content for  class "entry" Goes Here</div>
        <div class="meta">
     <div class="byline">Content for  class "byline" Goes Here</div>
 <div class="comments">Content for  class "comments" Goes Here</div>
        </div> </div> </div> </div>
  <div id="footer">Content for  id "footer" Goes Here</div> </div>
```



Hình 2.4: Template cho 1 trang web

PTIT

CASE STUDY 1 (HTML & CSS)

Xây dựng một trang web tĩnh (html - Template) gồm có các thành phần sau:

1. Thiết kế 1 website có các dạng (chỉ chọn 1):
   - Thương mại điện tử (bán sản phẩm trên mạng)
   - Blog (cho 1 chuyên mục cụ thể nào đó)
   - Mạng xã hội (ví dụ facebook, twitter...)
   - Dạng tùy chọn
2. Làm toàn bộ template cho site đó (ảnh, nút, nền...nói chung là đồ họa +css)
3. Mỗi bạn làm từ 2 pages trở lên (trong đó phải chứa thành phần form)
4. Được quyền tham chiếu các template trên mạng
5. Sẽ được 5-6 điểm nếu sử dụng css+ phần đồ họa sẵn có
6. 9 đ nếu chỉ "xem" của họ rồi tự làm lấy từ a-z
7. 1 đ nhóm (thiết kế bằng công cụ đồ họa)

CHƯƠNG 3: NGÔN NGỮ KỊCH BẢN JAVASCRIPT

1. GIỚI THIỆU

1.1 Giới thiệu về JavaScript

JavaScript là một ngôn ngữ kịch bản (scripting language) được dùng để tạo các kịch bản ở máy người dùng (client-side script) và máy chủ (server-side script). Các script ở máy client được thực thi tại trình duyệt, các script ở máy server được thực hiện trên server. Nó được phát triển bởi Sun Microsystems và Netscape. Nó được dùng để tạo các trang Web động và tương tác trên Internet. Đối với những người phát triển HTML, Javascript rất hữu ích trong việc xây dựng các hệ thống HTML có thể tương tác với người dùng. Mã Javascript có khả năng nhúng trong tài liệu HTML để điều khiển nội dung của trang Web và kiểm tra sự hợp lệ của dữ liệu mà người dùng nhập vào. Khi một trang hiển thị trong trình duyệt, các câu lệnh được trình duyệt thông dịch và thực thi.

Tính năng của JavaScript:

- Tương tác với người dùng. Các sự kiện phát sinh bởi người dùng như nhấp chuột hay được phát sinh từ hệ thống như thay đổi lại kích thước của trang …
- Thay đổi nội dung động.
- Kiểm tra tính hợp lệ dữ liệu khi người dùng nhập vào trước khi đẩy dữ liệu lên máy chủ.

PTIT

1.2 Chèn JavaScript vào trang web

Chúng ta có thể chèn các lệnh JavaScritp vào trong một tài liệu HTML theo những cách sau đây:

- Nhúng các câu lệnh trực tiếp vào trong tài liệu bằng cách sử dụng thẻ <SCRIPT>
- Liên kết file nguồn JavaScript với tài liệu HTML
- Đặt các biểu thức JavaScript làm giá trị cho thuộc tính của thẻ HTML.
- Dùng như trình xử lý sự kiện trong các thẻ HTML.

Dùng thẻ SCRIPT

Cú pháp như sau:

```
<script language="JavaScript">
<!--
JavaScript statements;     //-->
</script>
<! - -statements //- -> là các thẻ chú thích.
```

Các câu lệnh JavaScript phải được kết thúc bằng dấu chấm phẩy (;).

Chú ý: Về lý thuyết các câu lệnh JavaScript có thể đặt ở bất kỳ nơi nào trong tài liệu HTML. Tuy nhiên, nên đặt các câu lệnh script trong phần <HEAD> và </HEAD>. Điều này đảm bảo tất cả các câu lệnh đều được đọc và biên dịch trước khi nó được gọi từ trong phần BODY.

Dùng file bên ngoài

Cú pháp:

```
<script language="JavaScript" src="filename.js">
</script>
```

Đó là file văn bản chứa các mã lệnh JavaScript, tên file có phần mở rộng là ".js". Nó chỉ có thể chứa các câu lệnh và các hàm JavaScript. Ta không thể đưa các thẻ HTML vào trong nó.

Đặt các biểu thức JavaScript là giá trị cho các thuộc tính của thẻ HTML

Cú pháp: `& {expression};`

Trong đó expression là biểu thức JavaScript sẽ được thực hiện.

```
<HEAD>
<SCRIPT>
   var linewidth = 10;
</SCRIPT>
<BODY>
   <H2>Using entities</H2>
   <HR width="&{linewidth};%" align = "left">
</BODY>
```

Giá trị độ rộng được đặt là 10.

Dùng JavaScript trong trình xử lý sự kiện

Cú pháp là: `<TAG event handler="JavaScript code">`

TAG là một thẻ HTML. Event handler là tên của trình xử lý sự kiện, và JavaScript code là một loạt các câu lệnh JavaScript được thực thi khi sự kiện được kích hoạt.

Ví dụ:

```
<HTML>
<SCRIPT LANGUAGE =
"JavaScript">
function greeting () {
   alert ("Hi There !!");
}
</SCRIPT>
```

```
<BODY>
<FORM>
  <INPUT TYPE="button"
  VALUE="Greeting"onClick="greeting()">
</FORM>
</BODY>
</HTML>
```

Ví dụ:

PTIT

```
<HTML>
<HEAD>
<SCRIPT LANGUAGE =
"Javascript">
  confirm ("Are you Sure?");
  alert("OK");
  document.write(" Thank You
!");
</SCRIPT>
</HEAD> </HTML>
```



Hình 3.1: Dùng JavaScript – Confirm, Alert và Write

2. NỀN TẢNG VÀ CÚ PHÁP

2.1 Biến và dữ liệu

2.1.1 Biến

Các biến thường có các quy ước đặt tên như sau:

- Tên biến phải bắt đầu bằng một chữ cái hoặc ký tự gạch dưới ("_").
- Tên biến có thể chứa chữ số.

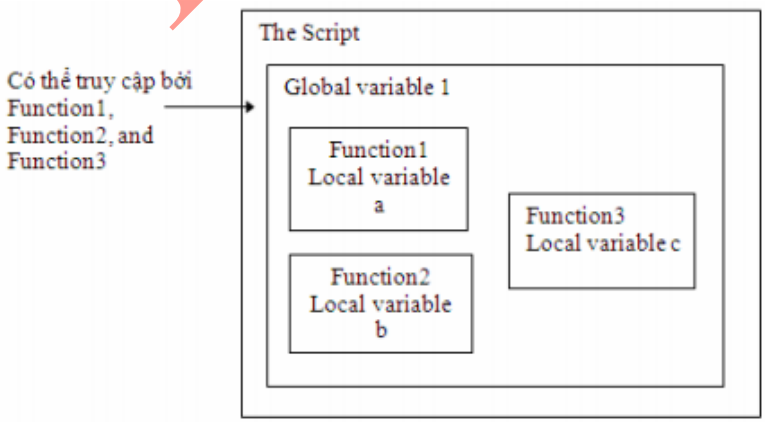
JavaScript phân biệt rõ chữ hoa và chữ thường, vì vậy tên biến bao gồm các ký tự từ"A" đến "Z" (chữ hoa) và các ký tự từ "a" đến "z" (chữ thường).

Khai báo biến

Sử dụng từ khóa 'var' để khai báo biến. Biến sẽ tồn tại ngay sau khi ta khai báo nó. Chúng ta có thể khởi tạo giá trị cho biến ngay khi khai báo.

Ví dụ: `var A = 10;  //hoặc bỏ var`

Khai báo nhiều biến trên cùng một dòng bằng cách tách tên các biến bằng dấu phẩy.



Hình 3.2: Phạm vi của biến (variables)

Chú ý: Các biến toàn cục không cần thiết phải sử dụng từ khóa var khi khai báo. Tuy nhiên, các biến cục bộ luôn luôn phải sử dụng từ khóa var khi khai báo.

2.1.2 Dữ liệu

PTIT

Bảng 3.1: Các loại dữ liệu trong Javascript

Kiểu dữ liệu	**Mô tả**
Numbers	Các số nguyên hoặc số thực. Ví dụ: 487 hoặc 25.95
Boolean	True hoặc False
String	“Hello World”
Null	Keyword đặc biệt chỉ giá trị null.

Ta viết:

```
var x = 7;
x = ”Chao mung”;       //Không lỗi.
A = “Gia tri: “ + 8;   //không lỗi
```

Hàm parseInt() và parseFloat() có chức năng chuyển đổi các chuỗi thành các số nguyên hoặc các số thực.

2.2 Các toán tử

Toán tử số học: + - * / % (phép chia lấy số dư) ++ -- và – (lấy số đối)

Toán tử so sánh: == (bằng), != (khác), >, >=, <, <=

Toán tử logic: && (and), || (or), ! (not)

Toán tử chuỗi: + (ghép 2 chuỗi)

Toán tử Evaluation: (condition) ? trueVal : falseVal

Kiểu toán tử	Các toán tử đơn
Gán	= += -= *= /= %= <<= >>= >>>= &= ^= |=
Điều kiện	?:
logic -or	||
logic -and	&&
bitwise-or	|
bitwise-xor	^
bitwise-and	&
bằng/không bằng	== !=
quan hệ	< <= > >=
dịch bit	<< >> >>>
cộng/trừ	+ -
nhân/chia	* / %
phủ định/tăng	! ~ - ++ -- typeof void

Hình 3.3: Mức ưu tiên của các toán tử

PTIT

Mảng

JavaScript không có kiểu dữ liệu mảng. Nhưng nó có một đối tượng mảng được xây dựng sẵn. Cú pháp tạo một mảng:

```
arrayObjectName = new Array([element0, element1, ..., elementN]);
```

Trong đó arrayObjectName là tên của đối tượng mảng, element là danh sách các giá trị của các phần tử mảng. Mảng được khởi tạo với các giá trị xác định như là các phần tử của nó. Thuộc tính length lưu số lượng các phần tử có trong mảng.

Ví dụ:

```
<script language="JavaScript">
      emp = new Array(3);
      emp[0] = "Ryan Dias";
      emp[1] = "Graham Browne";
      emp[2] = "David Greene";
      document.writeln(emp[0]);
      document.writeln(emp[1]);
      document.writeln(emp[2]);
</script>
```

Có hai cách để truy cập các phần tử của mảng. Chúng ta có thể:

- Chỉ ra tên của phần tử. Ví dụ, emp [“Ryan Dias”].
- Chỉ ra chỉ sốcủa phần tử. Ví dụ, emp [0].

Để thao tác trên mảng, chúng ta phải dùng các phương thức của đối tượng mảng.

Bảng 3.2: Các phương thức của đối tượng mảng

Phương thức	**Mô tả**
Join	Kết hợp các phần tử của mảng thành một chuỗi
Pop	Trả về phần tử cuối cùng của mảng, sau khi xoá nó từ mảng.
Push	Thêm một hoặc nhiều phần tử vào cuối mảng. Trả lại phần tử cuối cùng thêm vào.
Reverse	Đảo ngược các phần tử của mảng, phần tử đầu tiên của mảng trở thành phần tử cuối cùng và phần tử cuối cùng trở thành phần tử đầu tiên.
Shift	Xoá phần tử đầu tiên của mảng và trả về phần tử đó.
Sort	Sắp xếp các phần tử của mảng.

Chú ý: Một mảng có thể có nhiều hơn một chiều.

Ví dụ như sau không lỗi:

```
MyArray[0, 0] = "Ryan Dias";
MyArray[0, 1] = 1;
```

2.3 Biểu thức qui tắc

Biểu thức quy tắc (regular expression) là một tính năng đắc lực để kiểm tra, so sánh, thay thế, tách, ghép một chuỗi nào đó theo một quy tắc bạn quy định.

PTIT

Trong javaScript ta có 2 cách để khai báo biểu thức quy tắc:

- Khai báo với 2 dấu "/" ở đầu và cuối biểu thức
- Khai báo thông qua việc tạo đối tượng RegExp

Cách thứ nhất thường dùng nhiều hơn bởi sự đơn giản trong khai báo. Khi xử lý một chuỗi với biểu thức điều kiện một điều quan trọng là bạn muốn khi áp dụng ký tự có phân biệt chữ in hoa in thường hay không và áp dụng cho đoạn chuỗi đầu tiên tìm được hay tất cả các đoạn chuỗi hợp quy tắc, ta xét các qui tắc sau:

- "/quy tắc/" biểu thức có phân biệt chữ in hoa, in thường, và chỉ áp dụng cho đoạn chuỗi đầu tiên tìm thấy đúng quy tắc
- "/quy tắc/**g**" biểu thức có phân biệt hoa - thường và áp dụng cho tất cả đoạn chuỗi hợp quy tắc
- "/quy tắc/**i**" biểu thức không phân biệt hoa - thường, và chỉ áp dụng cho đoạn chuỗi đầu tiên tìm thấy đúng quy tắc
- "/quy tắc/**gi**" biểu thức không phân biệt hoa - thường, và áp dụng cho tất cả đoạn chuỗi hợp quy tắc

Quy tắc về vị trí:

- "^" nghĩa là bắt đầu một chuỗi
- "$" nghĩa là kết thúc một chuỗi
- "\b" nghĩa là điểm giữa một ký tự là từ và ký tự không phải là từ, ký tự là từ ở đây là ký tự từ A-z, 0-9
- "\B" ngược lại với '\b'

Quy tắc về những ký tự metasymbol:

- "\d" đại diện cho một ký tự số từ 0-9, tương đương với cách viết [0-9]
- "\D" đại diện cho một ký tự không phải là kiểu số từ 0-9, ngược lại với "\d"
- "\s" đại diện cho một ký tự là khoảng trắng
- "\S" đại diện cho một ký tự không phải là khoảng trắng, ngược lại với "\s"
- "\w" đại diện cho một ký tự từ A-Z hoặc a-z hoặc 0-9, hoặc ký tự gạch dưới "_"
- "\W" đại diện cho một ký tự không phải là từ A-Z, a-z, 0-9, ký tự gạch dưới "_"

Quy tắc về số lượng ký tự:

- "x?" nghĩa là một ký tự "x" hoặc không có ký tự nào
- "x*" nghĩa là một hoặc nhiều ký tự "x" hoặc không có ký tự nào
- "x+" nghĩa là có ít nhất một ký tự x
- "x{n}" nghĩa là ký tự x lặp lại n lần
- "x{n,}" nghĩa là ký tự x lặp lại ít nhất n lần
- "x{n,m}" nghĩa là ký tự x lặp lại từ n tới m lần

PTIT

Quy tắc về danh sách ký tự:

- "[...]" trong dấu ngoặc vuông liệt kê danh sách những ký tự hợp quy tắc, có thể dùng dấu gạch giữa "-"giữa 2 ký tự để thể hiện danh sách những ký tự trong khoảng 2 ký tự theo vị trí ASCII của ký tự
- "[^...]" chức năng thì ngược lại với "[...]" thay vì liệt kê những ký tự hợp quy tắc, nó cho biết bất kỳ ký tự nào ngoài danh sách các ký tự trong dấu ngoặc vuông

JavaScript xem một biểu thức quy tắc như một đối tượng. Vì vậy, chúng ta phải tạo một biểu thức quy tắc trước khi sử dụng chúng. Chúng ta có thể tạo một biểu thức quy tắc bằng một trong hai cách sau:

Sử dụng khởi tạo đối tượng (Object initializer).

Cú pháp của việc khởi tạo đối tượng:

```
Objectname = {expression};
```

Trong đó objectname là tên của đối tượng mới. Expression là một khuôn mẫu để tạo đối tượng.

```
re = /xy+z/;
```

Trong định nghĩa trên, một biểu thức quy tắc "xy+z" được tạo và gán cho đối tượng re. Sau đó dùng đối tượng này để tìm kiếm các mẫu theo yêu cầu. Khi chúng ta khởi tạo đối tượng, biểu thức quy tắc được dịch khi script được đánh giá. Nếu biểu thức quy tắc không thay đổi, sử dụng khởi tạo đối tượng thì hiệu quả.

Bảng 3.3: Một số phương thức dùng với biểu thức quy tắc.

Phương thức	**Mô tả**
Exec	Tìm kiếm một mẫu tương xứng trong một chuỗi. Nó trả về một mảng thông tin.
Test	Kiểm tra tương xứng trong một chuỗi. Trả về giá trị đúng hoặc sai
Match	Tìm kiếm tương xứng trong một chuỗi . Trả về một mảng thông tin hoặc giá trị null nếu sai.
Search	Kiểm tra sự tương xứng trong một chuỗi. Trả về giá trị chỉ số của tương xứng nếu tồn tại, -1 nếu bị sai
Replace	Tìm kiếm sự tương xứng trong một chuỗi, và thay thế chuỗi con tìm kiếm tương xứng bằng một chuỗi con thay thế khác.
Split	Dùng để tách một chuỗi thành một mảng các chuỗi con.

Chúng ta có thể dùng các cờ với biểu thức quy tắc. Hai cờ "g" và "i" được chọn tuỳ ý. Có thể dùng riêng hoặc dùng cả hai cờ.

```
re = /w+s/g; //Dùng cho tìm kiếm tổng thể
```

Đoạn mã dưới đây dùng để kiểm tra phương thức tìm kiếm một mẫu trong chuỗi. Nếu mẫu được tìm thấy, giá trị trả về "True" và ngược lại thì trả về "false".

PTIT

Ví dụ biểu thức qui tắc định dạng cho điện thoại:

```
var regex = /^\(?\d{3}\)?-?\s*\d{3}\s*-?\d{4}$/; //((456) 444-4444)
```

```
<html><head><title>Kiểm tra tính đúng đắn</title>
<script language="JavaScript">
function ok_Phone(phform){
  var regex = /^\(?\d{3}\)?-?\s*\d{3}\s*-?\d{4}$/;
  if(regex.test(phform.user_phone.value)){
       return true;
  }else{
       alert("Enter a valid phone number");
       return false;
  }  }
</script></head>
<body>
   <h2>Checking for a Valid Phone Number </h2>
   <form name="formtest" action="" method="post"
      onSubmit="return ok_Phone(this);">
    <p>Please enter your phone ((456) 444-4444): <BR>
       <input type="text" size=40 name="user_phone">
    <p><input type=submit value="Submit">
       <input type=reset value="Clear">
</form></body></html>
```

Ví dụ: định dạng cho email

```
<html><head><title>Validating E-Mail Addresses</title>
<script language="JavaScript">
function ok_Email(eform){
   var regex = /^(([\-\w]+)\.?)+@(([\-\w]+)\.?)+\.[a-zA-Z]{2,4}$/;
   if(regex.test(eform.user_email.value)){
      return true;
   }
   else{
      alert("Enter a valid email address");
      return false;
   }  }
</script>
</head><body>
    <h2> Checking for Valid Email Address </h2>
   <form name="formtest" action="" method="post" onSubmit="return
ok_Email(this);">
    <p>Please enter your email address: <BR>
       <input type="text" size=40 name="user_email">
    <p><input type=submit value="Send">
</body></html>
```

2.4 Tập lệnh

2.4.1 Các câu lệnh

Câu lệnh if . . . else

Cú pháp:

PTIT

`if (condition) { statements; }`	`if (condition) { statements 1; } else { statements 2; }`

Lệnh switch, cú pháp:

```
switch (expression){
case label1:
     statement1;
     break;
case label2:
    statement2;
    break;
    ...
default: statement; }
```

Vòng lặp "for", Cú pháp:

```
for (lệnh khởi tạo; điều kiện; lệnh tăng){
        statements; }
```

- Lệnh khởi tạo: được thực hiện duy nhất một lần và thường dùng để khởi tạo biến đếm.
- Lệnh điều kiện: điều kiện của vòng lặp.
- Lệnh tăng: thay đổi biến đếm trong vòng lặp.

Do . . . while, cú pháp:

```
do {
     statements;
   }while (condition);
```

Lệnh while, cú pháp :

```
while (condition) {
    statements; }
```

Câu lệnh break & continue

Lệnh break dùng để kết thúc việc thực thi của một câu lệnh. Khi được sử dụng trong một vòng lặp, lệnh break làm dừng ngay vòng lặp đó và không thực hiện thêm nữa. Continue dừng ngay lần lặp hiện tại và quay lại kiểm tra điều kiện để thực hiện lần lặp tiếp theo.

for...in

Câu lệnh for . . in được dùng để duyệt các thuộc tính của một đối tượng hay các phần tử của một mảng hoặc danh sách liệt kê.

Chú pháp:

```
for (variable in object) {
    statements; }
```

PTIT

with

Câu lệnh "with" được dùng để thực thi một tập các lệnh cùng tham chiếu đến một đối tượng xác định.

Cú pháp:

```
with (object) {
    statements; }
```

2.4.2 Hàm (function)

JavaScript có nhiều hàm định nghĩa trước mà chúng ta sẽ dùng trong script.

Hàm eval(): Hàm eval() được dùng để đánh giá một chuỗi và không cần tham chiếu đến bất kỳ một đối tượng cụ thể nào.

```
eval(string);
```

Chuỗi có thể là một biểu thức JavaScript, câu lệnh, hoặc một nhóm các câu lệnh. Trong biểu thức có thể bao gồm các biến và thuộc tính của một đối tượng. Nếu chuỗi được xác định là một biểu thức, hàm eval sẽ tính toán biểu thức. Nếu nó là một hoặc một tập các lệnh, nó sẽ thực thi các câu lệnh.

Hàm isNaN(): được dùng để kiểm tra xem đối số truyền vào có phải là một số hay không.

```
isNaN(testValue);
```

Tạo hàm do người dùng định nghĩa

Môt hàm JavaScript giống với thủ tục (procedure) hay chương trình con (subroutine) trong ngôn ngữ lập trình. Một hàm là một tập các câu lệnh, thực hiện các tác vụ xác định. Chúng ta có thể truyền các giá trị (tham số) cho hàm. Hàm cũng có thể trả về một giá trị.

Các thành phần của một hàm gồm:

- Từ khoá function.
- Tên hàm.
- Danh sách các đối số của hàm cách nhau bởi dấu phẩy, đặt trong cặp ngoặc đơn ().
- Các câu lệnh của hàm đặt trong cặp dấu móc {}.

```
function funcName(argument1,argument2,etc){
        statements; }
```

Một hàm có thể không có đối số, nhưng chúng ta vẫn phải có cặp dấu ngoặc đơn (): funcName().

3. CÁC ĐỐI TƯỢNG CƠ BẢN TRONG JAVASCRIPT

3.1 Các đối tượng cơ bản

PTIT

Một đối tượng là một gói dữ liệu toàn diện. Các thuộc tính (biến) dùng để định nghĩa đối tượng và các phương thức (hàm) tác động tới dữ liệu đều nằm trong đối tượng. Để truy cập các thuộc tính của đối tượng, chúng ta phải chỉ ra tên đối tượng và thuộc tính của nó:

```
objectName.propertyName
```

Ví dụ: `Document.bgcolor`

Trong đó: bgcolor (background color) là thuộc tính của đối tượng document. Để truy cập các phương thức của một đối tượng, chúng ta phải chỉ ra tên đối tượng và phương thức yêu cầu: objectName.method().

Câu lệnh This

Câu lệnh 'this' không chỉ là một thuộc tính nội tại. Giá trị của nó chỉ ra đối tượng hiện hành và có thể có các thuộc tính chuẩn chẳng hạn như tên, độ dài, và giá trị được áp dụng phù hợp. Câu lệnh 'this' chỉ được dùng trong phạm vi của một hàm hay các tham chiếu khi gọi hàm. Ví dụ: this.property

Đối tượng String

Đối tượng String được dùng để thao tác và làm việc với chuỗi văn bản. Chúng ta có thể tách nó ra thành các chuỗi con và biến đổi chuỗi đó thành các chuỗi hoa hoặc thường trong một chương trình. Cú pháp tổng quát là:

```
stringName.propertyName
Hoặc    stringName.methodName
```

Có 3 phương thức khác nhau để tạo ra chuỗi. Dùng lệnh var và gán cho nó một giá trị. Ví dụ như:

```
var newstr = " This is my script";
```

Dùng một toán tử (=) có gán với một tên biến. Ví dụ:

```
newstr = "This is my script";
```

Dùng hàm khởi tạo String (string). Ví dụ:

```
var  newstr = String ("This is my script")
```

Bảng 3.4: Một số phương thức làm việc với đối tượng String

Phương thức	**Ví dụ**	**Kết quả**
charAt( )	var str = "Hello world!"; document.write(str.charAt(1));	e
blink( )	var str = "Hello world!"; document.write(str.blink());	
bold( )	var str="Hello world!"; document.write(str.bold());	
charCodeAt()	var str="Hello world!"; document.write(str.charCodeAt(1));	101

PTIT

Concat()	var str1="Hello "; var str2="world!"; document.write(str1.concat(str2));	Hello world!
fontcolor( )	var str="Hello world!"; document.write(str.fontcolor("Red"));	
fontsize( )	var str="Hello world!"; document.write(str.fontsize(7));	
italics( )	var str="Hello world!"; document.write(str.italics());	
length	var txt="Hello World!"; document.write(txt.length);	12
match( )	var str="Hello world!"; document.write(str.match("world") + " "); document.write(str.match("World") + " "); document.write(str.match("worlld") + " "); document.write(str.match("world!"));	world null null world!
replace( )	var str="Visit Microsoft!"; document.write(str.replace(/Microsoft/, "W3Schools"));	Visit W3Schools!
search( )	var str="Visit W3Schools!"; document.write(str.search(/W3Schools/));	6 // -1
substring( )	var str="Hello world!"; document.write(str.substring(3));	lo world!
toLowerCase( )	var str="Hello World!"; document.write(str.toLowerCase());	
toUpperCase( )	var str="Hello world!"; document.write(str.toUpperCase());	

Đối tượng Math

Bảng 3.5: một số phương thức làm việc với đối tượng Math

Phương thức	**Ví dụ**	**Kết quả**
abs(a)	document.write(Math.abs(7.25) + " "); document.write(Math.abs(-7.25) + " ");	7.25 7.25
exp(a)	document.write(Math.exp(1) + " ");	2.718281828459
max(a,b)	document.write(Math.max(5,7) + " ");	7
min(a,b)	document.write(Math.min(5,7) + " ");	5
pow(a,b)	document.write(Math.pow(2,4) + " ");	16
round(a)	document.write(Math.round(-4.40) + " ");	-4
sqrt(a)	document.write(Math.sqrt(0.64) + " "); document.write(Math.sqrt(-9));	0.8 NaN

Đối tượng Date

PTIT

Date là một đối tượng có sẵn chứa thông tin về ngày và giờ. Đối tượng Date không có thuộc tính nào. Tuy nhiên, nó có nhiều phương thức dùng để thiết lập, lấy và xử lý các thông tin về thời gian.

Đối tượng Date lưu trữ thời gian theo số mili giây tính từ 1/1/1970 00:00:00.

Tạo một đối tượng Date:

```
DateObject = new Date(parameters);
```

Trong đó: DateObject là một biến lưu trữ một đối tượng thời gian mới.

Parameters có thể là:

- Không có gì: nếu không có tham số, nó sẽ trả về thời gian hiện tại của hệ thống. Ví dụ: today = new Date().
- String (chuỗi): biểu diễn ngày giờ theo dạng: "MM DD, YYYY, hh:mm:ss". Ví dụ: tdate = new Date (" July 29, 1998,10:30:00")

Bảng 3.6: Một số phương thức của đối tượng Date

Phương thức	**Ví dụ**	**Kết quả**
Date( )	document.write(Date());	Tue Nov 18 12:16:34 2008
getDate( )	var d = new Date(); document.write(d.getDate());	18
getDay( )	var d=new Date(); document.write(d.getDay());	2 // Sunday : 0
getMonth( )	var d=new Date(); document.write(d.getMonth());	10 // 0-11
getFullYear( )	var d = new Date(); document.write(d.getFullYear());	2008
getHours( )	var born = new Date("July 21, 1983 01:15:00"); document.write(born.getHours());	1
getMinutes( )	var born = new Date("July 21, 1983 01:15:00"); document.write(born.getMinutes());	15
getSeconds( )	var Birthday = new Date("July 21, 1983 01:15:00"); document.write(Birthday.getSeconds());	0
setDate( )	var d = new Date(); d.setDate(15);	
setMonth( )	var d=new Date(); d.setMonth(0);	
setFullYear( )	var d = new Date(); d.setFullYear(1992);	
setHours( )	var d = new Date(); d.setHours(15);	

PTIT

setMinutes( )	var d = new Date(); d.setMinutes(1);	
setSeconds( )	var d = new Date(); d.setSeconds(1);	
toString( )	var d = new Date(); document.write (d.toString());	Tue Nov 18 12:40:10 PST 2008

Ví dụ hiển thị đồng hồ

```
<HEAD>
<SCRIPT language="JavaScript">
  function disptime() {
      var time = new Date();
      var hour = time.getHours();
      var minute = time.getMinutes();
      var second = time.getSeconds();
      var temp = "" + ((hour > 12) ? hour - 12 : hour);
      temp += ((minute < 10) ? ":0" : ":") + minute;
      temp += ((second < 10) ? ":0" : ":") + second;
      temp += (hour >= 12) ? " P.M." : " A.M.";
      document.MyPage.digits.value = temp;
      id = setTimeout("disptime()",1000);
  }
</SCRIPT>
</HEAD> <BODY onLoad="disptime()">
   <P> The time is displayed on the page.
   <BR><BR><BR>
<FORM NAME="MyPage">
   <INPUT TYPE="text" NAME="digits" SIZE=12 VALUE="">
</FORM> </BODY>
```

PTIT

3.2 Đối tượng event

Các chương trình JavaScript thường là hướng sự kiện. Các sự kiện là các hành động xảy ra trên trang web. Một sự kiện có thể do người dùng tạo ra – click chuột vào một button - hoặc do hệ thống tạo ra – thay đổi kích thước của trang. Hầu hết các trình duyệt đều hỗ trợ một đối tượng Event. Mỗi sự kiện có một đối tượng Event tương ứng. Đối tượng Event cung cấp thông tin về sự kiện - loại sự kiện và vị trí của con trỏ tại thời điểm xảy ra sự kiện. Khi một sự kiện được phát sinh, nó được gửi đi như một đối số đến trình xử lý sự kiện. Dĩ nhiên, phải có một trình xử lý sự kiện trong trường hợp này.

Đối tượng Event chứa những thông tin sau:

- Loại sự kiện: trong trường hợp này là nhấp chuột
- Vị trí x và y của con trỏ khi nhấp chuột
- Nút chuột nào được nhấn

- Các phím bổ trợ (Control, Alt, Meta, hoặc Shift) được nhấn vào thời điểm xảy ra sự kiện.

Đối tượng Event không thể được sử dụng trực tiếp với đối tượng window. Nó được sử dụng như một phần của trình xử lý sự kiện.

Một sự kiện bắt đầu bằng hành động hoặc điều kiện khởi tạo sự kiện và kết thúc bằng việc đáp lại của trình xử lý sự kiện. Vòng đời của một sự kiện thông thường gồm những bước sau:

- Hành động người dùng hoặc điều kiện tương ứng với sự kiện xảy ra.
- Đối tượng Event được cập nhật tức thì nhằm phản ánh trạng thái của sự kiện.
- Sự kiện được kích hoạt.
- Trình xử lý sự kiện tương ứng được gọi.
- Trình xử lý sự kiện thực hiện hành động của nó và trả về điều khiển cho chương trình.

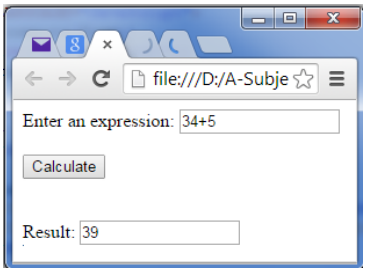
3.3 Các sự kiện

Tập hợp các sự kiện tương ứng với các phần tử khác nhau trên trang là một phần của mô hình đối tượng tài liệu (Document Object Model), chứ không phải của JavaScript. Nghĩa là, các sự kiện được một phần tử nào đó hỗ trợ có thể khác nhau trên các trình duyệt.

Một số sự kiện thường được hầu hết các đối tượng hỗ trợ: onClick (nhấn chuột), onChange (phần tử thay đổi), onFocus (phần tử được lựa chọn), onBlur (phần tử mất lựa chọn), onMouseOver (con chuột di chuyển lên trên phần tử), onMouseOut (con chuột di chuyển ra khỏi phần tử đó), onLoad (áp dụng cho đối tượng body - được phát sinh khi đã tải xong tài liệu.), onSubmit (thường sử dụng nút Submit), onMouseDown (kích hoạt khi hành động nhấp chuột xảy ra), onMouseUp (Sự kiện này được kích hoạt khi hành động nhả chuột xảy ra) …

onClick: sự kiện onClick được tạo ra bất cứ khi nào người dùng nhấp chuột lên các phần tử form nào đó (button, checkbox, radio button, và phần tử select..), hoặc lên các hyperlink. Ví dụ:

```
<HEAD>
<SCRIPT LANGUAGE="JavaScript">
function compute(form){
if (confirm("Are you sure?"))
   form.kết quả.value = eval(form.expr.value)
else
   alert("Please come back again.")}
</SCRIPT>
</HEAD> <BODY>
<FORM>
```



Hình 3.4: ví dụ với sự kiện onClick

PTIT

```
   Enter an expression:
  <INPUT TYPE="text" NAME="expr" SIZE=15 >
  <BR><BR>
  <INPUT TYPE="button" VALUE="Calculate"
     ONCLICK="compute(this.form)">
<BR><BR><BR>
  Kết quả:
  <INPUT TYPE="text" NAME="kết quả" SIZE=15>
<BR>
</FORM></BODY>
```

onChange: sự kiện onChange xảy ra khi một phần tử form thay đổi. Điều này có thể xảy ra khi nội dung của một trường văn bản thay đổi, hoặc khi một chọn lựa trong danh sách chọn lựa thay đổi. Tuy nhiên, sự kiện onChange không được tạo ra khi một radio button hoặc một checkbox được nhấp. Thay vào đó, sự kiện onClick sẽ được tạo ra. Sự kiện onChange được gửi đi khi một phần tử hoàn tất việc thay đổi. Vì vậy, khi một textbox đang được hiệu chỉnh, sự kiện onChange chỉ được phát sinh sau khi việc hiệu chỉnh đã hoàn tất, và khi người dùng thoát khỏi textbox đó.

onFocus: sự kiện onFocus được gửi đi bất cứ khi nào một phần tử form trở thành phần tử hiện thời. Chỉ khi một phần tử nhận được focus nó mới nhận được input từ người dùng. Điều này có thể xảy ra khi người dùng nhấp chuột lên phần tử, hoặc sử dụng phím Tab hoặc Shift+Tab (di chuyển tới các phần tử trên form).

onBlur: Blur ngược với focus. Khi người dùng rời khỏi một phần tử trên form, sự kiện onBlur được kích hoạt. Đối với một số phần tử, nếu nội dung của nó cũng bị thay đổi, thì sự kiện onChange cũng được kích hoạt.

Ví dụ:

```
<BODY BGCOLOR="lavender">
<FORM>
<INPUT type = text name = text1
  onblur="(document.bgColor='aqua')"
  onfocus="(document.bgColor='dimgray')">
</FORM>
</BODY>
```

onMouseOver: Sự kiện onMouseOver được tạo ra bất cứ khi nào con trỏ chuột di chuyển lên trên một phần tử.

onMouseOut: Sự kiện onMouseOut được tạo ra bất cứ khi nào con trỏ chuột di chuyển ra khỏi phần tử đó. Ví dụ:

```
<head>
<script language = "javascript">
  var num =0
function showLink(num){
  if (num==1) {
     document.forms[0].elements[0].value= "You have selected PTIT";}
```

PTIT

```
  if (num==2){
    document.forms[0].elements[0].value = "You have selected CNTT";}
  if (num==3) {
    document.forms[0].elements[0].value = "You have selected ME";}
}
</script>
</head> <body>
<form> <input type=text size=60 > </form>
<a href="#" onMouseOver="showLink(1)"document.bgcolor= "
green">PTIT</a><br>
<a href="#" onMouseOver="showLink(2)">CNTT</a><br>
<a href="#" onMouseOver="showLink(3)">ME</a><br>
</body>
```

OnLoad: Sự kiện onLoad (áp dụng cho đối tượng body) được phát sinh khi đã tải xong tài liệu. Nó cũng được phát sinh khi một ảnh đã tải xong.

onSubmit: Sự kiện onSubmit được tạo ra bất cứ khi nào người dùng truyền dữ liệu từ form đi (thường sử dụng nút Submit). Sự kiện xảy ra trước khi form thật sự được gửi đi. Trình xử lý sự kiện tương ứng với sự kiện này có thể ngăn chặn form không được gửi đi bằng cách trả về giá trị false. Cách này dùng để kiểm tra sự hợp lệ của dữ liệu nhập vào.

onMouseDown: Sự kiện này được kích hoạt khi hành động nhấp chuột xảy ra. Khi người dùng nhấp chuột. Đây là trình xử lý sự kiện cho các đối tượng button, document, và link.

onMouseUp: Sự kiện này được kích hoạt khi hành động nhả chuột xảy ra. Khi người dùng thả chuột. Đây là trình xử lý sự kiện cho đối tượng button, document, và link.

PTIT

CHƯƠNG 4: NGÔN NGỮ XML

1. CẤU TRÚC VÀ CÚ PHÁP CỦA XML

1.1 XML là gì

XML viết tắt cho chữ eXtensible Markup Language là ngôn ngữ xây dựng cấu trúc tài liệu văn bản, dựa theo chuẩn SGML (Standard Generalized Markup Language: siêu ngôn ngữ có khả năng sinh ngôn ngữ khác). SGML được phát triển cho việc định cấu trúc và nội dung tài liệu điện tử. Nó được phát minh bởi Ed Mosher, Ray Lorie và Charles F. Goldfarb của nhóm IBM research vào năm 1969. Lúc đầu nó có tên là Generalized Markup Language (GML), và được thiết kế để dùng làm meta-language, một ngôn ngữ được dùng để diễn tả các ngôn ngữ khác - văn phạm, ngữ vựng của chúng ,... Năm 1986, SGML được cơ quan ISO (International Standard Organisation) thu nhận làm tiêu chuẩn để lưu trữ và trao đổi dữ liệu. Vì SGML rất rắc rối, và HTML có nhiều giới hạn nên năm 1996 tổ chức W3C thiết kế XML. XML version 1.0 được định nghĩa trong hồ sơ February 1998 W3C Recommendation, giống như một Internet Request for Comments (RFC), là một "tiêu chuẩn".

Điểm quan trọng của kỹ thuật XML là nó không thuộc riêng về một công ty nào, nó là một sản phẩm mà trí tuệ của nó thuộc về cả thế giới, nó là một tiêu chuẩn được mọi người công nhận vì được soạn ra bởi World Wide Web Consortium - W3C (một ban soạn thảo với sự hiện diện của tất cả các chuyên gia Tin học) và những ý kiến đóng góp bằng cách trao đổi qua Email. Bản thân của XML rất là đơn giản, nhưng các công cụ chuẩn được định ra để làm việc với XML như Document Object Model - DOM, XPath, XSL, v.v.. thì rất hữu hiệu, và chính các chuẩn này được phát triển không ngừng. XML cũng giống như HTML đều là ngôn ngữ đánh dấu, nhưng điều cần nói ở đây là sự ra đời của XML để khắc phục cho một số yếu kém của HTML. HTML và XML đều sử dụng các thẻ nhưng các thẻ của HTML là một bộ dữ liệu thẻ được xây dựng và định nghĩa trước, tức là người lập trình phải tuân thủ theo các thẻ đã định nghĩa của HTML, hiện HTML có khoản hơn 400 thẻ, để nhớ hết 400 thẻ này cũng không có gì khó khăn đối với người lập trình web chuyên nghiệp nhưng thật khó đối với những người không chuyên. Hơn nữa các thẻ của HTML không thể hiện được mô tả dữ liệu trong đó. Nhưng đối với XML thì hoàn toàn khác bởi vì thẻ trong XML là do người lập trình định nghĩa và mỗi thẻ là một mô tả dữ liệu mà người lập trình muốn truyền đạt.

1.2 Một số qui tắc của XML

Một tài liệu XML phải well-formed và hợp lệ (valid). Một XML well-formed là một XML thích hợp cho phân tích (parser) thực thi. Tức là XML tuân thủ các luật lệ

PTIT

về Tag, Element, Attribute, value, ... chứa bên trong để phân tích có thể nhận diện và phân biệt mọi đặc tính. Để ý là một XML well-formed chưa chắc chứa đựng những dữ liệu hữu dụng trong công việc. Là well-formed chỉ có nghĩa là XML có cấu trúc đúng. Một tài liệu XML xác thực khi nó chứa những dữ liệu cần có trong loại tài liệu loại hay class ấy. Ví dụ một XML đặt hàng có thể bị đòi hỏi phải có một thuộc tính số hiệu đặt hàng (Attribute OrderNo) và một phần tử con ngày đặt hàng (Orderdate). Phân tích hợp lệ một XML bằng cách kiểm tra dữ liệu trong XML xem có đúng như định nghĩa trong một loại tài liệu cụ thể XML đó. Việc cụ thể này có thể là một Document Type Definition (DTD) hay một Schema.

Để well-formed, một tài liệu XML phải theo đúng các qui tắc sau:

1. Phải có một phần tử gốc (root Element) duy nhất, gọi là phần tử văn bản (Document Element), nó chứa tất cả các phần tử khác trong tài liệu.
2. Mỗi thẻ mở (opening Tag) phải có một thẻ đóng (closing Tag) giống như nó.
3. Các thẻ (Tags) trong XML thì phân biệt chữ thường và chữ hoa (case sensitive), tức là thẻ mở và thẻ đóng phải được đánh vần y như nhau, chữ hoa hay chữ thường.
4. Mỗi phần tử con (Child Element) phải nằm trọn bên trong phần tử cha của nó.
5. Giá trị thuộc tính (Attribute value) trong XML phải được đặt trong dấu nháy kép hoặc dấu nháy đơn.

Phải có một phần tử gốc duy nhất, nó chứa tất cả các phần tử khác trong tài liệu.

```
<Catalog>
    <Product ProductID="1">Chair</Product>
    <Product ProductID="2">Desk</Product>
</Catalog>
```

Mỗi thẻ mở phải có một thẻ đóng giống như nó.

```
1   <Order>
2       <OrderDate>2002-6-14</OrderDate>
3       <Customer>Helen Mooney</Customer>
4     <Item>
5       <ProductID>2</ProductID>
6       <Quantity>1</Quantity>
7     <Item>
8       <ProductID>4</ProductID>
9       <Quantity>3</Quantity>
10    </Item>
11  </Order>
```

Ví dụ trên cần thêm thẻ đóng </item> vào sau dòng thứ 6.

Trong một cặp thẻ đóng và mở phải được đánh vần như nhau, có nghĩa là các cặp ký tự của cặp thẻ đóng mở này phải giống nhau hoàn toàn.

PTIT

```
<Order>
   <OrderDate>2001-01-01</Orderdate>
   <Customer>Graeme Malcolm</Customer>
</Order>
```

Ví dụ này thẻ mở <OrderDate> và thẻ đóng </Orderdate> đánh vần không giống nhau, ta phải sửa thẻ đóng </Orderdate> thành </OrderDate> hoặc sửa thẻ <OrderDate> thành <Orderdate>.

Mỗi Phần tử con phải nằm trọn bên trong phần tử cha của nó.

```
1   <Catalog>
2     <Category
3       CategoryName="Beverages">
4       <Product ProductID="1">
5         Coca-Cola
6     </Category>
7       </Product>
    </Catalog>
```

Ví dụ này thẻ đóng </Product> đặt không đúng vị trí, ta cần đổi vị trí của dòng 5 cho dòng 6.

1.3 Chỉ thị xử lý (Processing Instructions) và lời chú thích (Comments)

Chúng ta thường thấy dòng lệnh

```
<?xml version="1.0" encoding="utf-8" standalone="yes"?>
```

Nằm ở đầu file XML, đây chính là chỉ thị xử lý, nó được đặt trong cặp Tag <? và ?>. Nó cho biết phiên bản đặc tả XML mà bộ phân tích cần làm theo, ngoài ra nó cho phép người lập trình cho biết dữ liệu trong XML dùng encoding nào, còn thuộc tính standalone sẽ cho biết tài liệu XML có cần đến một tài liệu khác không (có hai giá trịcho thuộc tính này đó là "yes" nếu không cần đến một tài liệu khác và "no" nếu cần). Trong mỗi tài liệu XML có thể có hoặc không có phần này.

Các dòng chú thích được đặt trong cặp thẻ <!-- và -->.

1.4 Không gian tên (namespace)

Có một khái niệm rất quan trọng trong XML là Namespace. Nó cho ta cách cùng một tên của phần tử nhưng chỉ đến hai kiểu dữ liệu khác nhau trong cùng một tài liệu XML. Giống như có hai học sinh trùng tên Tuấn trong lớp học, ta phải dùng thêm họ để phân biệt, ta gọi Tuấn Trần hay Tuấn Lê. Thí dụ như có một order được người ta đặt trong tiệm sách như sau:

```
<?xml version="1.0"?>
<BookOrder OrderNo="1234">
   <OrderDate>2001-01-01</OrderDate>
   <Customer>
```

PTIT

```
        <Title>Mr.</Title>
        <FirstName>Graeme</FirstName>
        <LastName>Malcolm</LastName>
    </Customer>
    <Book>
        <Title>Treasure Island</Title>
        <Author>Robert Louis Stevenson</Author>
    </Book>
</BookOrder>
```

Ta thấy có thể có sự nhầm lẫn về cách dùng phần tử Title. Trong tài liệu có hai loại Title, một cái dùng cho khách hàng Customer nói đến danh hiệu Mr., Mrs., Dr., còn cái kia để nói đến đề tựa của một quyển sách Book. Để tránh sự lầm lẫn, ta dùng Namespace để nói rõ tên phần tử thuộc về đối tượng nào. Đối tượng chỉ đến địa chỉ dữ liệu (Universal Resource Identifier - URI). Một URI có thể là một URL hay một chỗ nào định nghĩa tính cách độc đáo của nó. Một namespace cũng không cần phải nói đến một địa chỉ Internet, nó chỉ cần phải là có một, không hai. Ta có thể khai báo namespaces trong một phần tử bằng cách dùng thuộc tính xmlns (ns - namespace) bạn cũng có thể khai báo một namespace mặc định để áp dụng cho những gì nằm bên trong một phần tử, nơi khai báo namespace. Ví dụ:

```
<?xml version="1.0"?>
<BookOrder OrderNo="1234">
    <OrderDate>2001-01-01</OrderDate>
    <Customer
xmlns="http://www.northwindtraders.com/customer">
        <Title>Mr.</Title>
        <FirstName>Graeme</FirstName>
        <LastName>Malcolm</LastName>
    </Customer>
    <Book xmlns="http://www.northwindtraders.com/book">
        <Title>Treasure Island</Title>
        <Author>Robert Louis Stevenson</Author>
    </Book>
</BookOrder>
```

Trong Customer thì dùng namespace http://www.northwindtraders.com/customer và bên trong Book thì dùng namespace http://www.northwindtraders.com/book. Tuy nhiên, ta sẽ giải quyết làm sao nếu trong order có nhiều customer và nhiều book. Một cách giải quyết là khai báo chữ viết tắt cho các namespaces ngay ở đầu tài liệu, trong phần tử gốc (tức là phần tử Document). Sau đó bên trong tài liệu ta sẽ tiếp đầu (prefix) các phần tử cần xác nhận namespace bằng chữ viết tắt của namespace nó. Ví dụ:

```
<?xml version="1.0"?>
<BookOrder  xmlns="http://www.northwindtraders.com/order"
        xmlns:cust="http://www.northwindtraders.com/customer"
```

PTIT

```
        xmlns:book="http://www.northwindtraders.com/book"
OrderNo="1234">
   <OrderDate>2001-01-01</OrderDate>
   <cust:Customer>
      <cust:Title>Mr.</cust:Title>
      <cust:FirstName>Graeme</cust:FirstName>
      <cust:LastName>Malcolm</cust:LastName>
   </cust:Customer>
   <book:Book>
      <book:Title>Treasure Island</book:Title>
      <book:Author>Robert Louis Stevenson</book:Author>
   </book:Book>
</BookOrder>
```

2. DTD (DOCUMENT TYPE DEFINITION)

2.1 DTD là gì?

DTD (Document Type Definition) là kiểu tài liệu dùng để định nghĩa kiểu dữ liệu cho các phần tử trong tài liệu XML. Khi chúng ta định nghĩa các phần tử trong XML là tùy thích, miễn sao cho nó hợp quy tắc của tài liệu XML. Tuy nhiên để tường minh hơn thì ta nên định nghĩa kiểu dữ liệu cho từng phần tử trong tài liệu XML. Với cách định nghĩa này, khi chúng ta đọc một tài liệu XML nào thì chỉ cần đọc phần DTD thì chúng ta sẽ biết được cấu trúc của tài liệu XML. Ví dụ:

PTIT

```
<?xml version="1.0"?>
  <!DOCTYPE note [
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend</body>
</note>
```

Xem ví dụ trên chúng ta thấy phần được bôi đen là phần DTD dùng để mô tả kiểu dữ liệu của tài liệu XML, phần còn lại là các phần tử của tài liệu XML. Phần DTD trong ví dụ này được hiểu như sau: tài liệu XML có một phần tử gốc tên là note, phần tử gốc này có 3 phần tử con là from, heading, body và 3 phần tử con này có kiểu dữ liệu text.

2.2 Định nghĩa một tài liệu DTD

Phần tử <!DOCTYPE>: Phần tử này có chức năng dùng để khai báo bắt đầu định nghĩa kiểu tư liệu DTD. Định nghĩa kiểu tư liệu có 2 dạng, đó là DTD tham chiếu nội và DTD tham chiếu ngoại. DTD tham chiếu nội là DTD được định nghĩa ngay trong tài liệu XML còn DTD tham chiếu ngoại là DTD được định nghĩa bên ngoài tài liệu XML.

Định nghĩa DTD tham chiếu nội

Để bắt đầu định nghĩa kiểu tư liệu DTD tham chiếu nội chúng ta dùng cú pháp sau:

```
<!DOCTYPE root-element [DTD]>
```

Trong đó root-element là phần tử gốc của tài liệu XML, DTD là các định nghĩa cho các phần tử trong tài liệu XML.

```
<?xml version="1.0"?>
  <!DOCTYPE note [
  <!ELEMENT note body>
  <!ELEMENT body (#PCDATA)>
]>
<note>
  <body>Don't forget me this weekend</body>
</note>
```

Định nghĩa DTD tham chiếu ngoại

Sử dụng định nghĩa DTD tham chiếu ngoại sẽ làm cho các ứng dụng XML của chúng ta trở nên dễ dàng chia sẻ và dùng chung với các ứng dụng khác. Có hai cách để chỉ định một DTD tham chiếu ngoại: Tham chiếu ngoại riêng và tham chiếu ngoại chung. Những định nghĩa DTD tham chiếu ngoại riêng được sử dụng cho một nhóm người mang tính cá nhân, chúng không được dùng cho mục đích chung rộng lớn, mục đích phân phối. Còn những định nghĩa DTD tham chiếu ngoại chung sẽ mang tính cộng đồng hơn.

Để định nghĩa một DTD tham chiếu ngoại riêng chúng ta dùng cú pháp sau:

```
<!DOCTYPE root-element SYSTEM "filename">
```

Trong đó root-element là tên của phần tử gốc trong tài liệu XML, filename là tên file định nghĩa kiểu tư liệu DTD.

Ví dụ:

```
<?xml version="1.0"?>
  <!DOCTYPE note [
  <!ELEMENT note body>
  <!ELEMENT body (#PCDATA)>
]>
<note>
  <body>Don't forget me this weekend</body>
</note>
```

PTIT

Định nghĩa DTD tham chiếu ngoại

Sử dụng định nghĩa DTD tham chiếu ngoại sẽ làm cho các ứng dụng XML trở nên dễ dàng chia sẻ và dùng chung với các ứng dụng khác. Có hai cách để chỉ định một DTD tham chiếu ngoại: tham chiếu ngoại riêng và tham chiếu ngoại chung. Để định nghĩa một DTD tham chiếu ngoại riêng chúng ta dùng cú pháp sau:

```
<!DOCTYPE root-element SYSTEM "filename">
```

Trong đó root-element là tên của phần tử gốc trong tài liệu XML, filename là tên file định nghĩa kiểu tư liệu DTD.

3. CÁC ỨNG DỤNG CỦA PHÂN TÍCH (PARSER) XML

3.1 Các cách phân tích (parser) xâu XML trong Java

Java cung cấp một số thư viện để phân tích cú pháp và các văn bản xử lý XML. Một phân tích cú pháp XML cung cấp các chức năng cần thiết để đọc và chỉnh sửa một tập tin XML.

Các cách thực hiện:

- Phân tích tệp XML dùng DOM Parser
- Phân tích tệp XML dùng SAX Parser
- Phân tích tệp XML dùng StAX Parser

Giả sử ta có tệp Employees.xml như sau:

```
<?xml version="1.0" encoding="UTF-8"?>
<Employees>
     <Employee ID="1">
          <Firstname>Lebron</Firstname >
          <Lastname>James</Lastname>
          <Age>30</Age>
          <Salary>2500</Salary>
     </Employee>
     <Employee ID="2">
          <Firstname>Anthony</Firstname>
          <Lastname>Davis</Lastname>
          <Age>22</Age>
          <Salary>1500</Salary>
     </Employee>
     <Employee ID="3">
          <Firstname>Paul</Firstname>
          <Lastname>George</Lastname>
          <Age>24</Age>
          <Salary>2000</Salary>
     </Employee>
     <Employee ID="4">
          <Firstname>Blake</Firstname>
          <Lastname>Griffin</Lastname>
          <Age>25</Age>
```

PTIT

```
            <Salary>2250</Salary>
        </Employee>
</Employees>
```

Chúng ta xây dựng thực thể Employee.java như sau:

```
class Employee {
    private String ID;
    private String Firstname;
    private String Lastname;
    private int age;
    private double salary;
    public Employee(String ID, String Firstname, String Lastname, int age,
            double salary) {
        this.ID = ID;
        this.Firstname = Firstname;
        this.Lastname = Lastname;
        this.age = age;
        this.salary = salary;
    }
    @Override
    public String toString() {
        return "<" + ID + ", " + Firstname + ", " + Lastname + ", " + age
                + ", " + salary + ">";
    } }
```

3.2 Dùng DOM

3.2.1 *Phân tích dùng DOM*

DOM (Document Object Model): Cache all – cơ chế của nó là đọc toàn bộ nội dung tập tin XML vào bộ nhớ (do đó nếu XML lớn thì làm chậm chương trình và có thể phung phí bộ nhớ vì không phải lúc nào ta cũng muốn đọc hết nội dung XML. Sử dụng SAX Parser sẽ nhanh hơn và tốn ít bộ nhớ hơn DOM Parser), sau đó mô hình nó thành cấu trúc cây để dễ dàng di chuyển hoặc thao tác từng node. Trong DOM, mọi thứ trong một tài liệu XML là một node.

DOM cho phép lấy: NodeLists, .getElementsByTagName() , .item(i), .getName() , .getValue() , .getFirstChild() , .getAttributes(),…

Chúng ta dùng 2 lớp:

```
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
```

Ví dụ:

```
public class DomParserExample {
public static void main(String[] args) throws ParserConfigurationException,
            SAXException, IOException {
  if (args.length != 1)
     throw new RuntimeException("The name of the XML file is required!");
     DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
     DocumentBuilder builder = factory.newDocumentBuilder();
     // Load the input XML document, parse it and return an instance of the
        // Document class.
```

PTIT

```
      Document document = builder.parse(new File(args[0]));
      List<Employee> employees = new ArrayList<Employee>();
      NodeList nodeList = document.getDocumentElement().getChildNodes();
      for (int i = 0; i < nodeList.getLength(); i++) {
         Node node = nodeList.item(i);
             if (node.getNodeType() == Node.ELEMENT_NODE) {
                 Element elem = (Element) node;
                 // Get the value of the ID attribute.
         String ID = node.getAttributes().getNamedItem("ID").getNodeValue();
                 // Get the value of all sub-elements.
                 String firstname = elem.getElementsByTagName("Firstname")
                         .item(0).getChildNodes().item(0).getNodeValue();
                 String lastname =
elem.getElementsByTagName("Lastname").item(0)
                         .getChildNodes().item(0).getNodeValue();
             Integer age = Integer.parseInt(elem.getElementsByTagName("Age")
                         .item(0).getChildNodes().item(0).getNodeValue());
                 Double salary = Double.parseDouble(elem
                   .getElementsByTagName("Salary").item(0).getChildNo  des()
                         .item(0).getNodeValue());
        employees.add(new Employee(ID, firstname, lastname, age, salary));
             } }
          // Print all employees.
          for (Employee empl : employees)
             System.out.println(empl.toString());
     } }
```

3.2.2 *Phân tích rss trên trang dân trí*

Chúng ta mở trang dân trí mục xã hội (http://dantri.com.vn/xa-hoi.htm) có dạng:



Hình 4.1: mục xã hội trên trang dân trí

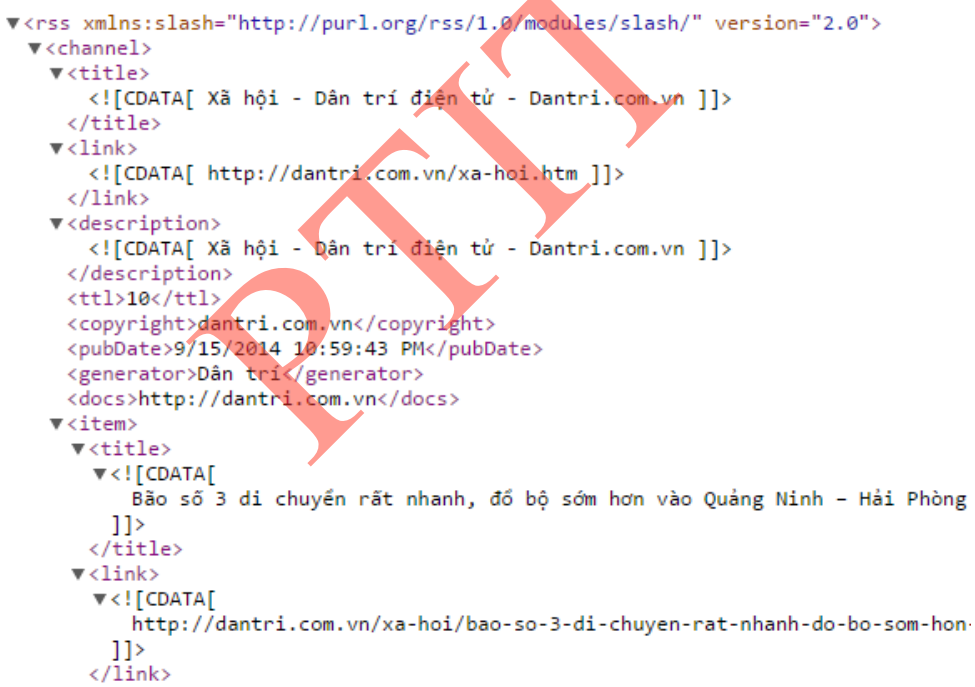
Khi ở dạng rss (http://dantri.com.vn/xa-hoi.rss) có dạng:

PTIT



Hình 4.2: trang dân trí ở dạng rss

Trang dân trí khi ở dạng rss thực chất chúng được lưu ở dạng XML như sau:



Hình 4.3: Lưu trữ trang dân trí dạng rss (bản XML)

Bây giờ chúng ta dùng DOM để phân tích trang này:

```
public NodeList readNews(String urlRSS){
        try {
            URL url= new URL(urlRSS);
            doc= builder.parse(url.openStream());
            NodeList nodes= doc.getElementsByTagName("item");
            return nodes;
        } catch (Exception ex) {
            System.out.println(ex);
```

PTIT

```
        }
        return null;
    }
    public void writeXML(NodeList nodes){
        try {
            doc= builder.newDocument();
            Element root=doc.createElement("tintuc");
            doc.appendChild(root);
            int max= nodes.getLength();
            for(int i=0; i<max; i++){
                Element item_E=doc.createElement("item");
                Element title_E= doc.createElement("title");
                Element link_E= doc.createElement("link");
                Element pubDate_E= doc.createElement("pubDate");
                Element author_E= doc.createElement("author");
                Element comments_E= doc.createElement("comments");
                Element description_E=
doc.createElement("description");
                Element e= (Element) nodes.item(i);
                title_E.setTextContent(getElementValue(e, "title"));
                link_E.setTextContent(getElementValue(e, "link"));
           pubDate_E.setTextContent(getElementValue(e, "pubDate"));
         author_E.setTextContent(getElementValue(e, "dc:creator"));
      comments_E.setTextContent(getElementValue(e, "wfw:comment"));
    description_E.setTextContent(getElementValue(e, "description"));
                item_E.appendChild(title_E);
                item_E.appendChild(link_E);
                item_E.appendChild(pubDate_E);
                item_E.appendChild(author_E);
                item_E.appendChild(comments_E);
                item_E.appendChild(description_E);
                root.appendChild(item_E);
        }
            Transformer transformer=
TransformerFactory.newInstance().newTransformer();
            boolean b=new File("C:\\RSS").mkdir();
            File file= new File("C:\\RSS\\output.xml");
            FileOutputStream out= new FileOutputStream(file);
   transformer.transform(new DOMSource(doc), new StreamResult(out));
            out.flush();
            out.close();
        } catch (Exception ex) {
            System.out.println(ex);
        }           }
    public String getElementValue(Element e, String tagName){
        return
getCharacterDataFromElement((Element)e.getElementsByTagName(tagName)
.item(0));
    }
    private String getCharacterDataFromElement(Element element) {
        try{
            Node child= element.getFirstChild();
```

PTIT

```
            if(child instanceof CharacterData){
                CharacterData cd= (CharacterData) child;
                return cd.getData();
            }
        }catch(Exception ex){    }
        return "";
    } }
```

3.3 Dùng SAX

Khi làm việc với tài liệu XML bằng kỹ thuật Sax, chúng ta thường sử dụng 4 sự kiện chính. Đó là:

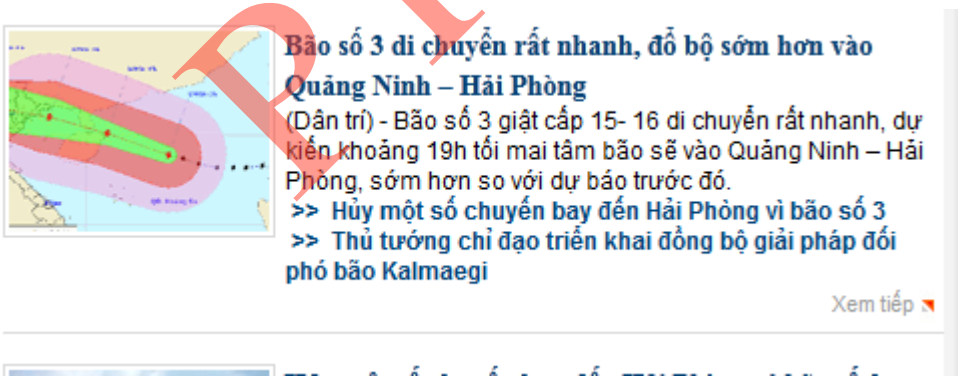
- startDocument(): Báo hiệu bắt đầu một tài liệu XML
- startElement(): Báo hiệu bắt đầu một thẻ XML
- character(): Sự kiện này giúp ta lấy nội dung của thẻ
- endDocument(): Báo hiệu kết thúc một tài liệu XML

Các thẻ là phần tử (element) trong SAX có thể dùng các hàm:

- .getAttributeCount()
- .getAttributeName()
- .getAttributeValue()

Ví dụ chúng ta lấy các tin tức trên 3 trang mạng: http://vnexpress.net/, http://www.24h.com.vn/ và http://dantri.com.vn/ sử dụng SAX.

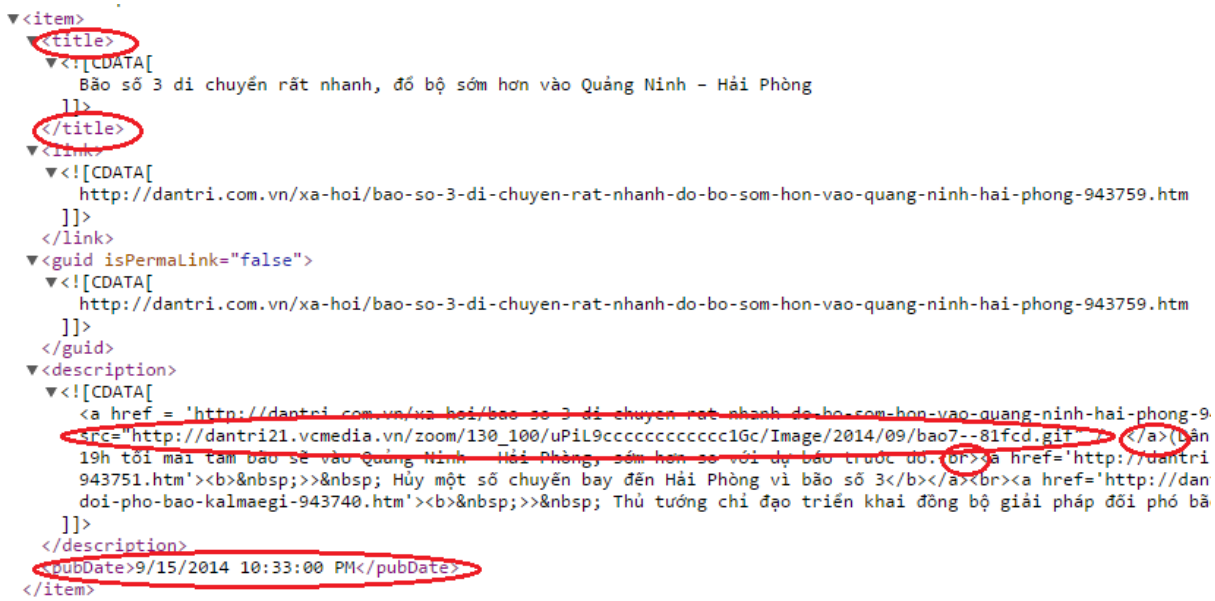
Ta thấy các tin trên trang đều có dạng:



Hình 4.4: Hình ảnh một bài báo (mục)

Khi phân tích từng bài (mục) có dạng:

PTIT



Hình 4.5: Phân tích 1 bài báo (item)

Đầu tiên tạo thực thể RSSItem.java:

```
public class RSSItem {
            private String title;
      private String date;
      private String description;
      private String link;
      private String image;
      public String getTitle() {
            return title;    }
      public void setTitle(String title) {
            this.title = title;   }
      public String getDate() {
            return date;}
      public void setDate(String date) {
            this.date = date;      }
      public String getDescription() {
            return description;}
      public void setDescription(String description) {
            this.description = description;}
      public String getLink() {
            return link;}
      public void setLink(String link) {
            this.link = link;}
     public String getImage() {
         return image;    }
     public void setImage(String image) {
         this.image = image;
     }}
```

Xây dựng lớp RSSHandle.java kế thừa từ lớp DefaultHandler:

```
public class RSSHandle extends DefaultHandler {
      public static final String ITEM = "item";
      public static final String TITLE = "title";
```

PTIT

```
      public static final String DESCRIPTION = "description";
      public static final String LINK = "link";
      public static final String DATE = "pubdate";
      private RSSItem item;
      private boolean STATE = false;
      private List<RSSItem> list = new ArrayList<RSSItem>();
      StringBuilder sb = new StringBuilder();
      @Override
      public void characters(char[] ch, int start, int length)
                  throws SAXException {
            super.characters(ch, start, length);
                 sb = new StringBuilder();
            if (STATE && (sb!=null)){
                  sb.append(ch,start,length);
            }}
      @Override
public void endElement(String uri, String localName, String qName)
                  throws SAXException {
            super.endElement(uri, localName, qName);
            if (qName.equalsIgnoreCase(ITEM)){
                  list.add(item);
            }else if (STATE) {
                  if (qName.equalsIgnoreCase(TITLE)){
                        item.setTitle(sb.toString().trim());
                  } else if (qName.equalsIgnoreCase(DESCRIPTION)){
                              try{
                                    int i = sb.indexOf("src=");
                                    int j = sb.indexOf(".jpg");
                                    if (j<0) j=sb.indexOf(".JPG");
                                    if (j<0) j=sb.indexOf(".gif");
                                    String s=null;
                                    if (j>0)  s=sb.substring(i+5,j+4);
                                    item.setImage(s);
                                    i = sb.indexOf("</a>");
                                    s= sb.substring(i+4);
                                    s=s.replaceAll("</br>", "");
                                    i= s.indexOf("<br>");
                                    if (i>0) s=s.substring(0, i);
                                    item.setDescription(s.trim());
                              }
                              catch(Exception e){
                                    e.printStackTrace();
                              }
                  } else if (qName.equalsIgnoreCase(DATE)){
                        item.setDate(sb.toString().trim());
                  } else if (qName.equalsIgnoreCase(LINK)){
                        item.setLink(sb.toString().trim());
                  }
                  sb = new StringBuilder();
            }     }
      @Override
public void startElement(String uri, String localName, String qName,
```

PTIT

```
                  Attributes attributes) throws SAXException {
            super.startElement(uri, localName, qName, attributes);
            if (qName.equalsIgnoreCase(ITEM)){
                  item = new RSSItem();
                  STATE = true;
            }     }
      public List<RSSItem> getList() {
            return list;
      }}
```

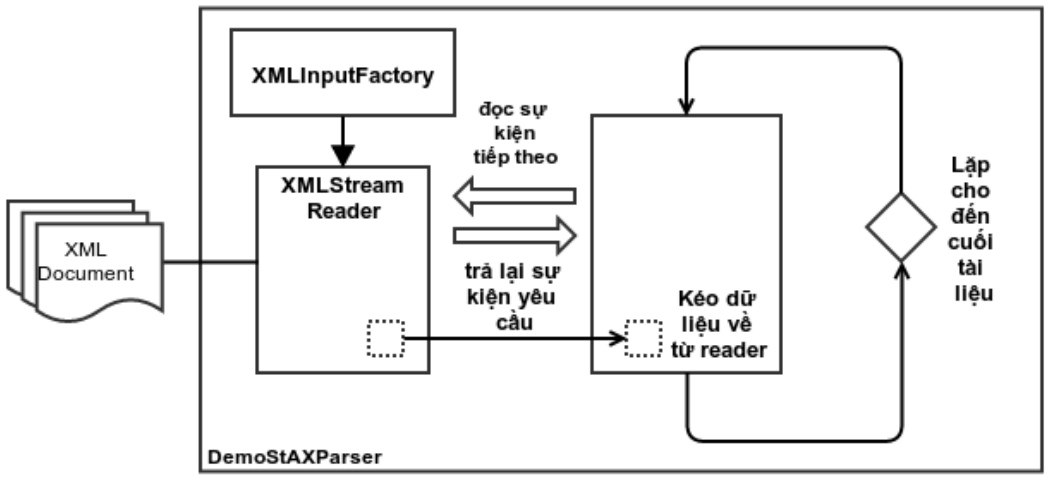
Tạo lớp phân tích dùng SAX RSSParser.java:

```
import org.xml.sax.InputSource;
import org.xml.sax.XMLReader;
public class RSSParser {
      public List<RSSItem> getList(String link){
            try {
                  URL url = new URL(link);
                  XMLReader rd =
SAXParserFactory.newInstance().newSAXParser().getXMLReader();
                  RSSHandle handle = new RSSHandle();
                  rd.setContentHandler(handle);
                  InputSource source = new
InputSource(url.openStream());
                  rd.parse(source);
                  return handle.getList();
                  } catch (Exception e) {
                  e.printStackTrace();
                  return null;
            }}}
```

PTIT

3.4 Dùng StAX

Streaming API for XML, được gọi là StAX, là API dành cho việc đọc và ghi dữ liệu đến file XML. StAX sử dụng mô hình Pull-Parsing, ứng dụng điều khiển toàn bộ quá trình phân tích tệp xml bằng cách lấy các phần tử từ trình phân tích. Tương tự như SAX, StAX cũng áp dụng cơ chế xử lý sự kiện trong việc phân tách nội dung XML. Tuy nhiên, giữa SAX và StAX có sự khác biệt cơ bản: bộ xử lý SAX (SAX Parser) chủ động gọi và "đẩy" dữ liệu cho phương thức xử lý mỗi khi có sự kiện phát sinh, trong khi bộ xử lý StAX (StAX Reader) chờ chương trình gọi đến sự kiện tiếp theo và chuẩn bị sẵn dữ liệu để chương trình "kéo" dữ liệu ra xử lý.



Hình 4.6: Qui trình phân tích tài liệu XML dùng StAX

Ví dụ ta phân tích dữ liệu tại http://www.dantri.com.vn/doi-song.rss, ta thực hiện tuần tự như sau:

Tạo thực thể Item.java:

```
public class Item {
   String title;
  String description;
  String link;
  String author;
  String guid;
  public String getTitle() {
    return title;}
  public void setTitle(String title) {
    this.title = title;}
  public String getDescription() {
    return description;}
  public void setDescription(String description) {
    this.description = description;}
  public String getLink() {
    return link;}
  public void setLink(String link) {
    this.link = link;}
  public String getAuthor() {
    return author;}
  public void setAuthor(String author) {
    this.author = author;}
  public String getGuid() {
    return guid;}
  public void setGuid(String guid) {
    this.guid = guid; }
  @Override
  public String toString() {
    return "FeedMessage [title=" + title + ", description=" +
description
        + ", link=" + link + ", author=" + author + ", guid=" + guid
```

PTIT

```
        + "]";
  } }
```

Sau đó tạo danh sách các đối tượng Item, Chanel.java:

```
public class Chanel {
  final String title;
  final String link;
  final String description;
  final String language;
  final String copyright;
  final String pubDate;
  final List<Item> entries = new ArrayList<Item>();
  public Chanel(String title, String link, String description,
String language,
      String copyright, String pubDate) {
    this.title = title;
    this.link = link;
    this.description = description;
    this.language = language;
    this.copyright = copyright;
    this.pubDate = pubDate;}
  public List<Item> getMessages() {
    return entries;}
  public String getTitle() {
    return title;}
  public String getLink() {
    return link;}
  public String getDescription() {
    return description;}
  public String getLanguage() {
    return language;}
  public String getCopyright() {
    return copyright;}
  public String getPubDate() {
    return pubDate;}
  @Override
  public String toString() {
    return "Feed [copyright=" + copyright + ", description=" +
description
        + ", language=" + language + ", link=" + link + ", pubDate="
        + pubDate + ", title=" + title + "]";
  }  }
```

Tạo cơ chế đọc rss, Docrss.java

```
public class Docrss {
    static final String TITLE = "title";
  static final String DESCRIPTION = "description";
  static final String CHANNEL = "channel";
  static final String LINK = "link";
  static final String ITEM = "item";
  static final String PUB_DATE = "pubDate";
  static final String COPYRIGHT = "copyright";
```

PTIT

```
  final URL url;
  public Docrss(String rssUrl) {
    try {
      this.url = new URL(rssUrl);
    } catch (MalformedURLException e) {
      throw new RuntimeException(e);
    }
  }
  @SuppressWarnings("null")
  public Chanel readrss() {
    Chanel chanel = null;
    try {
      boolean isFeedHeader = true;
      String description = "";
      String title = "";
      String link = "";
      String language = "";
      String copyright = "";
      String author = "";
      String pubdate = "";
      String guid = "";
      XMLInputFactory inputFactory = XMLInputFactory.newInstance();
      InputStream in = read();
 XMLEventReader eventReader = inputFactory.createXMLEventReader(in);
      while (eventReader.hasNext()) {
      XMLEvent event = eventReader.nextEvent();
      if (event.isStartElement()) {
     if (event.asStartElement().getName().getLocalPart() == (ITEM)){
            if (isFeedHeader) {
              isFeedHeader = false;
chanel = new Chanel(title,link,description,language,copyright,
pubdate);
            }
            event = eventReader.nextEvent();
            continue;
          }
   if (event.asStartElement().getName().getLocalPart() == (TITLE)) {
            event = eventReader.nextEvent();
            title = event.asCharacters().getData();
            continue;
          }
    if (event.asStartElement().getName().getLocalPart() == (LINK)) {
            event = eventReader.nextEvent();
            link = event.asCharacters().getData();
            continue;
          }
if(event.asStartElement().getName().getLocalPart() ==(DESCRIPTION)){
            event = eventReader.nextEvent();
            description = event.asCharacters().toString();
            continue;
          }
if (event.asStartElement().getName().getLocalPart() ==(COPYRIGHT)){
```

PTIT

```
            event = eventReader.nextEvent();
            copyright= event.asCharacters().toString();
            continue;
          }
if (event.asStartElement().getName().getLocalPart() == (PUB_DATE)){
            event = eventReader.nextEvent();
            pubdate = event.asCharacters().getData();
            continue;
          }
        } else if (event.isEndElement()) {
  if (event.asEndElement().getName().getLocalPart() == (ITEM)) {
            Item item = new Item();
            item.setDescription(description);
            item.setGuid(guid);
            item.setLink(link);
            item.setTitle(title);
            chanel.getMessages().add(item);
            event = eventReader.nextEvent();
            continue;
          }}    }
    } catch (XMLStreamException e) {
      throw new RuntimeException(e);
    }
    return chanel;}
  private InputStream read() {
    try {
      return url.openStream();
    } catch (IOException e) {
      throw new RuntimeException(e);
    } }}
```

Các bạn có thể tham khảo đầy đủ tại:
http://www.vogella.com/articles/RSSFeed/article.html

PTIT

3.5 Phân tích HTML trong Java dùng Jsoup

Jsoup, là thư viện phân tích HTML, nó rất dễ sử dụng và linh hoạt, đủ để có được bất cứ điều gì bạn muốn. Thư viện sử dụng Jsoup có thể download tại http://jsoup.org/download.

3.5.1 *Ví dụ dùng jsoup lấy ảnh*

Ví dụ chúng ta sử dụng Jsoup để lấy các hình ảnh trên trang http://yahoo.com như sau:

```
import java.io.IOException;
import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.jsoup.select.Elements;
public class DemoJsoup {
    public static void main(String[] args) {
        Document doc;
```

```
        try {
            // get all images
            doc = Jsoup.connect("http://yahoo.com").get();
            Elements images =
doc.select("img[src~=(?i)\\.(png|jpe?g|gif)]");
            for (Element image: images) {
                System.out.println("\nsrc: " + image.attr("src"));
                System.out.println("height: " +
image.attr("height"));
                System.out.println("width: " + image.attr("width"));
                System.out.println("alt: " + image.attr("alt"));
            }
        } catch (IOException e) {
            e.printStackTrace();
        }}}
```

3.5.2 *Ví dụ dùng jsoup lấy comment trên vnexpress.net*

Để get document của 1 link web sử dụng câu lênh:

```
Map<String, String> cookies = res.cookies();
        Document doc =
Jsoup.connect(url).method(Connection.Method.POST)
                .userAgent("Mozilla/5.0 (Windows NT 6.0)
AppleWebKit/536.5 (KHTML, like Gecko) Chrome/19.0.1084.46
Safari/536.5")
                .timeout(100 * 1000)
                .ignoreHttpErrors(true)
                .cookies(cookies)
                .get();
```

Trong đó timeout là thời gian chờ tối đa, sử dụng userAgent để trang web coi mình đang truy cập bằng trình duyệt. Để lấy 1 tập các thẻ có 1 mẫu chung nào đó sử dụng câu lệnh select. Ví dụ lấy thẻ div trong có thuộc tính id = 123 sử dụng lệnh

```
Elements IDs= doc.select("div[id=123]");
```

Ví dụ chúng ta lấy các comment của người dùng cho 1 mục trên trang vnexpress.net dùng jsoup như sau:

```
import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.select.Elements;
public class Vnexpress {
    public String forwardLink(String url) throws IOException {
        Connection.Response res = Jsoup.connect(url)
                .method(Connection.Method.POST)
                .userAgent("Mozilla/5.0 (Windows NT 6.0)
AppleWebKit/536.5 (KHTML, like Gecko) Chrome/19.0.1084.46
Safari/536.5")
                .timeout(100 * 1000)
                .ignoreHttpErrors(true)
                .execute();
        Map<String, String> cookies = res.cookies();
```

PTIT

```
        Document doc =
Jsoup.connect(url).method(Connection.Method.POST)
                .userAgent("Mozilla/5.0 (Windows NT 6.0)
AppleWebKit/536.5 (KHTML, like Gecko) Chrome/19.0.1084.46
Safari/536.5")
                .timeout(100 * 1000)
                .ignoreHttpErrors(true)
                .cookies(cookies)
                .get();
        Elements script =
doc.select("script[type=text/javascript]");
        Pattern p = Pattern.compile("objectid.+");
        Pattern p1 = Pattern.compile("objecttype.+");
        Pattern p2 = Pattern.compile("siteid.+");
        Pattern p3 = Pattern.compile("categoryid.+");
        Pattern p4 = Pattern.compile("sign.+");
        Pattern p5 = Pattern.compile("limit.+");
        Matcher m = p.matcher(script.html());
        Matcher m1 = p1.matcher(script.html());
        Matcher m2 = p2.matcher(script.html());
        Matcher m3 = p3.matcher(script.html());
        Matcher m4 = p4.matcher(script.html());
        Matcher m5 = p5.matcher(script.html());
        StringBuffer text = new StringBuffer();
        while (m.find()) {
            text.append(m.group());
        }
        while (m1.find()) {
            text.append(m1.group());
        }
        while (m2.find()) {
            text.append(m2.group());
        }
        while (m3.find()) {
            text.append(m3.group());
        }
        while (m4.find()) {
            text.append(m4.group());
        }
        while (m5.find()) {
            text.append(m5.group());
        }
        String[] temp = text.toString().split(":");
String objectid = temp[1].trim().substring(0,temp[1].length() - 25);
String objecttype = temp[2].trim().substring(0,temp[2].length()- 8);
 String siteid = temp[3].trim().substring(0, temp[3].length() - 22);
String categoryid = temp[4].trim().substring(0,temp[4].length()- 6);
   String sign = temp[5].trim().substring(1, temp[5].length() - 8);
        String limit = temp[6].trim();
        String linkFor =
"http://interactions.vnexpress.net/index/get?callback=jQuery17105671
586426906288_1413521283854&offset=0&limit=" + limit
```

PTIT

```
                + "&frommobile=0&sort=like&objectid=" + objectid +
"&objecttype=" + objecttype + "&siteid=" + siteid + "&categoryid=" +
categoryid
                + "&sign=" + sign + "&cookie_aid=null&usertype=4";
        System.out.println(linkFor);
        return linkFor;
    }
    public static void main(String[] args) throws Exception {
        System.out.println("Nhap link:");
        String s = new Scanner(System.in).next();
        Vnexpress vnexpress = new Vnexpress();
        String url = vnexpress.forwardLink(s);
        ArrayList<Binhluan> list = JsonParser.getComment(url);
        for (Binhluan i : list) {
            System.out.println(i.getNickname());
            System.out.println(i.getComment());
            System.out.println(i.getSolike());
        }}}
```

3.6 Dùng Json

JSON (JavaScript Object Notation - ký hiệu đối tượng JavaScript). Đây là cú pháp để lưu trữ và trao đổi thông tin văn bản, giống như XML. Nhưng nhỏ hơn, nhanh hơn và dễ dàng hơn để phân tích (parse) tệp xml.

- JSON là định dạng trao đổi dữ liệu độc lập, tương tự như XML
- JSON được hỗ trợ trực tiếp trong Javascript
- JSON chỉ cho phép lưu trữ dữ liệu dưới dạng chữ và số
- Cấu trúc của JSON dựa trên các cặp Key/Value
- Key là một chuỗi
- Value có thể là chuỗi số, giá trị boolean hoặc object

Một đối tượng JSON bao gồm các cặp Key/Value và được bắt đầu bởi dấu "{" và kết thúc bởi dấu "}". Một mảng các giá trị được nằm trong cặp dấu ngoặc vuông [ ] và cách nhau bởi dấu phẩy ",". Mô tả đối tượng JSON gồm 2 phần: mô tả đối tượng, mô tả 1 tập đối tượng (List). Ví dụ sử dụng JSON để mô tả đối tượng Book có 3 thuộc tính *bookId, title, price* như sau:

```
{"bookId": 1, "title": "JSON Tutorial", "price": 40}
```

Hiện nay có rất nhiều thư viện hỗ trợ chúng ta làm việc với JSON trên Java: Json.simple, org.json, … Có 3 lớp thao tác với đối tượng JSON:

```
import org.json.simple.JSONArray;
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
```

PTIT

3.6.1 *Ví dụ về sử dụng JSON*

Chúng ta có một đối tượng Student, với các thuộc tính: ***id, name, email***. Được mô tả:

```
import org.json.simple.JSONAware;
public class Student implements JSONAware {
	private int id;
	private String name;
	private String email;
	public Student() {}
	public Student(int id, String name, String email) {
		super();
		this.id = id;
		this.name = name;
		this.email = email;	}
	// Các phương thức get và set
	public int getId() {
		return id;}
	public void setId(int id) {
		this.id = id;	}
	public String getName() {
		return name;	}
	public void setName(String name) {
		this.name = name;	}
	public String getEmail() {
		return email;	}
	public void setEmail(String email) {
		this.email = email;	}
	// Phần quan trọng nhất: toJSONString()
	@Override
	public String toJSONString() {
		StringBuffer sb = new StringBuffer();
	sb.append("{");//Bắt đầu JSON dấu mở ngoặc nhọn
	sb.append("\"id\":\"" + getId() + "\"");//là "id":"Giá_Trị"
		sb.append(","); // sau mỗi cặp key/value là một dấu phẩy
		sb.append("\"name\":\"" + getName() + "\"");
		sb.append(",");
		sb.append("\"email\":\"" + getEmail() + "\"");
		sb.append("}");
		return sb.toString();
	}}
```

Xử lý với đối tượng JSON:

```
public class Main {
	public static void main(String[] args) {
		genJSONString();// Phần 1-1
		genJSONArray();// Phần 1-2
		parseJSONObject();// Phần 2-1
		parseJSONArray();// Phần 2-2
	}
	private static void genJSONString() {
		 //Phần 1-1: Mô tả đối tượng bằng JSON.
Student student = new Student(1, "Duy hung", "duyhung@gmail.com");
```

PTIT

```
            String JSONResult = student.toJSONString();
            System.out.println(JSONResult);
      }
      private static void genJSONArray() {
             //Phần 1-1: Mô tả đối tượng bằng JSON.
            // Khởi tạo list với 3 đối tượng Student
            List<Student> list = new ArrayList<Student>() {
            {add(new Student(1, "Duy hung", "duyhung.ws@gmail.com"));
            add(new Student(2, "Do manh", "code4lifevn@gmail.com"));
            add(new Student(3, "Thai trung", "trungvt@gmail.com"));
                  }};
            // Sử dụng JSONArray
            String JSONResult = JSONArray.toJSONString(list);
            System.out.println(JSONResult);
      }
      private static void parseJSONObject() {
            //Phần 2-1: Phân tích chuỗi JSON thành đối tượng.
            // Chuỗi JSON mẫu mô tả một đối tượng Student
            String jsonString = "{\"id\": \"1\", \"name\": \"Duy
hung\", \"email\": \"duyhung.ws@gmail.com\"}";
            // Phân tích
            try {
                  // 1. Tạo ra một JSONParser
                  JSONParser jsonParser = new JSONParser();
                  // 2. Parser chuỗi JSON về một JSONObject
JSONObject jsonObject = (JSONObject) jsonParser.parse(jsonString);
            // 3. Lấy các giá trị trong jsonObject thông qua các Key
            int id = Integer.parseInt((String) jsonObject.get("id"));
                  String name = (String) jsonObject.get("name");
                  String email = (String) jsonObject.get("email");
                  Student student = new Student(id, name, email);
                  System.out.println("ID    : " + student.getId());
                  System.out.println("Name  : " + student.getName());
                  System.out.println("Email : " + student.getEmail());
            } catch (ParseException e) {
                  e.printStackTrace();
            }     }
      private static void parseJSONArray() {
      //Phần 2-2: Phân tích chuỗi JSON thành tập đối tượng.
      // Chuỗi JSON mẫu mô tả một danh sách các đối tượng Student
            String jsonString = "[{\"id\": \"1\", \"name\": \"Duy
hung\", \"email\": \"duyhung.ws@gmail.com\"}, {\"id\": \"2\",
\"name\": \"Do manh\", \"email\": \"code4lifevn@gmail.com\"},
{\"id\": \"3\", \"name\": \"Thai trung\", \"email\":
\"trungvt@gmail.com\"} ]";
            // Phân tích
            try {
                  // 1. Tạo ra một JSONParser
                  JSONParser jsonParser = new JSONParser();
                  // 2. Parser chuỗi JSON về một JSONArray
      JSONArray jsonArray = (JSONArray) jsonParser.parse(jsonString);
// 3. Lấy các giá trị trong jsonArray bằng cách lặp qa từng phần tử
```

PTIT

```
            for (int i = 0; i < jsonArray.size(); i++) {
            // Mỗi phần tử của JSONArray lại là một JSONObject
        JSONObject jsonObject = (JSONObject) jsonArray.get(i);
        int id = Integer.parseInt((String) jsonObject.get("id"));
                String name = (String) jsonObject.get("name");
            String email = (String) jsonObject.get("email");
                // Gán vào đối tượng và in ra màn hình
            Student student = new Student(id, name, email);
            System.out.println("ID    : " + student.getId());
            System.out.println("Name  : " + student.getName());
            System.out.println("Email : " + student.getEmail());
                System.out.println();
            }
        } catch (ParseException e) {
            e.printStackTrace();
        }}}
```

3.6.2 *JSON với Twitter*

Chúng ta hiểu thêm về API twitter tại trang:

https://dev.twitter.com/overview/documentation. Khi muốn lấy dữ liệu từ twitter chúng ta cần có API Key (gồm Consumer Token và Access Key) tạo 1 lần tại trang: http://themepacific.com/how-to-generate-api-key-consumer-token-access-key-for-twitter-oauth/994/. Giả sử sau khi sinh API key chúng ta thu được kết quả như sau:

```
AccessToken="833677290-lQXUMWWNRWeJRWTX8NYT0Ec7sDqpehOZoFoxlJYA";
AccessSecret = "kSHUlm15Tr8mV2EEnY56z1tUt2YNV2L7OahuWMVjs";
ConsumerKey = "lpJ73sgPmYWiyIUz5qyBQ";
ConsumerSecret = "PLeUQfll9IAwGFMhf10EvYlDqs8up7SGkFLaGCX0";
```

Sau đó viết ứng dụng lấy các tweets từ twitter với truy vấn, ví dụ ở 1 số dạng:

Tìm kiếm sử dụng giao thức HTML dựa trên đường link:

```
http://search.twitter.com/search.json?q=Key&rpp=100&lang=vi&page=1&c
allback=?
```

Với tham số page truyền vào ta sẽ lấy được dữ liệu trên nhiều trang, với 1 trang chứa tối đa 100 tweets và có thể lấy tối đa 15 trang nên số lượng tweets tối đa có thể lấy ở kiểu tìm kiếm này là 1500 tweets.

Sử dụng API Streamming:

```
"https://stream.twitter.com/1.1/statuses/filter.json?delimited=lengt
h&track=key"
```

Với kiểu tìm kiếm này có thể lấy không giới hạn số lượng tweets, nhưng nếu xử lý tiếng việt thì thời gian tìm kiếm rất lâu do phải tìm trên khối dữ liệu lớn.

Chương trình ví dụ tìm các tweets với từ khóa “Iphone”:

```
import org.json.JSONArray;
import org.json.JSONObject;
import org.json.JSONTokener;
```

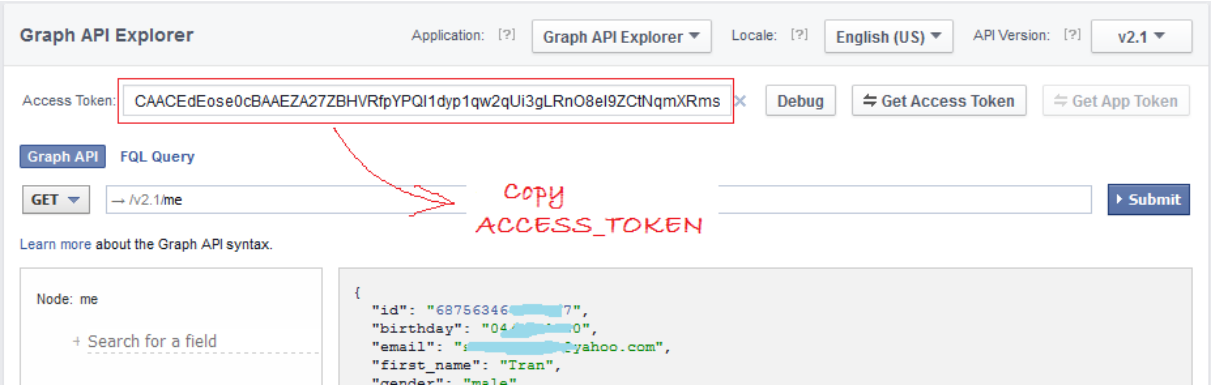
PTIT

```
import Model.Tweet;
public class Tw {
  static String AccessToken = "833677290-
lQXUMWWNRWeJRWTX8NYT0Ec7sDqpehOZoFoxlJYA";
  static String AccessSecret =
"kSHUlm15Tr8mV2EEnY56z1tUt2YNV2L7OahuWMVjs";
  static String ConsumerKey = "lpJ73sgPmYWiyIUz5qyBQ";
  static String ConsumerSecret =
"PLeUQfll9IAwGFMhf10EvYlDqs8up7SGkFLaGCX0";
      public static void main(String[] args) throws Exception {
            /* Streamming API */
            OAuthConsumer consumer = new CommonsHttpOAuthConsumer(
                  ConsumerKey,
                  ConsumerSecret);
            String
url="https://stream.twitter.com/1.1/statuses/filter.json?track=";
        consumer.setTokenWithSecret(AccessToken, AccessSecret);
        String keyword="Iphone";
        keyword = URLEncoder.encode(keyword, "UTF-8");
        System.out.println(keyword);
        HttpGet request = new HttpGet(url+keyword);
        consumer.sign(request);
        HttpClient client = new DefaultHttpClient();
        HttpResponse response = client.execute(request);
        int statusCode = response.getStatusLine().getStatusCode();
        System.out.println(statusCode + ":" +
response.getStatusLine().getReasonPhrase());
        InputStream instream = response.getEntity().getContent();
        JSONTokener jsonTokener = new JSONTokener(new
InputStreamReader(instream, "UTF-8"));
            int count=0;
            while (true){
             count++;
             JSONObject jsonObject = new JSONObject(jsonTokener);
             String id =jsonObject.getString("id");
             String text=jsonObject.getString("text");
             Tweet t = new Tweet(id,text);
             System.out.println(id+":"+text);
            } }}
```

PTIT

3.6.3 *JSON với facebook*

Trước hết Facebook coi các mối hệ giữa các thực thể như là một *"Đồ thị xã hội"* (Social Graph). Và họ xây dựng ra một API, gọi là "Facebook Graph API", API này cho phép lấy ra thông tin, sửa đổi thông tin của các thực thể, chẳng hạn như tải ảnh, viết comment, lấy ra danh sách bạn bè,... API này cơ sở trên HTTP như vậy nó cho phép bất cứ một ngôn ngữ nào có thư viện HTTP đều có thể làm việc với nó, trong đó có Java. Restfb là một Java API cho phép bạn tương tác với Facebook Graph API. Lấy Restfb từ http://restfb.com. Muốn lấy Access token và id của người dùng tại https://developers.facebook.com/tools/explorer.



Hình 4.7: Lấy Access token và id của người dùng trên Facebook

Chú ý: ACCESS_TOKEN trên *"Facebook Graph Explorer"* không tồn tại lâu, nó sẽ hết hạn trong khoảng thời gian nào đó, khi đó chúng ta cần phải copy lại. Trong ứng dụng chúng ta có thể lấy giá trị Access_Token một cách tự động tại thời điểm runtime.

CASE STUDY 2 (JAVASCRIPT & XML)

Các em lựa chọn một trong các lựa chọn sau đây:

Mức điểm từ 5-6 điểm:

1. Nghiên cứu và viết một số hiệu ứng cho site dùng JQuery (không quá lạm dụng làm rối trang web mà phải hợp lý và có sáng tạo
2. Tạo database cho trang web (thiết kế và cài đặt)

Mức điểm từ 5-8 điểm:

3. Viết bằng javascript:
   - Hợp lệ tất cả các form trong trang web của mình (validationn form) cả phía client-side và server-side, ví dụ: định dạng tên, mật khẩu, ngày tháng, mã sản phẩm, số, một số ràng buộc (nếu có, ví dụ ngày đi lớn hơn ngày đến, ngày thanh toán lớn hơn ngày đặt hàng …)….
   - Viết các hàm thấy cần cho trang web của mình (ví dụ: hàm tách số 2.135.000 hoặc 2,135,000. Hoặc hàm chuẩn hóa đoạn văn bản, các công tính toán trong trang web như tính tiền, tổng tiền, trừ thời gian, trả về tháng, quí, các loại doanh thu……
   - Viết một số hiệu ứng cho trang web của mình thêm sinh động 1 cách hợp lý.
   - Dùng thư viện bằng javascript để viết phần thống kê trong module admin bằng biểu đồ, tham khảo tại : http://www.highcharts.com/demo

Mức điểm từ 6-10 điểm

4. Làm các việc sau (cho văn bản tiếng Việt)

PTIT

- Lấy các đánh giá (reviews) của 1 sản phẩm cụ thể nào đó (điện thoại hoặc máy tính hoặc ca sĩ hoặc khách sạn….) từ trên mạng về (trang báo hoặc blog hoặc forum hoặc mạng xã hội….) lưu lại (text hoặc xml).
- Viết chương trình xử lý dữ liệu đó cho chuẩn hơn (nếu cần)
- Viết chương trình tách từ (Tokenizer), gán nhãn từ loại (POS tagger) – dùng tool có sẵn lấy từ trên mạng về, ví dụ Tool của thầy Lê Hồng Phương: http://mim.hus.vnu.edu.vn/phuonglh/softwares
  Hoặc Tool: http://vlsp.vietlp.org:8080/demo/?page=resources

5. Làm các việc sau (cho văn bản tiếng Việt)
   - Lấy các comments trên mạng xã hội (twitter hoặc facebook) lưu lại (text hoặc xml).
   - Viết chương trình xử lý dữ liệu đó cho chuẩn hơn (nếu cần)
   - Viết chương trình tách từ (Tokenizer), gán nhãn từ loại (POS tagger) – dùng tool có sẵn lấy từ trên mạng về (ví dụ Tool của thầy Lê Hồng Phương: http://mim.hus.vnu.edu.vn/phuonglh/softwares
     Hoặc Tool: http://vlsp.vietlp.org:8080/demo/?page=resources
6. Làm các việc sau:
   - Lấy dữ liệu từ các mạng media (youtube hoặc instagram hoặc flick) gồm: ảnh, tiêu đề, số lượng like, số lượng view…..
   - Tìm hiểu bag-of-visual-word
     https://www.youtube.com/watch?v=iGZpJZhqEME
   - Tìm hiểu và cài đặt tool đồ họa surf (Speeded-up Robust Features) cho lưu trữ ảnh

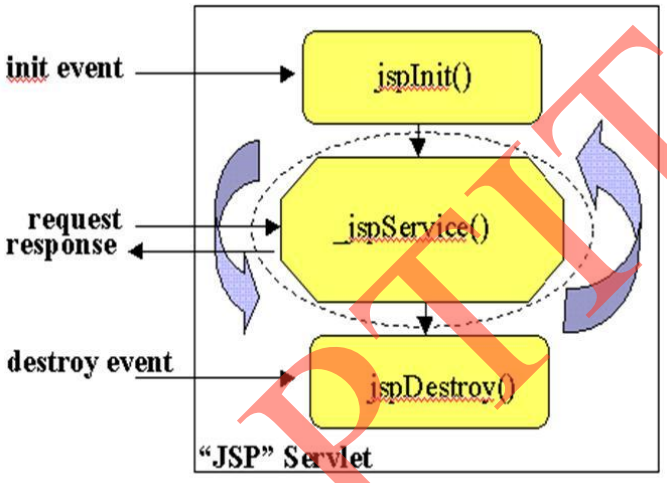
PTIT

CHƯƠNG 5: SỬ DỤNG CÁC THẺ JSP

1. GIỚI THIỆU JSP (JAVA SERVER PAGES)

Jsp được sử dụng để tạo web động (dynamic web), trong ứng dụng web đa tầng (multi layered web app), JSP được sử dụng làm tầng trình diễn. Nó cũng có khả năng tương tác với tầng ứng dụng (application layer) và tạo ra web động dựa trên business logic.

Khi JSP được gọi lần đầu, nó được trình biên dịch phân tích và biên dịch thành lớp servlet. Nếu biên dịch thành công, lớp jsp servlet này sẽ được nạp vào bộ nhớ. Trong lần gọi tiếp theo, lớp servlet này đã ở sẵn trong bộ nhớ, tuy nhiên nó có thể được update. Bởi vậy, trình biên dịch sẽ so sánh timestamp của jsp servlet với jsp. Nếu jsp mới hơn thì nó sẽ biên dịch lại. Vì vậy ta có thể cảm thấy mỗi khi vào trang jsp lần đầu bao giờ cũng thấy chậm hơn các lần sau.



Hình 5.1: vòng đời của JSP

- Translation: JSP được dịch thành JSP servlet với 3 phương thức: jspInit(), _jspService() và jspDestroy(). Ví dụ: *public class SimplePage_jsp extends HttpJspBase.*
- Compilation: JSP servlet được biên dịch sang class
- Thực thi các phương thức init() và service() và chuyển về dạng html để hiển thị lên browser.

Biểu thức (Expresssion)

<%=3+2%> viết ra giá trị 5

Scriptlets: đặt trong thẻ <% %>

```
<% String a= "abc";
      int i = 0;
      out.println("This is scriptlets sample"); %>
```

PTIT

Khai báo (Declarations)

Khai báo cho phép chúng ta định nghĩa các phương thức (method) và biến, mà chúng ta có thể sử dụng ở bất cứ đâu trong trang jsp.

```
<%@page import="java.util.Calendar"%>
<%!  String getSystemTime() {
        return Calendar.getInstance().getTime().toString();  }
%>
<%
        out.println("Current Time: " + getSystemTime());
%>
```

Chú thích (Comments)

Server side: `<%-- --%>` hoặc `<% //  %>`

Client side: `<!-- -->`

Điều hướng (Directives)

Điều hướng là chỉ dẫn tới JSP container chứa thông tin chỉ dẫn JSP container phải biên dịch JSP thành servlet tương ứng. Có 3 dạng thẻ điều hướng.

Bảng 5.1: Thẻ điều hướng

Directives	**Miêu tả**
<%@ page %>	Định nghĩa các thuộc tính thuộc trang, chẳng hạn như error, import, buffer, session,...
<%@ include %>	Sử dụng để nhúng một file vào JSP tại thời điểm biên dịch từ JSP ra servlet
<%@ taglib %>	Sử dụng để khai báo các thẻ JSP mở rộng, hoặc các thẻ tùy biến

Bảng 5.2: Các thuộc tính của thẻ trang <%@ page %>

Attribute	**Kiểu giá trị**	**Giá trị mặc định**
Language	Xác định ngôn ngữ lập trình được sử dụng trong các trang JSP.	language = "java"
contentType	Định nghĩa mã hóa ký tự.	contentType="text/html; charset=UTF8"
Extends	Chỉ định class Servlet cha, mà servlet được sinh ra từ trang JSP này phải mở rộng từ nó.	extends = "mypackage.MyServlet"
Import	Chỉ định rõ danh sách các package, hoặc các class được import, sử dụng trong trang JSP này, cách nhau bởi dấu phẩy.	import="java.utils.*,java.io.InputStream"
Info	Định nghĩa một String có thể được truy cập bởi phương thức getServletInfo() của servlet.	info = "Login page"
Buffer	Chỉ định mô hình đệm (buffering) cho luồng đầu ra của servlet.	8192
autoFlush	Điều khiển hành vi của bộ đệm (buffer) đầu ra servlet.	"true"
Session	Xác định có hay không trang JSP tham gia	"true"

PTIT

	vào HTTP Session.	
isThreadSafe	Xác định mô hình luồng (threading) cho các servlet được tạo ra.	errorPage="error.jsp"
errorPage	Định nghĩa một URL của một JSP khác sẽ hiển thị khi có lỗi xảy ra trên trang JSP này.	None
isErrorPage	Kiểm tra xem trang JSP này có phải là trang errorPage của một trang nào đó theo khai báo thuộc tính errorPage.	"false"
isELIgnored	Xác định có hay không EL expression trong trang JSP sẽ bị bỏ qua.	
isScriptingEnabled	Cho phép hay không các thẻ kịch bản (scripting elements) được phép sử dụng.	

Ví dụ: **errorPage:** Đây là trang web phát sinh lỗi. Khi phát sinh lỗi sẽ gọi đến trang errorPage.jsp (directive của trang này phải đặt là isErrorPage = true)

```
<%@ page errorPage="errorPage.jsp" %>
<%      int i = 10;
        i = i / 0;
%>
```

Trang hiển thị lỗi errorPage.jsp

```
<%@ page isErrorPage="true"%>
<html> <head> <title>Error Page</title>
</head>  <body>
      <b>Exception:</b>
      <br>
      <%=exception.toString()%>
</body>  </html>
```

Cú pháp thẻ <%@ include %>: <%@ include file = "URLname" %>

Trong đó các tệp có thể là JSP, html, text hoặc mã java.

Ví dụ: một trang web được chia thành các tệp Header.html, Logo.html, Content.html, Footer.html sau đó chúng ta ghép các tệp đó lại trong tệp index.jsp.

```
<%@page contentType="text/html" pageEncoding="windows-1252"%>
<html> <head>
<meta http-equiv="Content-Type" content="text/html; charset=windows-
1252"> <title>JSP Page</title>
   <link href="default.css" rel="stylesheet" type="text/css" />
    </head><body>
        <div id="wrapper">
           <%@include file="header.html" %>
           <%@include file="logo.html" %>
           <%@include file="content.html" %>
           <%@include file="footer.html" %>
       </div>
    </body> </html>
```

PTIT

2. CÚ PHÁP JSP

2.1 Các toán tử trong ngôn ngữ jsp

Toán tử số học: **+, –, *, /, %**

Toán tử quan hệ: **== , != , < , > , <= , >=**

Toán tử logic: **&&, ||, !**

Toán tử empty: nếu đối tượng có giá trị là null thì trả về true và ngược lại, cú pháp:

```
${empty x}
```

Biểu thức điều kiện (conditionally expression)

```
${test ? expression1 : expression2 }
```

Ví dụ: `${1==2 ? "aaa" : "bbb"}`

2.2 Các câu lệnh trong ngôn ngữ jsp

Giống hệt java các câu lệnh như sau:

- Điều kiện:
  - `IF (điều kiện) {câu lệnh;}`
  - `IF (điều kiện) {câu lệnh;} ELSE {câu lệnh; }`
- Bảng chọn:
  -
```
switch (điều kiện) {
        case Value 1:
          câu lệnh 1;
          break;
        case Value 2:
          câu lệnh 2;
          break;
          ……
        default:
          câu lệnh default;
    }
```
- Vòng lặp có bước lặp xác định:

```
for( ; ; ) {
câu lệnh;
}
```

- Vòng lặp với điều kiện:
  - `While (điều kiện)`
  - `Do - While (điều kiện)`
  - `Break`
  - `Continue`

PTIT

3. ĐỐI TƯỢNG REQUEST VÀ RESPONSE

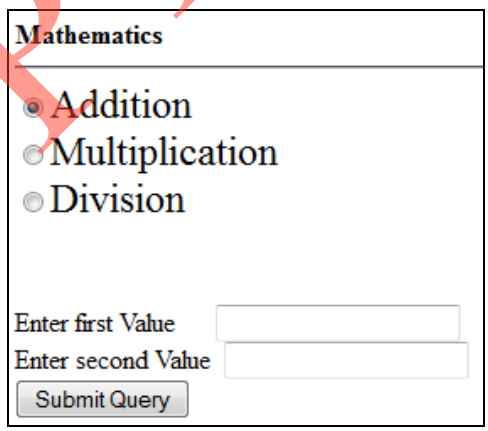
3.1 Đối tượng Request

Khi muốn lấy giá trị từ một thẻ nhập liệu đưa lên từ form (submit) hay từ chuỗi QueryString, có nghĩa là lấy giá trị từ client gởi lên server. Đối với trường hợp này ta sử dụng đối tượng Request. Đối tượng này thuộc lớp javax.servlet.ServletRequest.

Request với thẻ nhập liệu:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html> <html>
<title>Sample Example </title> <body>
   <b> Mathematics</b> <hr>
   <form method="post" action="a.jsp">
   <font size=5 face="Times New Roman">
   <input type="radio" name="a1" value="add" checked> Addition
    </input><br>
    <input type="radio" name="a1" value="mul" >Multiplication
    </input><br>
    <input type="radio" name="a1" value="div" >Division</input><br>
</font> <br><br>
Enter first Value      
    <input type="text" name="t1" value=""><br>
Enter second Value  
    <input type="text" name="t2" value=""><br>
    <input type="submit" name="result">
    </form> </body>  </html>
```

Kết quả:



Hình 5.2: Form nhập liệu

Khi người sử dụng gọi trang JSP này trên trình duyệt và nhập các thông tin yêu cầu như hình trên. Để lấy được giá trị của các phần đã nhập trên hình, chúng ta sử dụng phương thức getParameter của đối tượng request. Tệp a.jsp như sau:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html> <html> <body>
<H1><center>Result for <%=request.getParameter("a1")%></center></H1>
<%  int i=Integer.parseInt(request.getParameter("t1"));
```

PTIT

```
    int j=Integer.parseInt(request.getParameter("t2"));
    int k=0;
    String str=request.getParameter("a1");
    if(str.equals("add"))
         k=i+j;
    if(str.equals("mul"))
        k=i*j;
    if(str.equals("div"))
       k=i/j;
%>
Result is <%= k%>
</body> </html>
```

Request từ tham số trên QueryString

Giả sử khi chúng ta hiển thị ra danh sách các sản phẩm, khi trỏ vào sản phẩm chúng ta tạo liên kết đến chi tiết sản phẩm ta viết như sau:

```
<a href="sachchitiet.jsp?id=<%=b.getId()%>"><img
src="<%=b.getImage()%>"width="120px" height="150px"/></a><br/>
```

Tức là ta viết thêm vào dòng lệnh mã của sản phẩm: id=<%=b.getId()%>">, sau đó tại tệp sachchitiet.jsp chúng ta lấy id bằng lệnh: request.getParameter("id").

Request với mảng tham số

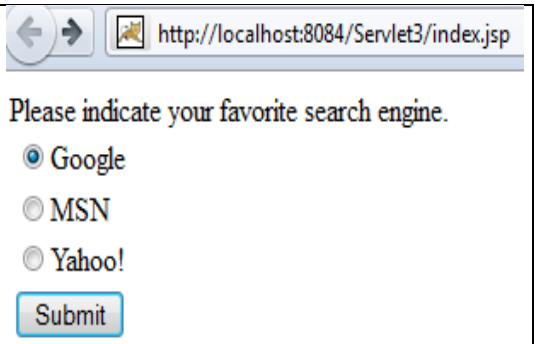
Nếu như có nhiều thẻ trên form hay tham số trên QueryString cùng tên, chúng ta phải sử dụng phương thức getParameterValues() của đối tượng request. Ví dụ, trường hợp chúng ta khai báo trang JSP có nhiều thẻ input dạng checkbox cùng tên (giả sử sothich) nhưng khác giá trị, ta lấy giá trị bằng lệnh:

```
String[] request.getParameterValues("sothich");
```

3.2 Đối tượng Response

Khi muốn trả giá trị từ server xuống client thì sử dụng đối tượng response. Đối tượng này thuộc lớp javax.servlet.ServletResponse. Ta có phương thức sendRedirect() thuộc đối tượng này dùng để chuyển trang. Xét ví dụ chúng ta có tệp index.jsp như sau:

```
<form method="post"
action="Redirection.jsp">
      Please indicate your favorite
search engine.
<input type="radio"
name="searchEngine"
value="http://www.google.com">Google
<input type="radio"
name="searchEngine"
value="http://www.msn.com">MSN
<input type="radio"
name="searchEngine"
value="http://www.yahoo.com">Yahoo!
<input type="submit" value="Submit"
```



Hình 5.3: Kết quả hiển thị

PTIT

```
/>
</form>
```

Tệp Redirection.jsp như sau

```
String url = request.getParameter("searchEngine");
 if (url != null){
       response.sendRedirect(url);  }
 else{
       out.println("No search engine was selected.");}
```

4. CÁC ACTIONS CHUẨN CỦA JSP

Các Actions chuẩn (Standard Actions) là các hành động được xây dựng sẵn trong JSP, nó không cần phải được khai báo với một chỉ thị "taglib". Tất cả chúng trong JSP có một tiền tố mặc định "jsp".

Bảng 5.3: danh sách các hành động chuẩn trong JSP

Cú pháp	**Mục đích sử dụng**
jsp:include	Dùng để nhúng một file vào trang JSP tại thời điểm Request. ***(Chú ý:** khác với Directive inclure là nhúng file vào JSP tại thời điểm biên dịch JSP ra Servlet).*
jsp:useBean	Tìm hoặc khởi tạo một đối tượng JavaBean.
jsp:setProperty	Sét thuộc tính cho JavaBean
jsp:getProperty	Trèn thuộc tính của JavaBean vào đầu ra (Output)
jsp:forward	Chuyển hướng request tới một trang khác.
jsp:plugin	Tạo ra mã trình duyệt cụ thể tạo thẻ OBJECT hoặc EMBED cho các plugin Java.
jsp:element	Định nghĩa các thành phần XML động. (XML elements)
jsp:attribute	

PTIT

4.1 <jsp:include >

Cho phép chúng ta nhúng một trang vào JSP tại thời điểm request. Khác với sử dụng điều hướng <%@ include %> nhúng một tệp vào JSP tại thời điểm biên dịch JSP thành Servlet. Ta có tệp dateTime.jsp như sau

```
<%@ page import="java.util.Date,java.text.*"%>
<%
 Date now = new Date();
 DateFormat df = new SimpleDateFormat("dd-MM-yyyy HH:mm:ss
SSS");
%>
<h4>Current Time:</h4>  <%=df.format(now)%>
```

Và tệp jspIncludeDemo.jsp như sau:

```
<html> <head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Action jsp:include</title> </head> <body>
  <h2>JSP Action jsp:include demo:</h2>
  <jsp:include page="dateTime.jsp"></jsp:include>
</body>  </html>
```

4.2 JavaBean

JavaBean: nói một cách đơn giản chỉ là một class thông thường trong đó các thuộc tính được lưu trữ và truy xuất bằng các phương thức getter và setter. JSP truy cập JavaBean qua các thẻ action và nhận kết quả trả về mà không cần biết cấu trúc của JavaBean và cách thức xử lý của nó. Một vài quy tắc chung khi xây dựng lớp java bean:

- Có một hàm khởi tạo không tham số.
- Có các hàm getter và setter cho việc truy cập thuộc tính.

Khai báo

```
<jsp:useBean id =“định danh” class =“tên class”[scope=“loại scope”]/>
```

Trong đó:

- id: tương tự khai báo biến, đại diện cho đối tượng truy xuất đến Bean
- class: tên class được sử dụng
- scope: khả năng chia xẻ hay tầm hoạt động của Bean. Mặc định là request

Các giá trị của thuộc tính scope:

- *Page*: Bean với giá trị này sẽ có sẵn từ javax.servlet.jsp.PageContext trong trang hiện hành. Bean sẽ được loại bỏ khi một response gởi trở về client hoặc request được chuyển tới một trang mới.
- *Request*: Bean có giá trị này thì có sẵn từ đối tượng ServletRequest của trang hiện hành và bất kỳ trang được include hay forward nào bằng cách sử dụng phương thức getAttribute(name). Bean sẽ bị huỷ bỏ khi một response gửi trả về client.
- *Session*: Bean được sử dụng trong bất kỳ trang nào có tham gia vào một session của client. Có phạm vi trong cả một session của client. Bean này được lưu trữ trong đối tượng HttpSession.
- *Application*: Bean được sử dụng trong bất kỳ trang nào trong ứng dụng hiện hành (cả request hiện tại và tương lai).

Cách khai báo biến dùng scriptlet tương đương Java Bean:

```
<% <tên class> <định danh> = new <tên class>(); %>
```

Ví dụ:

```
<jsp:useBean id=“book1” class=“store.book”/> tương đương
<% store.book book1 = new store.book(); %>
```

PTIT

Cách truy cập thuộc tính bean:

- <jsp:getProperty name =“<định danh>” property =“<tên thuộc tính>” />
  - name: tên id đã được xác định trong <jsp:useBean>
  - property: tên thuộc tính tương ứng khai báo trong Java Bean
- Cách khai báo biến dùng scriptlet tương đương Java Bean
- `<%= <định danh>.phương thức get() %>`

Ví dụ:

```
<jsp:getProperty name="book1" property="title"/> tương đương
<%= book1.getTitle()%>
```

Cách thiết lập thuộc tính của bean:

- <jsp:setProperty name =“<định danh>” property =“<tên thuộc tính>” value=“<giá trị>” />
- name: tên id đã được xác định trong <jsp:useBean>
- property: tên thuộc tính tương ứng khai báo trong Java Bean
- value: giá trị (JSP expression, hằng số) được gán cho thuộc tính
- Các khai báo tương đương trong scriptlet:
  - `<% <định danh>.phương thức set(<giá trị>) %>`

Ví dụ:

```
<jsp:setProperty name="book1" property="title" value="JSP Book" />
tương đương  <% book1.setTitle("JSP Book")%>
```

Ví dụ: viết chương trình các phép tính số học (+,-,*,/)

Tệp CalcBean.java

```
public class CalcBean implements
java.io.Serializable{
private int operand1 = 0 ;
private int operand2 = 0 ;
private double result = 0 ;
private String operation = "" ;
//No-arg constructor for bean....
public CalcBean() { }
public void setOperand1( int op1 ) { //4
operand1 = op1 ;}
public int getOperand1() { //5
return operand1 ;}
…….
private double performTheOperation( ) {
double aResult = 0.0 ;
if ( operation.equals("+") )
aResult = operand1 + operand2 ;
else  if ( operation.equals("-") )
```

PTIT

```
aResult = operand1 - operand2 ;
else if ( operation.equals("*") )
aResult = operand1 * operand2 ;
else if ( operation.equals("/") )
aResult = operand1 / operand2 ;
return aResult ;}}
```

Tệp index.jsp:

```
<%@page contentType="text/html" pageEncoding="windows-1252"%>
<html><head> <title>JSP Sample Page - Calculator Bean</title>
</head>
<body bgcolor="#dddddd">
<center> <h1>A Simple Calculator</h1>
<form name="calcform" action="calculate.jsp" method="POST" >
<center><p>Enter Operand1 and Operand2 (Integers)
<br>and Select an operation From the Pull Down
<br>Menu, then Click <b>Calculate</b> to Continue
<br><hr width="50%">
<table><tr>
<td><P>Enter Operand1:</td>
<td><input type="text" name="operand1" value=""
width="25"></td></tr>
<tr> <td><P>Enter Operand2:</td>
<td><input type="text" name="operand2" value=""
width="25"></td></tr>
<tr><td><P>Select Operation:</td>
<td><select NAME="operation">
<option> +
<option> -
<option> *
<option> :   </select>  </td>
<tr> <td><input type="submit" name="Calculate"
value="Calculate"></td> </tr>
</table>
<hr width="50%"> </center>
</form></body> </html>
```

Tệp `calculate.jsp`:

```
<jsp:useBean id="CalcBean" class="bean.CalcBean" />
<jsp:setProperty name="CalcBean" property="*" />
<p>Operand1 <b><jsp:getProperty name="CalcBean"
property="operand1" /></b>
<p>Operand2 <b><jsp:getProperty name="CalcBean"
property="operand2" /></b>
<p>Operation <b><jsp:getProperty name="CalcBean"
property="operation" /></b>
<p>Result <b><jsp:getProperty name="CalcBean"
property="result" /></b>
```

4.3 <jsp:element>, <jsp:attribute> và <jsp:body>

PTIT

Trong một số trường hợp chúng ta muốn trả dữ liệu xml về phía người dùng thay vì dữ liệu HTML. Việc sử dụng kết hợp *<jsp:element>*, *<jsp:attribute>*, *<jsp:body>* làm cho code dễ nhìn hơn, hãy xem ví dụ sau với việc sử dụng kết hợp 3 thẻ nói trên.

Tệp Employee.java như sau:

```
public class Employee {
    private String empNo;
    private String empName;
    public Employee() {}
    public String getEmpNo() {
        return empNo;}
    public void setEmpNo(String empNo) {
        this.empNo = empNo;}
    public String getEmpName() {
        return empName;}
    public void setEmpName(String empName) {
        this.empName = empName;
    }}
```

Và tệp jspElementDemo.jsp:

```
<?xml version="1.0" ?>
<%@ page pageEncoding="UTF-8"%>
<jsp:element name="data">
    <h3>Please view source of this page</h3>
<%-- Tạo đối tượng Employee và sét giá trị cho các field của nó --%>
    <jsp:useBean
id="emp"class="org.o7planning.tutorial.jsp.beans.Employee">
     <jsp:setProperty name="emp" property="empNo" value="E01" />
     <jsp:setProperty name="emp" property="empName" value="Smith" />
    </jsp:useBean>
    <jsp:element name="employee">
        <jsp:attribute name="empNo" trim="true">
            <jsp:getProperty name="emp" property="empNo" />
        </jsp:attribute>
        <jsp:body>
            <jsp:getProperty name="emp" property="empName" />
        </jsp:body>
    </jsp:element>
</jsp:element>
```

PTIT

5. KẾT NỐI CSDL

5.1 Các bước làm việc với CSDL

JDBC (Java DataBase Connectivity) là API giúp thực hiện việc kết nối với các hệ quản trị cơ sở dữ liệu.

- Để kết nối vào cơ sở dữ liệu cần phải cài đặt JDBC driver.
- Mỗi hệ quản trị cơ sở dữ liệu có một driver riêng [1]

Các bước khi làm việc với cơ sở dữ liệu

- Bước 1: Loading driver ứng với hệ cơ sở dữ liệu xây dựng
- Bước 2: Thiết lập 1 kết nối
- Bước 3: tạo tập lệnh
- Bước 4: thực thi lệnh SQL
- Bước 5: Xử lý ResultSet (nếu có)
- Bước 6: đóng kết nối

Bước 1: Loading các driver gồm

- Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver"*);(SQL server)*
- Class.forName("com.mysql.jdbc.Driver"); *(Mysql)*
- Class.forName("com.oracle.jdbc.OracleDriver"); *(Oracle)*

Bước 2: Thiết lập 1 kết nối

Khai báo đối tượng Connection bằng cách sử dụng phương thức DriverManager.getConnection. Cú pháp có dạng:

Khi tập cơ sở dữ liệu BookStore tạo bởi SQL server:

```
String url =
"jdbc:sqlserver://localhost:1433;databasename=BookStore";
Connection con = DriverManager.getConnection(url, "sa", "123456");
```

Khi tập cơ sở dữ liệu BookStore tạo bởi mySQL :

```
String url="jdbc:mysql://localhost:3306/ BookStore "+"root"+
"password"
Connection  con = DriverManager .getConnection(url);
```

Bước 3: Tạo tập lệnh

Có 3 dạng lệnh:

- PreparedStatement prepareStatement(String sql) throws SQLException
- CallableStatement prepareCall(String sql) throws SQLException
- Statement createStatement() throws SQLException

Tạo đối tượng Statement:

```
Statement st = con.createStatement();
```

Hoặc:

```
Statement st = null;
st = con.createStatement();
```

Đối tượng PreparedStatement:

```
PreparedStatement prepareStatement(String sql);
```

Bước 4: Thực thi lệnh SQL

Sau khi tạo statement thì sử dụng các phương thức để thực thi các câu lệnh sql tương ứng.

- Phương thức execute(String query) sử dụng để thực thi bất kì câu lệnh SQL nào.

PTIT

- Phương thức executeUpdate() để cập nhật dữ liệu qua JDBC, phương thức này trả về số dòng bị ảnh hưởng sau khi câu lệnh thực thi.
- Phương thức executeQuery() dùng để đọc dữ liệu từ cơ sở dữ liệu. Dữ liệu được đọc sẽ được lưu lại trong một *resultset.*

Bước 5: xử lý Resultset

Một số phương thức trong resultset:

- Next() di chuyển đến dòng kết tiếp, trả về true nếu có dòng kế tiếp, false nếu đến cuối resultset.
- Previous() di chuyển đến dòng trước đó.
- First() di chuyển đến dòng đầu tiên.
- Last() di chuyển đến dòng cuối cùng.
- Beforefirst() trước dòng đầu tiên.
- Afterlast() sau dòng cuối cùng.
- Absolute (int row) di chuyển đến dòng thứ row.
- getXXX(string columname) lấy dữ liệu từ resultset, XXX là kiểu trả về và columname tên của cột cần lấy dữ liệu.
- updateXXX(string columname, value) cập nhật dữ liệu trong resultset, XXX kiểu giá trị, columname tên của cột cần cập nhật dữ liệu và value: giá trị cần cập nhật.
- Sau đó thực gọi phương thức updaterow() dữ liệu sẽ được cập nhật.
- *Transaction:* là một giao dịch được khai báo bắt đầu bằng phương thức SetAutocommit (true) điều này chỉ ra một tập câu lệnh sql chỉ được thực thi khi resultset gọi phương thức commit. Transaction bị hủy khi conectionstring gọi phương thức rollback

Bước 6: đóng kết nối

```
public void Close() throws SQLException{
    if(rs!=null && rs.isClosed()){
       rs.close();
       rs=null; }
    if(st!=null){
       st.close();
       st=null; }
    if(con!=null && con.isClosed()){
       con.close();
       con=null;
    } }
```

5.2 Một số ví dụ cho cơ sở dữ liệu (CSDL)

Tạo lớp kết nối với CSDL BookStore:

```
Public class SqlConnection {
    public Connection getConnection(){
```

PTIT

```
        Connection conn = null;
        try {
            try {
Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
            } catch (ClassNotFoundException ex) {
Logger.getLogger(SqlConnection.class.getName()).log(Level.SEVERE,
null, ex);           }
String url =
"jdbc:sqlserver://localhost:1433;databasename=BookStore";
            conn = DriverManager.getConnection(url, "sa", "123456");
        } catch (SQLException ex) {
Logger.getLogger(SqlConnection.class.getName()).log(Level.SEVERE,
null, ex);  }
        return conn;
 }  }
```

Một số phương thức thao tác với bảng Books:

Thêm:

```
public void insert(Book book) {
        try {
            conn = sqlConn.getConnection();
            String command = "INSERT INTO Books (NAME,AUTHOR_ID,"
  + "CATE_ID,DESCRIPTION,STATUS,PRICE,IMAGE) VALUES(?,?,?,?,?,?,?)";
            PreparedStatement psmt = conn.prepareStatement(command);
            psmt.setString(1, book.getName());
            psmt.setInt(2,book.getAuthor_id());
            psmt.setInt(3,book.getCate_id());
            psmt.setString(4, book.getDescription());
            psmt.setString(5, book.getStatus());
            psmt.setFloat(6, book.getPrice());
            psmt.setString(7, book.getImage());
            psmt.executeUpdate();
            conn.close();
        } catch (SQLException ex) {
Logger.getLogger(AdminDB.class.getName()).log(Level.SEVERE, null,
ex);     }  }
```

Cập nhật:

```
public void update(Book book){
        try {
            conn = sqlConn.getConnection();
            String command = "UPDATE Books SET NAME=?,AUTHOR_ID=?,"
    + "CATE_ID=?,DESCRIPTION=?,STATUS=?,PRICE=?,IMAGE=? WHERE ID=?";
            PreparedStatement psmt = conn.prepareStatement(command);
            psmt.setString(1, book.getName());
            psmt.setInt(2, book.getAuthor_id());
            psmt.setInt(3, book.getCate_id());
            psmt.setString(4, book.getDescription());
            psmt.setString(5, book.getStatus());
            psmt.setFloat(6, book.getPrice());
            psmt.setString(7, book.getImage());
            psmt.setInt(8, book.getId());
            psmt.executeUpdate();
```

PTIT

```
            conn.close();
        } catch (SQLException ex) {
Logger.getLogger(BookDB.class.getName()).log(Level.SEVERE, null,
ex);
        }  }
```

Xóa:

```
public void delete(int id){
        try {
            conn = sqlConn.getConnection();
            String command = "DELETE FROM Books WHERE ID=?";
            PreparedStatement psmt = conn.prepareStatement(command);
            psmt.setInt(1, id);
            psmt.executeUpdate();
            conn.close();
        } catch (SQLException ex) {
Logger.getLogger(AuthorDB.class.getName()).log(Level.SEVERE, null,
ex);
        }   }
```

Đưa ra danh sách sách:

```
public ArrayList<Book> getBookAll(){
        ArrayList<Book> list = new ArrayList<Book>();
        try {
            conn = sqlConn.getConnection();
            String command = "SELECT * FROM Books ORDER BY ID ASC";
            PreparedStatement psmt = conn.prepareStatement(command);
            ResultSet rs = psmt.executeQuery();
            while(rs.next()){
                Book book = new Book();
                book.setId(rs.getInt(1));
                book.setName(rs.getString(2));
                book.setAuthor_id(rs.getInt(3));
                book.setCate_id(rs.getInt(4));
                book.setDescription(rs.getString(5));
                book.setStatus(rs.getString(6));
                book.setPrice(rs.getFloat(7));
                book.setImage(rs.getString(8));
                list.add(book);  }
            conn.close();
        } catch (SQLException ex) {
Logger.getLogger(AuthorDB.class.getName()).log(Level.SEVERE, null,
ex);
        }
        return list; }
```

Tìm kiếm sách theo mã (ID):

```
public Book getBookById(int id){
        Book book = new Book();
        try {
            conn = sqlConn.getConnection();
            String command = "SELECT * FROM Books WHERE ID=?";
            PreparedStatement psmt = conn.prepareStatement(command);
            psmt.setInt(1, id);
```

PTIT

```
            ResultSet rs = psmt.executeQuery();
            if(rs.next()){
                book.setId(id);
                book.setName(rs.getString(2));
                book.setAuthor_id(rs.getInt(3));
                book.setCate_id(rs.getInt(4));
                book.setDescription(rs.getString(5));
                book.setStatus(rs.getString(6));
                book.setPrice(rs.getFloat(7));
                book.setImage(rs.getString(8));
            }
            conn.close();
        } catch (SQLException ex) {

Logger.getLogger(AuthorDB.class.getName()).log(Level.SEVERE, null,
ex);  }
        return book; }
```

CASE STUDY 3 + 4 (DÙNG JSP)

Làm theo nhóm (dựa trên template + css+ hình ảnh + dữ liệu … đã làm ở case study 1)

1. Module Administrator
   - Phân tích + thiết kế bài toán (tự làm với nhau không cần nộp bản mềm và cứng)
   - Cài đặt CSDL (nộp cô theo chương trình)
   - Phần chương trình thường theo các chủ điểm sau (chọn lấy 1 để làm):
     - Thương mại điện tử (buôn bán cái gì đó)
     - Văn phòng ảo (du lịch, trưng bày tranh…)
     - Cẩm nang, tra cứu….(sức khỏe, ăn uống…)
     - Blog, forum…..
   - Dùng JSP xây dựng module admin :
     - Xây dựng nội dung (nhập mới, sửa, xóa một, xóa nhiều…. Cho các bảng)
     - Tổng hợp, báo cáo, thống kê…..(cả dạng bảng và dạng biểu đồ)
2. Module người dùng
   - Viết trang chủ phía người dùng: liệt kê sản phẩm, liệt kê theo thể loại, nhãn mác…, 1 số thống kê (mới, khuyến mại, liên quan, ….)
   - Tìm kiếm chung, nâng cao…

PTIT

CHƯƠNG 6: GIỚI THIỆU SERVLET

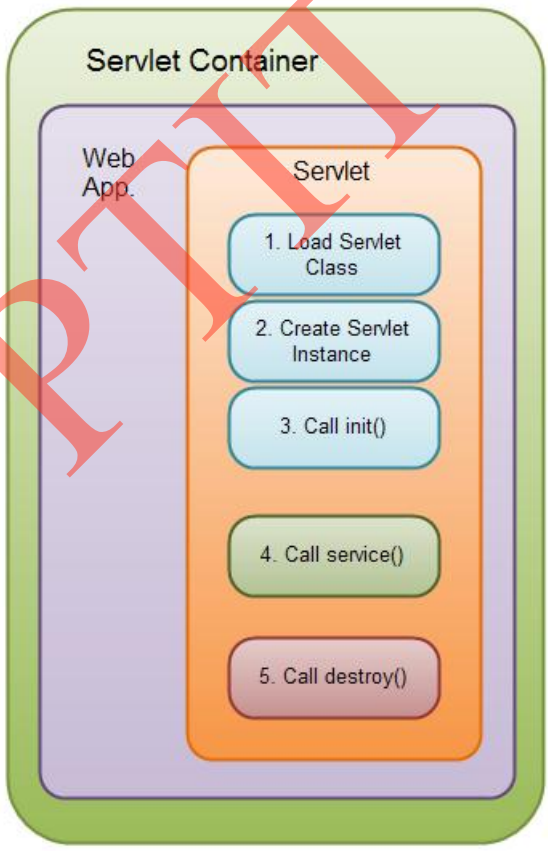
1. GIỚI THIỆU SERVLET

1.1 Servlet là gì

Java Servlets là chương trình chạy trên một Web hoặc ứng dụng máy chủ và hành động như một lớp trung gian giữa một yêu cầu đến từ một trình duyệt Web hoặc HTTP khách khác và cơ sở dữ liệu hoặc các ứng dụng trên máy chủ HTTP. Sử dụng Servlets, bạn có thể thu thập đầu vào từ người dùng thông qua các hình thức trang web, từ một cơ sở dữ liệu hoặc một nguồn khác, và tạo ra các trang web động.

Một servlet thực chất là một lớp Java vì thế cần được thực thi trên một máy ảo Java (JVM) bằng một dịch vụ gọi là servlet engine. Servlet engine tải lớp servlet lần đầu tiên servlet được yêu cầu, hoặc ngay khi servlet engine được bắt đầu. Servlet ngừng tải để xử lý nhiều yêu cầu khi servlet engine bị tắt hoặc nó bị dừng lại.

Vòng đời của một Servlet: hình dưới đây minh họa về vòng đời của nó.



Hình 6.1: vòng đời của một Servlet

Có 5 bước:

1. Tải Servlet Class vào bộ nhớ.
2. Tạo đối tượng Servlet.
3. Gọi method servlets init()

PTIT

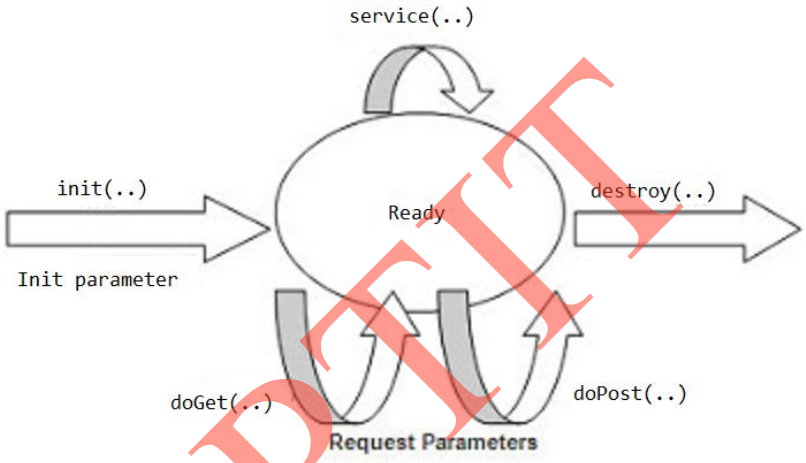
4. Gọi method servlets service().
5. Gọi method servlets destroy().

Bước 1, 2 và 3 được thực thi một lần duy nhất, khi mà servlet được nạp lần đầu. Mặc định các servlet không được tải lên cho tới khi nó nhận một đòi hỏi đầu tiên từ người dùng. Bạn có thể buộc ServletContainer (Bộ chứa các servlet) tải các servlet khi nó khởi động. Bạn có thể xem thêm tại web.xml Servlet Configuration.

Bước 4 được thực thi nhiều lần, mỗi khi có đòi hỏi từ phía người dùng tới servlet.

Bước 5 được thực thi khi bộ chứa servlet (Servlet Container) trút bỏ (unloaded) servlet.

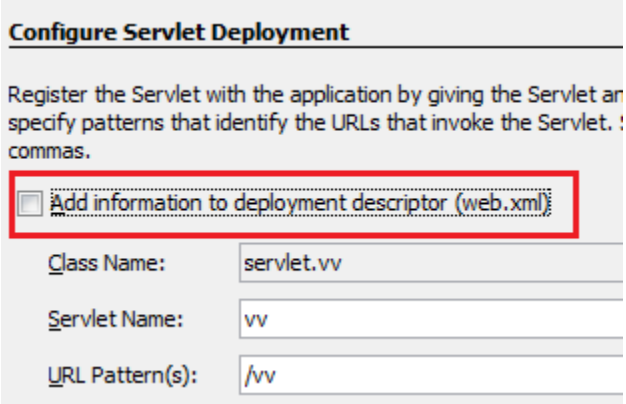
Phương thức service() của servlet được gọi mỗi khi có request từ client đến. Tùy vào các tình huống cụ thể nó gọi đến một trong các phương thức doGet(..), doPost(...). Tại các servlet của bạn, bạn phải ghi đè và sử lý tại các phương thức này.



Hình 6.2: Phương thức service() của servlet

1.2 Tạo dự án Web với servlet

Tạo mới một Servlet: chỉ vào project/ bật chuột phải / new / servlet / đặt tên, xuất hiện màn hình sau:

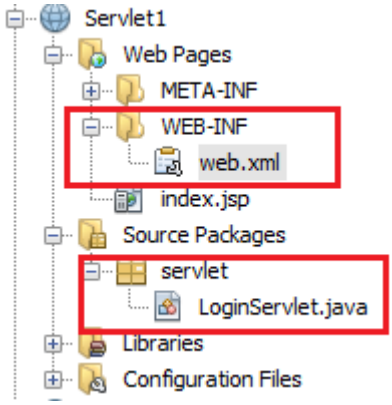


Hình 6.3: tạo một servlet với thông tin mô tả trong tệp web.xml

PTIT

Nếu chúng ta lựa chọn add information to deployment descriptor (web.xml) thì thông tin về servlet vừa tạo sẽ được lưu vào tệp web.xml, còn không chúng ta sẽ phải ghi vào sau.

Một project có chứa servlet sẽ như hình sau:



Hình 6.4: một project có chứa servlet

Ví dụ ta có 1 form như sau:

```
<%@page contentType="text/html"
pageEncoding="windows-1252"%>
<html>  <head> <title>JSP Page</title>
</head><body>
  <h1>Hello World!</h1>
  <form action="LoginServlet" method="post">
  User name: <input type="text"
name="username" value=""><br>
   Password:  <input type="password"
name="password" value="">
   <input type="submit" value="OK" >
 </form>    </body></html>
```



Hình 6.5: đăng nhập

Ta thấy khi chọn submit thì sẽ gọi thao tác đặt trong thuộc tính action của thẻ <Form>. Ở đây chúng ta tạo servlet có tên LoginServlet để xử lý.

Chúng ta tạo 1 servlet LoginServlet như sau (lớp LoginServlet.java):

```
public class LoginServlet extends HttpServlet {
    protected void processRequest(HttpServletRequest request,
HttpServletResponse response)
            throws ServletException, IOException {
        PrintWriter out = null;
        try {
            response.setContentType("text/html");
            out = response.getWriter();
            out.println("<html>");
            String result = new String("");
            String strusername = request.getParameter("username");
            String strpassword = request.getParameter("password");
            boolean flag=true;
         if (( strusername== null) || (strusername.length() == 0)) {
                result = "Name Khong hop le, ";
```

PTIT

```
                flag=false;
            }
    else if (( strpassword== null) || (strpassword.length() == 0)) {
                result = "Pass Khong hop le";
                flag=false;
            }
            if(flag==true){
    out.println("<h1> Username: "+ strusername + "</h1>"+"<br>");
            out.println("<h1> Password: "+ strpassword + "</h1>");
            }
            else
                out.println("<h1>"+result+"</h1>");
        } catch (IOException ex) {
Logger.getLogger(LoginServlet.class.getName()).log(Level.SEVERE,
null, ex);
        } finally {
            out.close();
        } }
protected void doPost(HttpServletRequest request,
HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    } }
```

2. CÁC ĐỐI TƯỢNG REQUEST VÀ RESPONSE

2.1 Đối tượng request

Lấy dữ liệu từ web client gởi đến bằng servlet ta dùng đối tượng của lớp HttpServletRequest. Các phương thức để lấy thông số gồm: getParameter("para-name"), getParameterValues("para-name") và getParameterNames(). Ví dụ:

```
String username= request.getParameter("username");
String[] choice = request.getParameterValues("comments");
Enumeration parameter_names = request.getParameterNames();
while(parameter_names.hasMoreElements()){
      String para = parameter_names.nextElement();
      out.print(para + " = ");
      String[] paraValues = getParameterValues(para);
      if(paraValues.lenght()==1){
             out.println(paraValues[0]);
      }else{
             for(int i = 0, i< paraValues.lenght(),i++){
                 out.print(paraValues[i]+ "-");
             }
      }
}
```

Dùng đối tượng của lớp HttpServletRequest để lấy các thông tin HTTP header. Một số phương thức như sau:

String getHeader(header-name): lấy nội dung của header-name

PTIT

Enumeration getHeaderNames(): lấy tất cả các header-name.

Một số method điển hình:

- *Cookie[] getCookies()*: dãy Cookie từ client
- *int getContentLength()*: trả giá trị Content-Length
- *int getContentType()*: trả giá trị Content-Type
- *int getRemoteUser()*: giá trị username nếu có authenticate

2.2 Đối tượng response

Gửi dữ liệu cho web client: chúng ta dùng đối tượng của lớp HttpServletResponse. Đầu tiên tạo đối tượng *PrintWriter* để ghi dữ liệu gửi, ví dụ:

```
PrintWriter out = response.getWriter();
```

Xử lý các mã HTTP trả về với các phương thức của lớp *HttpServletResponse*:

void setStatus(int statusCode): gửi các mã response

void sendError(int errorCode,String msg): gửi mã lỗi theo giao thức HTTP và message

void sendRedirect(String URL): chuyển đến một trang URL khác

Các mã có thể theo giao thức HTTP hoặc dùng các hằng số trong class *HttpServletResponse*.

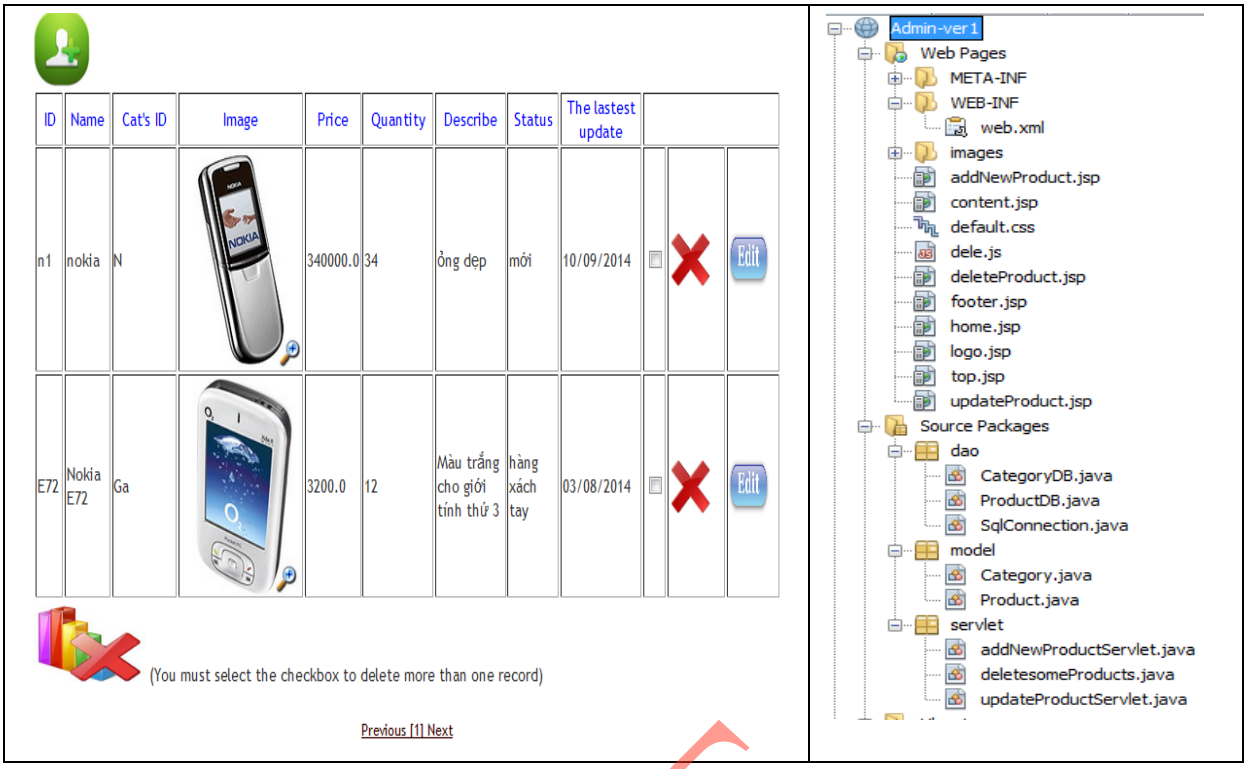
3. Ví dụ cho một số module phía server

Phần xử lý phía admin ở đây gồm: hiển thị danh sách sản phẩm (có phân trang), hiển thị chi tiết 1 sản phẩm, thêm mới, sửa, xóa 1 sản phẩm, xóa nhiều sản phẩm.

Bảng 6.1: Giao diện và project cài đặt

Giao diện	Project

PTIT



Trong bài toán này ta có:

Database: Trading (gồm 2 bảng Products và Categories)

model: Category, Product

dao: SqlConnection.java, CategoryDB.java, ProductDB.java

servlet: addNewProductServlet.java, deletesomeProducts.java, và updateProductsServlet.java

Web: home.jsp, addNewProduct, deleteProduct, updateProduct,…

Phần model:

Product.java (lớp Product)	Category.java (Category)

public class Product {
private String ID;
private String name;
private String cat_ID;
private String image;
private float price;
private int quantity;
private String describe;
private String status;
private String adddate;
public Product() { }…..}
	public class Category {
	private String ID;
	private String name;
	private String description;
	
	public Category() {
	}
	…..
	}

Kết nối CSDL (SqlConnection.java)

```
public class SqlConnection {
    public Connection getConnection(){
        Connection conn = null;
        try {
```

PTIT

```
            try {
Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
            } catch (ClassNotFoundException ex) {
Logger.getLogger(SqlConnection.class.getName()).log(Level.SEVERE,
null, ex);
            }
            String url =
"jdbc:sqlserver://localhost:1433;databasename=Trading";
            conn = DriverManager.getConnection(url, "sa", "123456");
        } catch (SQLException ex) {
  Logger.getLogger(SqlConnection.class.getName()).log(Level.SEVERE,
null, ex);}
        return conn; }}
```

Liệt kê sản phẩm:

Tạo phương thức getProductAll() (trong lớp ProductDB)

```
public ArrayList<Product> getProductAll(){
        ArrayList<Product> list = new ArrayList<Product>();
        try {
            conn = sqlConn.getConnection();
     String command = "select
ID,name,cat_ID,image,price,quantity,describe,status,
convert(char(12),adddate,103) as adddate from products ORDER BY ID
DESC";
            PreparedStatement psmt = conn.prepareStatement(command);
            ResultSet rs = psmt.executeQuery();
            while(rs.next()){
                Product p = new Product();
                p.setID(rs.getString(1));
                p.setName(rs.getString(2));
                p.setCat_ID(rs.getString(3));
                p.setImage(rs.getString(4));
                p.setPrice(rs.getFloat(5));
                p.setQuantity(rs.getInt(6));
                p.setDescribe(rs.getString(7));
                p.setStatus(rs.getString(8));
                p.setAdddate(rs.getString(9));
                list.add(p);      }
            conn.close();
        } catch (SQLException ex) {
Logger.getLogger(ProductDB.class.getName()).log(Level.SEVERE, null,
ex);
        }
        return list; }
```

Phân trang (số sản phẩm nhiều)

```
int i = 0;
int iPage=0;
int RecordsPerPage = 6;// số sản phẩm trên 1 trang
int size = list.size();
  int counter = 0;
    try {         // Select current page
      iPage = Integer.parseInt(request.getParameter("page"));
```

PTIT

```
    }
    catch (NumberFormatException e ) {
      iPage = 0;
    }
    if (iPage == 0) { iPage = 1;}
    counter = (iPage-1)*RecordsPerPage ;
    int iCounter = 0;
while ((iCounter < RecordsPerPage) && counter < size){
              p = list.get(counter);
                        counter++;
                        iCounter++;
                 if(iCounter >= RecordsPerPage)
                   break;
                }
……..……//Navigation
    String sFileName="home.jsp";
    if (counter == 0) {   // Recordset is empty
      out.println("No record");
    }else {
      if ( iPage == 1 ) {
        out.print("<a href=#>Previous</a>");
      }
      else {
out.print("<a href="+sFileName+"?page="+(iPage-1)+">Previous</a>");
      }
         i=1;
         int numpage;
         if (size % RecordsPerPage==0){
            numpage = size/RecordsPerPage;
         } else
             numpage = size/RecordsPerPage+1;
         while(i<=numpage){
        out.print("<a href="+sFileName+"?page="+i+"> ["+i+"] </a>");
            i++;
          }
      if (counter==size) {
        out.print("<a href=#>Next</a>");
      }
      else {
  out.print("<a href="+sFileName+"?page="+(iPage + 1)+">Next</a>");
      }}
```

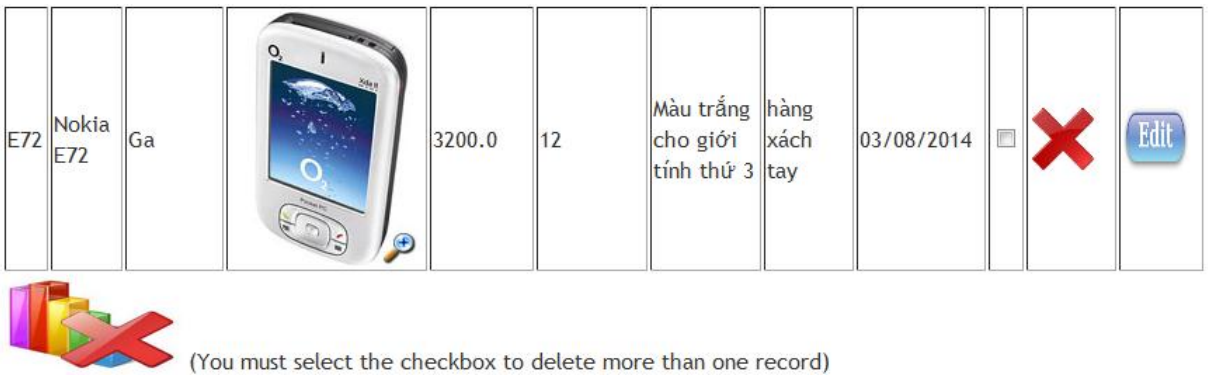
Liệt kê sản phẩm (cho vào bảng):

```
ProductDB productdb = new ProductDB();
  ArrayList<Product> list = productdb.getProductAll(); // danh sách
toàn bộ sản phẩm
  Product p = new Product();
……..
while ((iCounter < RecordsPerPage) && counter < size){
              p = list.get(counter);
              fID = p.getID();
              fname = p.getName();
```

PTIT

```
                fcat_ID=p.getCat_ID();
                fimage = p.getImage();
                fprice=p.getPrice();
                fquantity=p.getQuantity();
                fdescribe = p.getDescribe();
                fstatus=p.getStatus();
                fadddate=p.getAdddate();
…….
<tr>
    <td><%= fID %></td>
    <td><%= fname %></td>
    <td><%= fcat_ID %></td>
    <td><img src="<%= fimage %>" width="139" height="184"/></td>
    <td><%= fprice %></td>
    <td><%= fquantity %></td>
    <td><%= fdescribe %></td>
    <td><%= fstatus %></td>
    <td><%= fadddate %></td>
<td width="3%">
    <label> <!-- Tao 1 checkbox de luu gia tri cua Product id trong
form  -->
    <input type="checkbox" name="chon" value="<%=fID %>" /> </label>
    </td>
    <td width="9%"><a
href="javascript:confirmDelete('deleteProduct.jsp?fID=<%= fID %>')">
    <img src="images/del.jpg" width="48" height="48"></a></td>
    <td width="8%"><a href="updateProduct.jsp?fID=<%= fID %>">
    <img src="images/edit.jpg" width="48" height="48"></a></td>
  </tr>
<tr>
<td colspan="12"><input type="image" name="Button" src="images/
delall.jpg" width="128" height="64" onClick="DeleteLinks();" />
 (You must select the checkbox to delete more than  one record)</td>
  </tr>
```

Hiển thị:

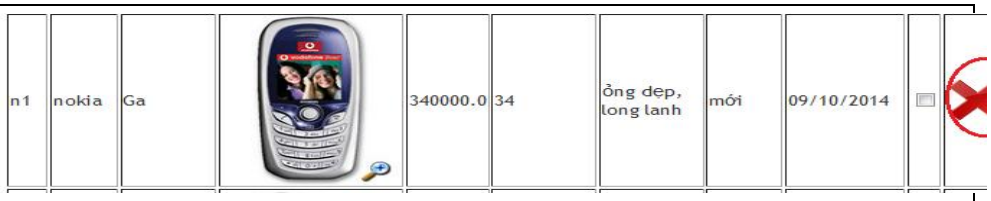


Hình 6.6: Hiển thị 1 sản phẩm

Xóa 1 sản phẩm:

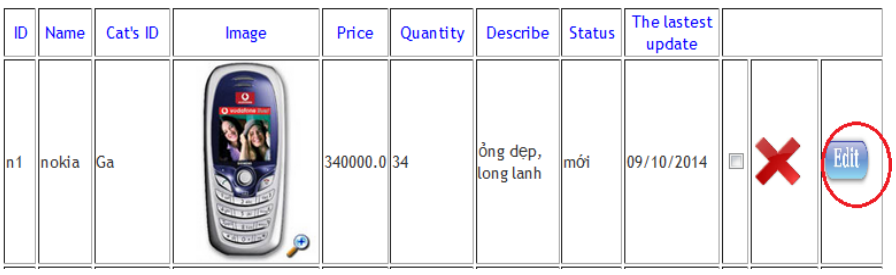
PTIT

Bảng 6.2: Các bước xóa 1 sản phẩm

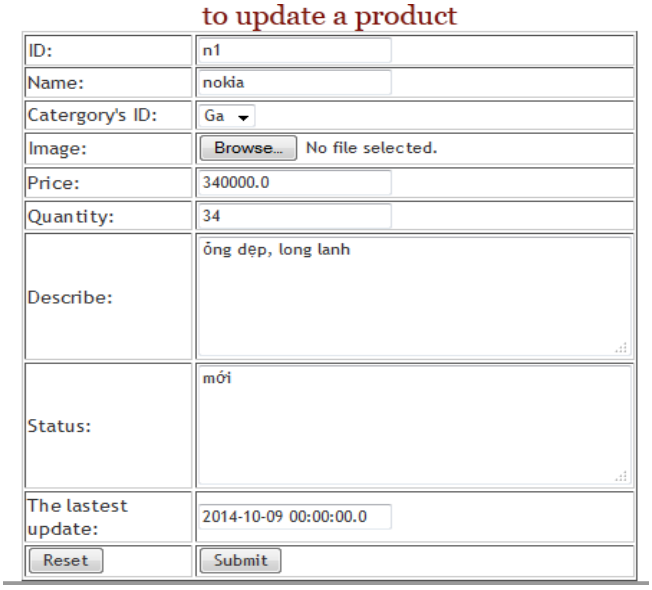
Giao diện	
Code	`<td width="9%"><a href="javascript:confirmDelete('deleteProduct.jsp?fID=<%= fID %>')"><img src="images/del.jpg" width="48" height="48"></a></td>`
Javascript Hỏi xóa?	`<SCRIPT> function confirmDelete(delUrl) { if (confirm("Are you sure to delete")) { window.location.replace(delUrl); } } </SCRIPT>`
CSDL: ProductDB	`public void delete(String id){ try { conn = sqlConn.getConnection(); String command = "DELETE FROM products WHERE ID=?"; PreparedStatement psmt = conn.prepareStatement(command); psmt.setString(1, id); psmt.executeUpdate(); conn.close(); } catch (SQLException ex){ Logger.getLogger(ProductDB.class.getName()).log(Level.SEVERE, null, ex); } }`
Xóa (JSP) deleteProduct.jsp	`<%@page contentType="text/html" pageEncoding="UTF-8"%> <%@page import="dao.ProductDB"%> <% String ID = null; ProductDB p = new ProductDB(); ID = request.getParameter("fID").trim(); p.delete(ID); response.sendRedirect("home.jsp"); %>`

Cập nhật 1 sản phẩm:

Bảng 6.3: Các bước cập nhật 1 sản phẩm

Giao diện	
Code	`<td width="8%"><a href="updateProduct.jsp?fID=<%= fID %>"><img src="images/edit.jpg"`

PTIT

	width="48"height="48"></td>
updateProduct.jsp	
Code	

```
<h2 align="center">UPDATE A PRODUCT</h2>
<form action="updateProductServlet" method="post" name="form1">
<%
    String ID = null;
    String name = null;
    String cat_ID = null;
    String image = null;
    float price = 0;
    int quantity = 0;
    String describe = null;
    String status = null;
    String adddate = null;
        ID=request.getParameter("fID");
        ProductDB productdb = new ProductDB();
        Product p = productdb.getProductbyID(ID);
        name = p.getName();
        cat_ID = p.getCat_ID();
        image = p.getImage();
        price = p.getPrice();
        quantity = p.getQuantity();
        describe = p.getDescribe();
        status = p.getStatus();
        adddate = p.getAdddate();

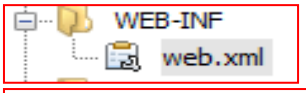
   %>
<input name="fID" type="text" readonly value="<%=ID %>" >

<input type="text" name="fname" id="fname" value="<%=name %>"


<td>Catergory's ID:</td>
<td><label>
  <select name="fcat_ID">
    <%
    CategoryDB cdb = new CategoryDB();
    ArrayList<Category>  lc= cdb.getCategoryAll();
    Category c = new Category();
    String fID = null;
    int i=0;
    for(i=0;i<lc.size();i++){
      c=lc.get(i);
      fID=c.getID();

   %>
    <option value="<%=fID %>"><%=fID %></option>
    <% } %>
```

PTIT

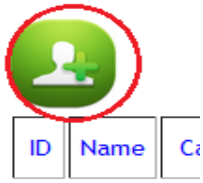
	`<input type="file" name="fimage" id="fimage" value="<%=image %>">`
CSDL: ProductDB.java	`public void update(Product p){ try { conn = sqlConn.getConnection(); String command = "UPDATE products SET [NAME]=?, cat_ID=?,image=?,price=?,quantity=?,describe=?,STATUS=?,adddate=? WHERE ID=?"; PreparedStatement psmt = conn.prepareStatement(command); psmt.setString(1,p.getName()); psmt.setString(2,p.getCat_ID()); psmt.setString(3, p.getImage()); psmt.setFloat(4, p.getPrice()); psmt.setInt(5, p.getQuantity()); psmt.setString(6, p.getDescribe()); psmt.setString(7, p.getStatus()); psmt.setString(8, p.getAdddate()); psmt.setString(9, p.getID()); psmt.executeUpdate(); conn.close(); } catch (SQLException ex) { Logger.getLogger(ProductDB.class.getName()).log(Level.SEVERE, null, ex); } }`
Servlet: updateProductServlet.java	`@Override protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { request.setCharacterEncoding("UTF-8");//tiếng Việt String ID = request.getParameter("fID").trim(); String name = request.getParameter("fname").trim(); String cat_ID = request.getParameter("fcat_ID").trim(); String image = request.getParameter("fimage").trim(); image ="images/"+image; float price = Float.parseFloat(request.getParameter("fprice").trim()); int quantity = Integer.parseInt(request.getParameter("fquantity").trim()); String describe = request.getParameter("fdescribe").trim(); String status = request.getParameter("fstatus").trim(); String adddate = request.getParameter("fadddate").trim(); ProductDB productdb = new ProductDB(); Product p=new Product(); p.setID(ID); p.setName(name); p.setCat_ID(cat_ID); p.setImage(image); p.setPrice(price); p.setQuantity(quantity); p.setDescribe(describe); p.setStatus(status); p.setAdddate(adddate); productdb.update(p); response.sendRedirect("home.jsp"); }`     `<servlet> <servlet-name>updateProductServlet</servlet-name> <servlet-class>servlet.updateProductServlet</servlet-class> </servlet>`

DEF

	<servlet-mapping> <servlet-name>updateProductServlet</servlet-name> <url-pattern>/updateProductServlet</url-pattern> </servlet-mapping>

Nhập mới 1 sản phẩm:

Bảng 6.4: Các bước nhập mới 1 sản phẩm

Hiển thị	
Code	<td colspan="12"><img src="images/add.jpg" width="64" height="64"></td>
Giao diện nhập mới (addNewProduct.jsp)	
Code (addNewProduct.jsp) Lấy danh sách mã Loại sản phẩm	<H2> <% if (request.getAttribute("message")!= null){%> <%=request.getAttribute("message")%> <%}%> </H2> ADD A NEW PRODUCT <form action="addNewProductServlet" method="post" name="form1"> <table width="50%" border="1" align="center">

PTIT

Lấy ngày hiện thời

```
<td>Catergory's ID:</td>
<td><label>
  <select name="cat_ID">
    <%
        CategoryDB cat = new CategoryDB();
        ArrayList<Category> list = cat.getCategoryAll();
        Category c = new Category();
        int i = 0;
        String fcID = null;
        for (i = 0; i < list.size(); i++) {
            c = list.get(i);
            fcID=c.getID();
   %>
    <option value="<%=fcID %>"><%=fcID %></option>
    <% } %>
  </select>
</label></td>
```

```
<td>The lastest update:</td>
<td><label>
  <%java.text.DateFormat df = new java.text.SimpleDateFormat("MM/dd/yyyy"); %>
  <input type="text" name="adddate" id="adddate" value="<%=df.format(new java.util.Date()) %
</label></td>
```

Kiểm tra xem phẩm có chưa? Thuộc lớp ProductDB.java

```
public boolean isProductbyID(String ID){
        try {
            conn = sqlConn.getConnection();
            String command = "select * from products where ID =?";
            PreparedStatement psmt = conn.prepareStatement(command);
            psmt.setString(1, ID);
            ResultSet rs = psmt.executeQuery();
            int i=0;
            while(rs.next()){
                i++;
            }
            if(i!=0){
                return true;
            }
            conn.close();
        } catch (SQLException ex) {
            Logger.getLogger(ProductDB.class.getName()).log(Level.SEVERE, null, ex)
        }
        return false;
    }
```

Thêm 1 sản phẩm Thuộc lớp ProductDB.java

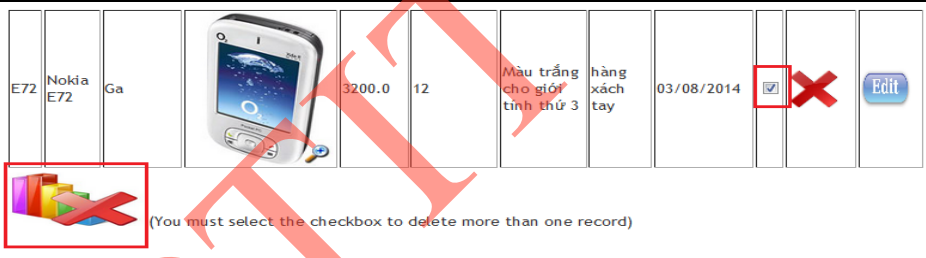
```
public void insert(Product p) {
    try {
        conn = sqlConn.getConnection();
        String command = "INSERT INTO products (ID,[name],cat_ID,image,price,quantity,"
                + "describe,status,adddate) VALUES(?,?,?,?,?,?,?,?,?)";
        PreparedStatement psmt = conn.prepareStatement(command);
        psmt.setString(1, p.getID());
        psmt.setString(2,p.getName());
        psmt.setString(3,p.getCat_ID());
        psmt.setString(4, p.getImage());
        psmt.setFloat(5, p.getPrice());
        psmt.setInt(6, p.getQuantity());
        psmt.setString(7, p.getDescribe());
        psmt.setString(8, p.getStatus());
        psmt.setString(9, p.getAdddate());
        psmt.executeUpdate();
        conn.close();
    } catch (SQLException ex) {
        Logger.getLogger(ProductDB.class.getName()).log(Level.SEVERE, null, ex);
    }
}
```

PTIT

Servlet:
addNewProductServlet.java
	@Override
	protected void doPost(HttpServletRequest request, HttpServletResponse response)
	throws ServletException, IOException {
	request.setCharacterEncoding("UTF-8");//tiếng Việt
	String ID = request.getParameter("ID").trim();
	String name = request.getParameter("name").trim();
	String cat_ID = request.getParameter("cat_ID").trim();
	String image = "images/"+request.getParameter("image").trim();
	float price = Float.parseFloat(request.getParameter("price").trim());
	int quantity = Integer.parseInt(request.getParameter("quantity").trim());
	String describe = request.getParameter("describe").trim();
	String status = request.getParameter("status").trim();
	String adddate = request.getParameter("adddate").trim();
	
	ProductDB productdb = new ProductDB();
	if(!productdb.isProductbyID(ID)){
	Product p = new Product(ID, name, cat_ID, image, price, quantity, describe, status, adddat
	productdb.insert(p);
	response.sendRedirect("home.jsp");
	}
	else{
	request.setAttribute("message", "products's ID : "+ID+" existed!!!");
	RequestDispatcher dispatcher = request.getRequestDispatcher("/addNewProduct.jsp");
	if (dispatcher != null) {
	dispatcher.forward(request, response);
	}
	}
	}

Xóa nhiều sản phẩm:

Bảng 6.5: Từng bước xóa 1 số sản phẩm

Giao diện	
Code	<label> <!-- Tao 1 checkbox de luu gia tri cua Product id trong form --> <input type="checkbox" name="chon" value="<%=fID %>" /> </label> <tr> <td colspan="12"><input type="image" name="Button" src="images/delall.j width="128" height="64" onClick="DeleteLinks();" /> (You must select the checkbox to delete more than one record)</td> </tr>

Javascript	<SCRIPT LANGUAGE="JavaScript">
	<!--Ham xu ly khi ta tich chuot vao Delete -->
	function DeleteLinks()
	{
	if (confirm("Are you sure to delete?"))
	{
	document.frmDelete.action = "deletesomeProducts";
	document.frmDelete.submit();
	}
	}

PTIT

Xóa 1 số SP thuộc lớp productDB.java	`public void deleteSome(String[] id){`
	`  try {`
	`  conn = sqlConn.getConnection();`
	`  String command ="DELETE FROM Products WHERE ID IN ('";`
	`  int i;`
	`  for(i=0;i<id.length-1;i++)`
	`  command += id[i] + "','";`
	`  command += id[i] + "')";`
	`  PreparedStatement psmt = conn.prepareStatement(command);`
	`  psmt.executeUpdate();`
	`  conn.close();`
	`  } catch (SQLException ex) {`
	`  Logger.getLogger(ProductDB.class.getName()).log(Level.SEVERE, null, ex);`
	`  }`
	
	`}`

Servlet: deletesomeProduct.java	`@Override`
	`protected void doPost(HttpServletRequest request, HttpServletResponse response)`
	`  throws ServletException, IOException {`
	`  String[] fID = request.getParameterValues("chon");`
	`  ProductDB productdb = new ProductDB();`
	`  productdb.deleteSome(fID);`
	`  response.sendRedirect("home.jsp");`
	
	`}`

CASE STUDY 3 (DÙNG JSP + SERVLET)

Viết lại module admin nhưng phần xử lý dùng servlet

PTIT

CHƯƠNG 7: ĐỐI TƯỢNG SESSION

1. ĐỐI TƯỢNG SESSION

Khi muốn truyền giá trị từ trang web này trang web khác trong một phiên làm việc thì sử dụng đối tượng Session. Đối tượng này thuộc lớp HttpSession. Mỗi khi có 1 phiên làm việc được tạo ra, thì trình chủ web sẽ cấp 1 định danh gồm 1 chuỗi bao gồm các ký tự và số cho phiên làm việc đó, định danh này là duy nhất. Khi kết thúc phiên làm việc, định danh này bị thu lại và phân phát lại cho phiên làm việc khác mới tạo ra.

Khởi tạo và gán giá trị cho session:

```
HttpSession session = request.getSession(true);
session.setAttribute("name",object);
```

Lấy giá trị từ session

```
session.getAttribute("name");
```

Hủy session

```
session.removeValue("name");
session.invalidate();
hoặc session.setAttribute("giohang", "0");
```

2. ỨNG DỤNG ĐỐI TƯỢNG SESSION KHI ĐĂNG NHẬP

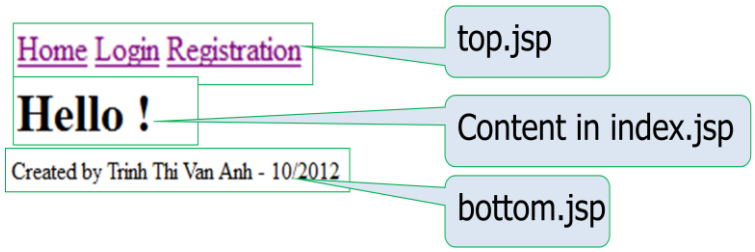
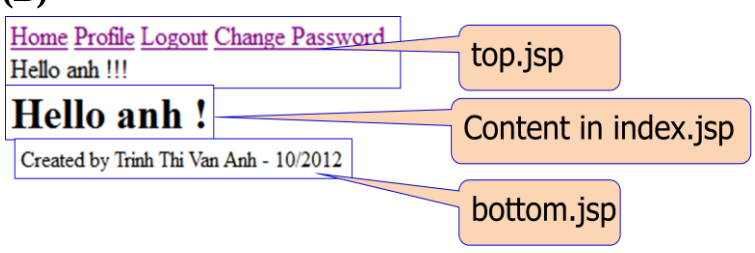
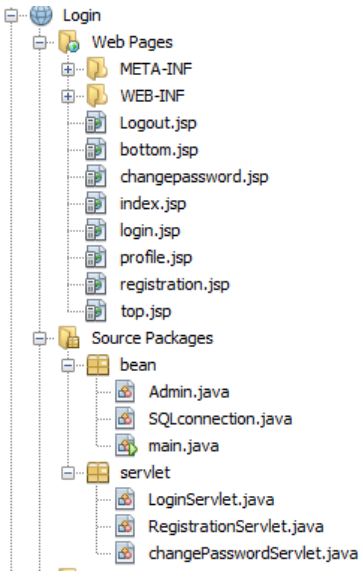
Xây dựng module đăng nhập gồm có:

Database: Trading (có bảng Admin: username, password, email,…)

Model (bean): Admin.java, SQLconnection.java

Servlet: LoginServlet.java, RegistrationServlet.java và changePasswordServlet.java

Bảng 7.1: Module đăng nhập

Giao diện	Project
(A)  **(B)**  	

PTIT

Bảng 7.2: Các thành phần trong module đăng nhập

Top.jsp Kiểm tra đăng nhập chưa? Thông qua session Khi chưa đăng nhập thì hiện (A) Khi đăng nhập rồi thì hiện (B) (Xem hình bảng 7.1)	<%@page import="bean.Admin"%> Home <% if (session.getAttribute("xx")== null) { %> Login Registration <% } else { Admin p = (Admin)session.getAttribute("xx"); %> Profile Logout Change Password Hello <%=p.getUsername()%> !!! <%}%>
bottom.jsp	</br> Created by Trinh Thi Van Anh - 10/2012
Index.jsp	<%@page import="bean.Admin"%> <jsp:include page = "top.jsp"/> Hello <% Admin p = (Admin) session.getAttribute("xx"); if (p != null) {%> <%=p.getUsername()%> <%}%>! <jsp:include page = "bottom.jsp"/>
Login.jsp Chọn Login Giao diện nhập Nhập sai Nhập đúng	
Code của Login.jsp	<jsp:include page = "top.jsp"/> <H2> <% if (request.getAttribute("message")!= null){%> <%=request.getAttribute("message")%> <%}% </H2> <FORM ACTION="LoginServlet" METHOD="POST"> User Name:<input type="text" name="username"> Password:<input type="password" name="password">

PTIT

	<input type="submit" value="Submit"> </FORM> <jsp:include page = "bottom.jsp"/>

Servlet: loginServlet.java	String userName = request.getParameter("username").trim();
	String password = request.getParameter("password").trim();
	HttpSession session = request.getSession(true);
	SQLconnection con=new SQLconnection();
	Statement st=null;
	ResultSet rs=null;
	String use=null;
	String pass=null;
	String email=null;
	boolean t=true;
	st = con.getConnection().createStatement();
	rs=st.executeQuery("select * from Admin");
	while(rs.next()){
	use = rs.getString("username");
	pass = rs.getString("password");email=rs.getString("email");
	if (userName.equals(use)&& password.equals(pass)){
	Admin p = new Admin(userName, password,email);
	session.setAttribute("xx",p);
	response.sendRedirect("index.jsp");
	t=false; break; } }
	if(t==true){
	request.setAttribute("message", "Please try again!");
	RequestDispatcher dispatcher = request.getRequestDispatcher("/login.jsp");
	if (dispatcher != null) {
	dispatcher.forward(request, response); } }

profile.jsp giao diện	Home Profile Logout Change Password Hello anh !!! **Your profile:** **PERSONAL INFORMATION** User name: anh Password: 1234 Email: anh12@yahoo.com Created by Trinh Thi Van Anh - 10/2012
profile.jsp code	<jsp:include page = "top.jsp"/> Your profile: <% Admin p = (Admin)session.getAttribute("xx"); if (p != null){%> PERSONAL INFORMATION User name: <%=p.getUsername()%> Password: <%=p.getPassword()%> Email: <%=p.getEmail()%> <%}%> <jsp:include page = "bottom.jsp"/>
Logout.jsp	<% if (session.getAttribute("xx")!= null){

PTIT

```
                    session.removeValue("xx");
                    session.invalidate();
                    response.sendRedirect("index.jsp");
                }
                else{
                  response.sendRedirect("index.jsp");}
%>
```

3. GIỎ HÀNG

Xây dựng module giỏ hàng ta cần có:

Database : BookStore (có các bảng: Customers, Orders, Orderdetails)

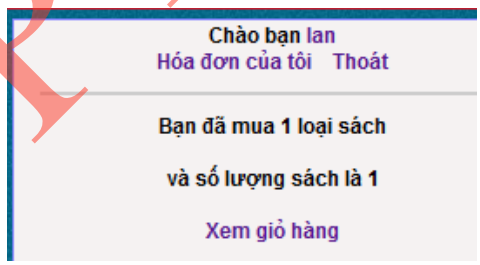
Model: Book.java, Order.java, Orderdetail.java

Servlet: doAddGiohang.java, doDatHang.java, doDangNhap.java



Hình 7.1: giao diện phía người dùng chưa đăng nhập

Sau khi đăng nhập thì Sidebar như sau:



Hình 7.2: side bar sau khi đăng nhập

Khi chọn 1 món hàng định mua thì xuất hiện thông tin chi tiết món hàng này:

PTIT



Hình 7.3: chi tiết món hàng

```
<% int id_chitiet;
   id_chitiet = Integer.parseInt(request.getParameter("id"));
 if (session.getAttribute("customer") !=null
!session.getAttribute("customer").equals("0")) { %>
 <a id="dathang"href="doAddGiohang?id=<%=id_chitiet%>"> Cho sách vào
giỏ hàng</a> <%                                          }else{    %>
<a id="dathang" href="javascript:onclick"  onclick="alert('Bạn phải
đăng nhập')">Cho sách vào giỏ hàng</a>
 <%  }  %>
```

Tiếp theo ta phải đặc tả 1 giỏ hàng để client có thể sử dụng. Ta nhận thấy rằng 1 giỏ hàng chứa nhiều món hàng, mỗi món hàng giữ thông tin về mã số của sản phẩm, số lượng đang có trong giỏ và đơn giá của mỗi sản phẩm.

Giohang.jsp



Hình 7.4: hiện thị giỏ hàng

```
<%    float total = 0;
 if (session.getAttribute("giohang") != null &&
!session.getAttribute("giohang").equals("0")) {
 ArrayList<OrderDetail> listOdGiohang = (ArrayList<OrderDetail>)
session.getAttribute("giohang");
  BookDB bookdb = new BookDB();
  for (int i = 0; i < listOdGiohang.size(); i++) {
      Book book = new Book();
      book = bookdb.getBookById(listOdGiohang.get(i).getBook_id());
      String bookName = book.getName();
      float bookPrice = book.getPrice();
```

PTIT

```
      int quantity = listOdGiohang.get(i).getQuantity();
      float totalPrice = bookPrice * quantity;
      total += totalPrice;
  %>
<tr>
            <td><%=(i+1)%></td>
           <td><%=bookName%></td>
           <td><%=bookPrice%>K</td>
            <td><%=quantity%></td>
            <td><%=totalPrice%>K</td>
  </tr>   <%    }   }            %>
<b>Tổng tiền: <%=total%>K</b></p>
```

Khi chọn cho sách vào giỏ, ta xử lý bằng servlet: doAddGiohang.java (servlet)

Trong đối tượng giỏ hàng ta dùng 1 ArrayList để lưu trữ 1 danh sách các món hàng. Ở đây ta có các thao tác là đặt hàng và xóa giỏ hàng. Sau đó ArrayList này được gán cho đối tượng giohang để gửi sang phần xử lý đặt hàng thông qua đối tượng session:

session.setAttribute("giohang", listOrderDetail);

```
Protected void processRequest(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {
……
    ArrayList<OrderDetail> listOrderDetail;
    int bookId =Integer.parseInt(request.getParameter("id"));
    OrderDetail od = new OrderDetail();
    od.setBook_id(bookId);
    if(session.getAttribute("giohang")==null ||
      session.getAttribute("giohang").equals("0")){
       listOrderDetail = new ArrayList<OrderDetail>();
       od.setQuantity(1);   listOrderDetail.add(od);
}else{
   listOrderDetail =
(ArrayList<OrderDetail>)session.getAttribute("giohang");
   boolean check = true;
   for(int i=0;i<listOrderDetail.size();i++){
       OrderDetail odetail = listOrderDetail.get(i);
       if(odetail.getBook_id()==bookId){
           int quantity = odetail.getQuantity();
           listOrderDetail.get(i).setQuantity(quantity+1);
           check = false;      break;      }     }
    if(check==true){
          od.setQuantity(1);    listOrderDetail.add(od);  }    }
            session.setAttribute("giohang", listOrderDetail);
            response.sendRedirect(request.getHeader("referer"));
……
```

Chọn chức năng đặt hàng: doDatHang.java (servlet)

Chúng ta lấy được giỏ hàng thông qua đối tượng session. Xử lý đặt hàng là chuyển giỏ hàng vào bảng Orderdetail và xóa giỏ hàng.

```
if(session.getAttribute("giohang")==null ||
session.getAttribute("giohang").equals("0")){
```

PTIT

```
                response.sendRedirect("index.jsp"); }
            else{
                Order od = new Order();
od.setCus_username(session.getAttribute("customer").toString());
                OrderDB odDB = new OrderDB(); odDB.insert(od);
                int id = odDB.getLastId();   out.print(id);
                ArrayList<OrderDetail> listOrderDetail =
     (ArrayList<OrderDetail>)session.getAttribute("giohang");
                OrderdetailsDB odetailDB = new OrderdetailsDB();
for(int i=0;i<listOrderDetail.size();i++){
                    OrderDetail odetail = listOrderDetail.get(i);
                    odetail.setOrder_id(id);
                    odetailDB.insert(odetail);  }
                session.setAttribute("giohang", "0");
                response.sendRedirect("index.jsp");
            }
        } finally {
            out.close();
        }
```

PTIT

CASE STUDY 3 + 4

- Viết thêm đăng nhập, đăng ký, đăng xuất,… cho cả 2 module admin và phía người dùng
- Viết thêm phần giỏ hàng, đặt hàng, thanh toán,… cho module phía người dùng

CHƯƠNG 8: CÁC THẺ MỞ RỘNG (JSP CUSTOM TAGS)

1. GIỚI THIỆU CUSTOM TAG

1.1 Giới thiệu custom tag

Bộ thư viện JSTL (JSP Standard Tag Library) là bộ thư viện của Java phát triển dành cho việc xây dựng trang JSP. Bên cạnh bộ thư viện này, trang JSP còn cho phép người lập trình xây dựng một thư viện thẻ của mình: custom tag. Custom tag được mô tả chi tiết trong 1 file XML gọi là tag library, trong file XML chúng ta khai báo các thẻ có trong bộ custom tag và các lớp thực thi của từng thẻ. Các lớp thực thi được gọi là các tag handler. Chúng ta có 2 loại lớp handler:

- Simple tag handler: chỉ dùng cho các thẻ không có các thành phần chứa scripting và không có body
- Classic tag handler: được sử dụng khi các thành phần của thẻ có chứa scripting.

Khi một trang JSP có chứa custom tag được chuyển đổi sang servlet, các custom tag sẽ được chuyển đổi sang các phương thức được khai báo trong tag handler. Khi Web container thực thi trang JSP, các phương thức này sẽ được gọi kèm theo.

1.2 Ưu điểm của Custom tag

Custom tag có 3 ưu điểm chính sau:

- Tách rời các phương thức logic khỏi lớp View: Custom tag và Scriptlet cung cấp các chức năng tương đương. Điểm khác biệt giữa 2 kỹ thuật này là chúng được viết ở nơi nào. Trong khi scriptlet cho phép nhúng trực tiếp 1 đoạn code vào trong trang JSP, custom tag là những đoạn code chạy phía sau để tổ chức lại HTML.
- Dễ sử dụng: Custom tag rất dễ sử dụng cho người lập trình Java hay người không biết lập trình Java.
- Dễ cài đặt: Custom tag sau khi xây dựng xong sẽ được đóng gói vào file jar và có thể sử dụng cho nhiều trang web khác nhau.

Những thông tin về Custom Tags.

Để có thể sử dụng tốt Custom Tags, ta cần chú ý:

- JSP 1.0 không hỗ trợ “Tag Libralies”.
- JSP 1.1 hỗ trợ sự sát nhập của người dùng được tạo bởi Custom tags trong một file JSP.
- Cấu trúc của Custom Tags trong trang JSP cũng giống như XML tags.

Các loại thẻ Custom Tags.

Empty Tag

PTIT

Ví dụ <td:welcome/> Đây là thẻ không có body cũng như Attribute.

Custom tags với thuộc tính (Attribute).

Ví dụ <td:welcome color = "blue"></tdwelcome> Đây là loại thẻ Custom Tags có thuộc tính color là "blue".

Custom tags có body.

Ví dụ

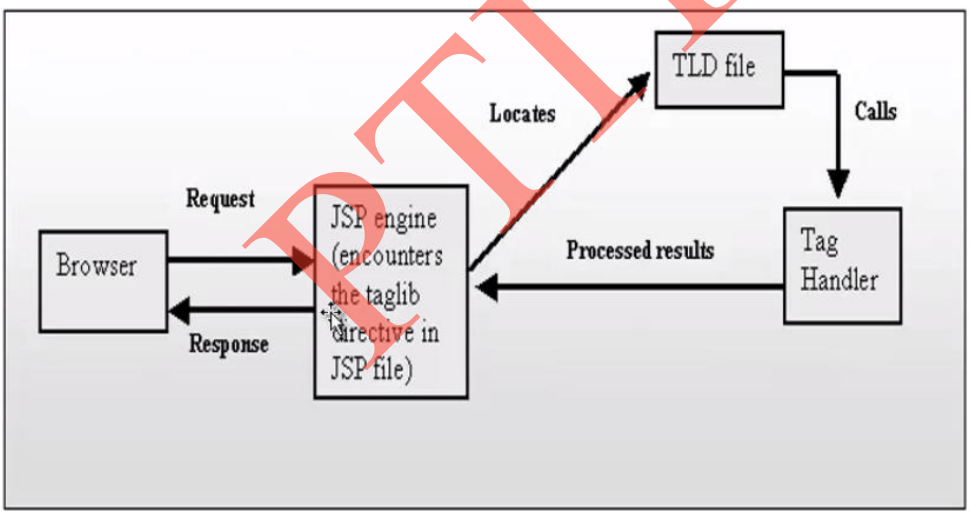
```
<td:welcome>
        <%= today_date %>
</welcome>
```

Custom tags lồng nhau.

Các phần của Custom Tags gồm: Tag Library Description và Tag Handler

- Tag Handler: Là đoạn code thực sự xử lý công việc như nêu ở phần ưu điểm phía trên.
- Tag Library: Là nơi mô tả.

Mô hình làm việc của custom tag



PTIT

Hình 8.1: Mô hình làm việc của custom tag

Như biểu đồ ta thấy khi trình duyệt thực hiện Request lên server, request sẽ đi qua JSP engine và tìm trong TLD file hiểu được đặc tả cho file đó. Khi một thẻ (tag) được tạo ra chúng biết chúng được xử lấy ở đâu, với tag handler nào nó sẽ gọi đến tag handler đó và xử lý, kết quả trả về cho JSP Engine sau đó trả về cho trình duyệt. Đây là một vòng đời của một JSP file với Custom Tags.

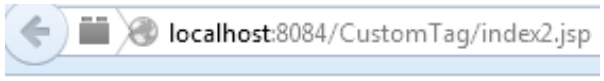
2. CÁC BƯỚC ĐỂ CÀI ĐẶT CUSTOM TAG DÙNG JSP

2.1 Từng bước tạo 1 Custom tag

Tuần tự các bước cài đặt và sử dụng thẻ.

Bước 1: Tạo lớp Tag Handler
Bước 2: Tạo file TLD (Tag Library Descriptor), file dạng xml
Bước 3: Khai báo thẻ <%@ Taglib %>>
Bước 4: Dùng thẻ trong file jsp

Ví dụ giả sử chúng ta tạo thẻ như sau:



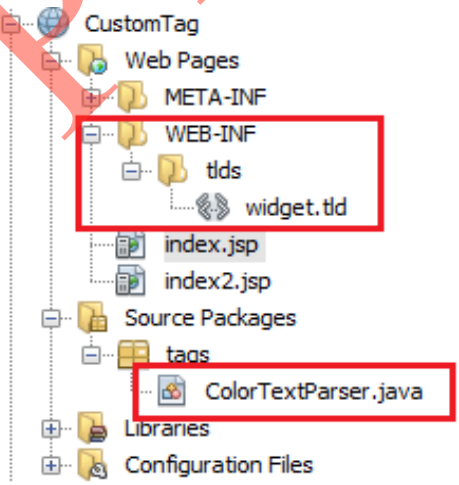
JSP Custom Tag Demo

Hình 8.2: mẫu để tạo thẻ

```
<%@page contentType="text/html"
pageEncoding="UTF-8"%>
 <html>  <head>
 <meta http-equiv="Content-Type"
content="text/html; charset=UTF-
8">
 <title>JSP Custom Tag
Demo</title>
 </head>  <body>
      <span style="color:
#ff3c00;">JSP Custom Tag
Demo</span>
 </body>  </html>
```

Hình : Ví dụ dự kiến tạo thẻ nội dung text có màu.

Cài đặt xong thẻ nội dung text có màu sẽ có dạng:



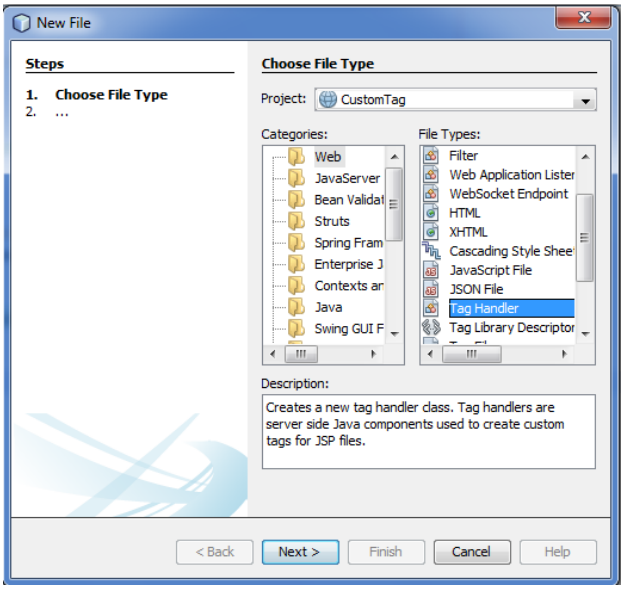
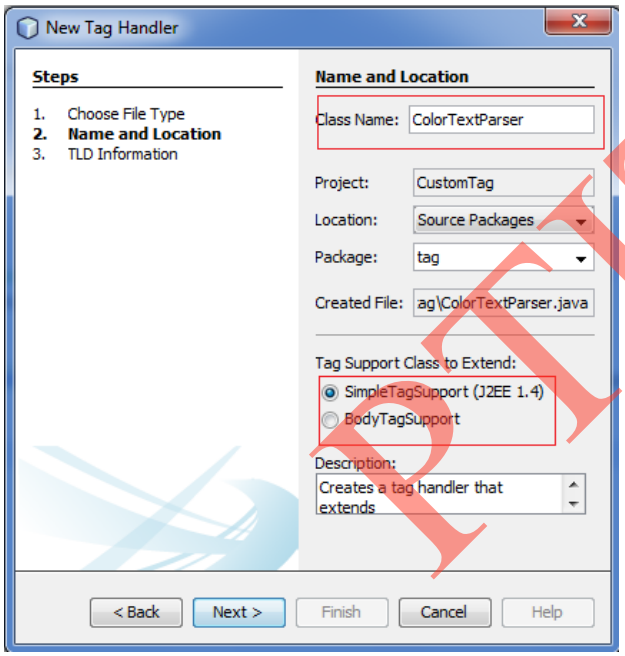
Hình 8.3: Dạng của cấu trúc Project

Bước 1: Tạo Handler tag

Tag Handler bản chất là 1 lớp Java, nó kế thừa (extends) từ 1 trong các lớp của Java dành cho Custom tag: SimpleTagSupport hay BodyTagSupport.

BTIT

Bảng 8.1: Tạo Handler tag

<table>
<tr>
<td> 
</td>
<td>- Chúng ta bật chuột phải, chọn New/Other… xuất hiện giao diện như hình bên.
- Sau đấy chọn Tag Handler
- Chọn Next</td>
</tr>
<tr>
<td> 
</td>
<td>- Xuất hiện như hình bên
- Gõ vào Class Name: ColorTextParser (tên lớp tạo thẻ)
- Kế thừa từ 2 lớp: SimpleTagSupport hoặc BodyTagSupport
- Chọn Next
- Bỏ chọn
 

Chọn finish</td>
</tr>
</table>

PTIT

```
public class ColorTextParser extends SimpleTagSupport{
    private String content;
    private String color;

    public ColorTextParser(){}

    @Override
    public void doTag() throws JspException, IOException {
        JspWriter out = getJspContext().getOut();
        JspFragment f = getJspBody();
            if (f != null) {
                f.invoke(out);
            }
            String html = "<span style=\"color:";
            html += color;
            html += "\">";
            html += content;
            html += "</span>";

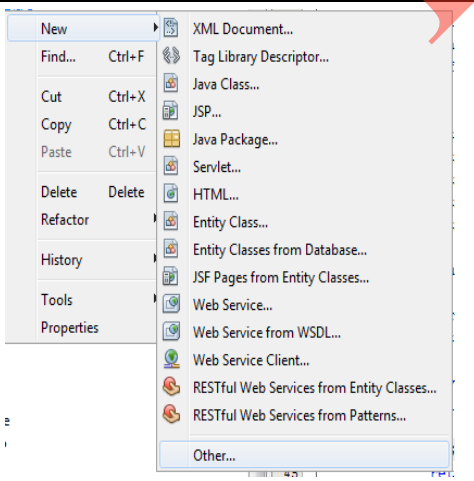
            out.print(html);
    }
    public String getContent() {
            return content;
      }
      public void setContent(String content) {
            this.content = content;
      }
      public String getColor() {
            return color;
```

Hình 8.4: Tệp ColorTextParser.java

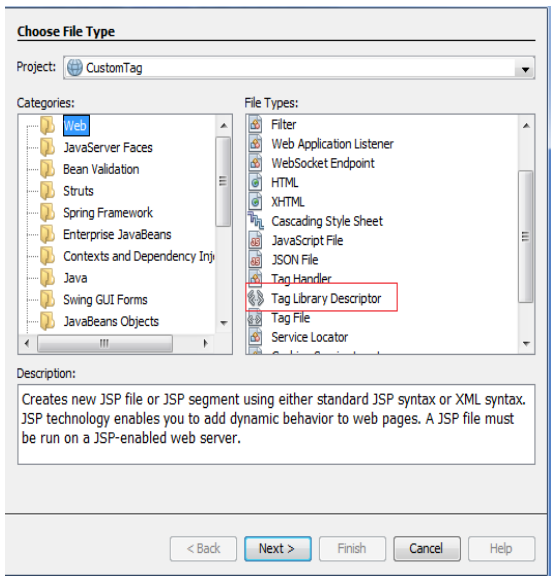
Bước 2: Tạo file TLD (Tag Library Descriptor), file dạng xml

Tạo một file widget.tld là nơi chứa mô tả cho các custom tag như tên của tag, các thuộc tính của tag …

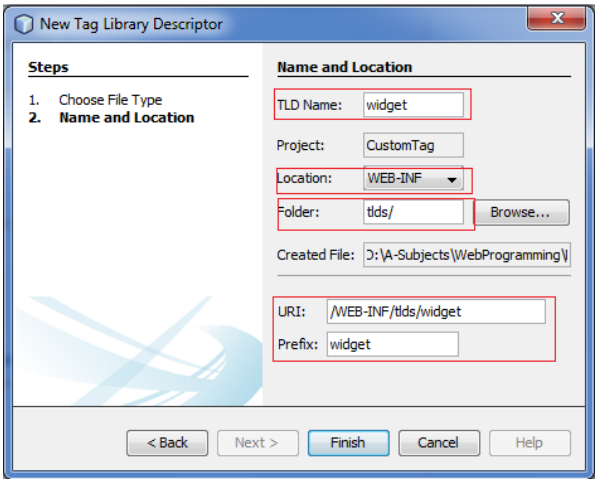
Bảng 8.2: Tạo TLD chứa mô tả thẻ

	• Name of project: **New/Other…**

PTIT



- Chọn Tag Library Descriptor
- Chọn Next



- Gõ vào như hình trên



- File widget.tld tạo ra như hình bên

File widget.tld

```
<?xml version="1.0" encoding="UTF-8"?>
<taglib version="2.1" xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-jsptaglibrary_2_1.xsd">
  <tlib-version>1.0</tlib-version>
  <short-name>widget</short-name>
  <uri>/WEB-INF/tlds/widget</uri>
  <tag>
        <name>ColorText</name>
        <tagclass>tag.ColorTextParser</tagclass>
        <body-content>empty</body-content>
        <info>Color a paragraph</info>
        <attribute>
         <name>content</name>
         <required>true</required>
        </attribute>
        <attribute>
         <name>color</name>
         <required>true</required>
        </attribute>
  </tag>
</taglib>
```

Bước 3: Khai báo thẻ <%@ Taglib %>>

```
<%@taglib prefix="wgt" uri="/WEB-INF/tlds/widget"%>
```

Bước 4: Dùng thẻ trong file jsp

PTIT

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@taglib prefix="wgt" uri="/WEB-INF/tlds/widget"%>
<!DOCTYPE html>
<html>  <head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>JSP Custom Tag Demo</title>  </head> <body>
<wgt:ColorText color="#ff3c00" content="JSP Custom Tag Demo at Web
programming"/>
    </body> </html>
```

2.2 CKEditor trong jsp

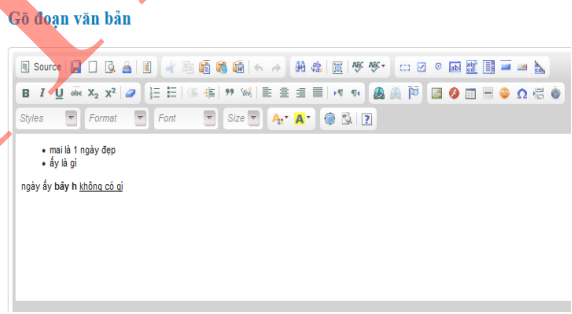
CKEditor (còn gọi là FCKeditor) là một trình soạn thảo mã nguồn mở theo kiểu WYSIWYG của CKSource. Chương trình này có thể tích hợp vào các web site mà không cần cài đặt. Phiên bản đầu tiên được phát hành năm 2003 và đến nay được rất nhiều người sử dụng. Cũng giống các trình soạn thảo dành cho web khác, nó sử dụng JavaScipt làm nền tảng và tương thích với hầu hết các trình duyệt Internet. Các bước cài đặt:

Bước 1: Tải CkEditor tại đây: http://ckeditor.com/download

Bước 2: Sau đó giải nén ra folder "ckeditor".

Bước 3: Chạy thử CkEditor với scripts

```
<%@taglib  uri="http://ckeditor.com"
prefix="ckeditor" %>
<h2 style="color: #0782C1"><label>Gõ
đoạn văn bản</label></h2>
<textarea
name="description"></textarea>
<ckeditor:replace
replace="description"
basePath="../ckeditor/"/>
```



Hình 8.5: Ví dụ dùng CkEditor

2.3 CKFinder trong jsp

CKFinder là một bộ công cụ mạnh mẽ dùng để đưa các tệp lên máy chủ (upload), quản lý ảnh. Chúng ta đã biết, một bài viết tốt không thể thiếu hình ảnh, khi lời nói không thể mô tả được hết điều ta muốn diễn đạt, hình ảnh là biện pháp hữu hiệu. CKFinder có giao diện quản lý ảnh thân thiện, dễ sử dụng.

Bước 1: tải CKFinder giải nén vào thư mục "ckfinder"

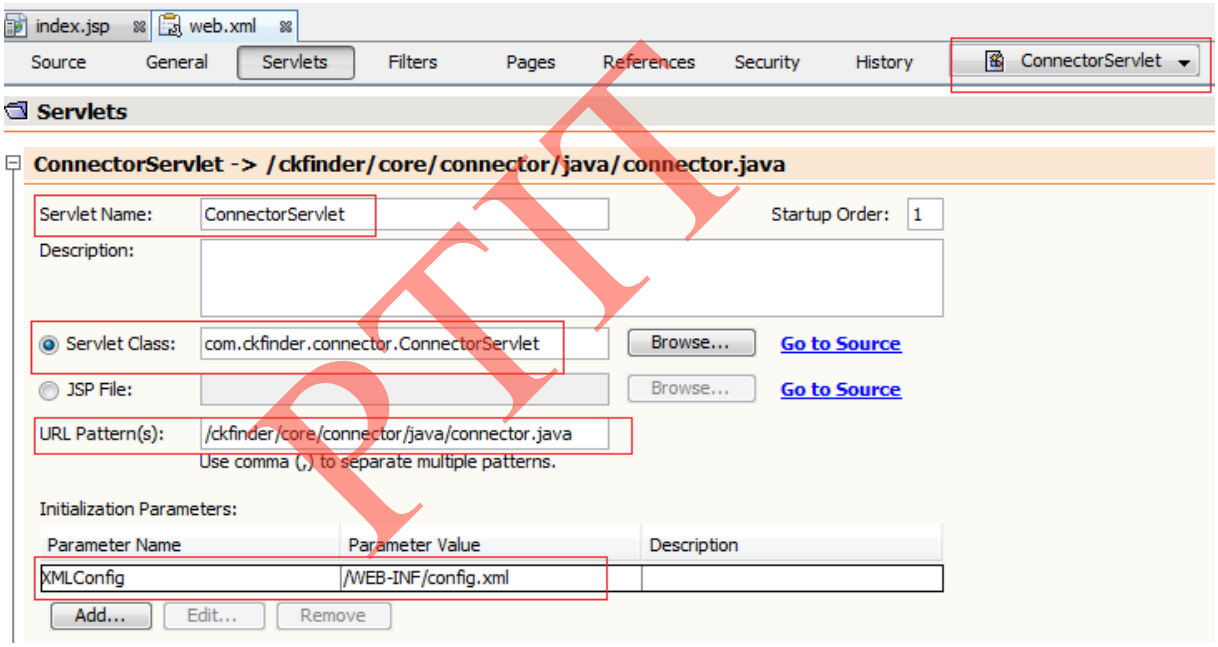
Bước 2: tạo file mô tả ckfinder.tld.

```
<short-name>ckfinder</short-name>
  <uri>http://ckfinder.com</uri>
 <tag>
```

PTIT

```
            <name>ckfinder</name>
            <tag-class>com.ckfinder.connector.CKFinderTag</tag-class>
            <body-content>empty</body-content>
            <attribute>
                  <name>basePath</name>
                  <required>true</required>
            </attribute>
           ……..
<tag>
            <name>setupCKEditor</name>
            <tag-
class>com.ckfinder.connector.CKFinderSetupCKEditor</tag-class>
            <body-content>empty</body-content>
            <description>Setups params for CKEditor.</description>
            <attribute>
                  <name>basePath</name>
                  <required>false</required>
```

Bước 3: web.xml và config.xml



Bước 4 : tạo file jsp

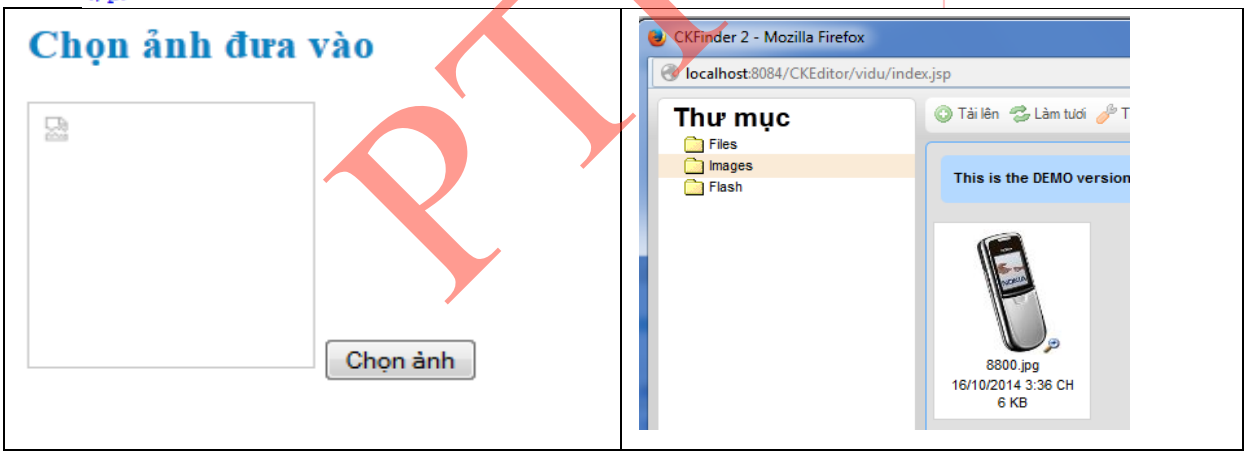
```
<%@taglib uri="http://ckeditor.com" prefix="ckeditor" %>
<script type="text/javascript" src="../ckfinder/ckfinder.js"></script>
```

PTIT

```
<script type="text/javascript">
    function BrowseServer()
    {
        // You can use the "CKFinder" class to render CKFinder in a page:
        var finder = new CKFinder();
        finder.basePath = '../';        // The path for the installation of
        finder.selectActionFunction = SetFileField;
        finder.popup();

        // It can also be done in a single line, calling the "static"
        // popup( basePath, width, height, selectFunction ) function:
        // CKFinder.popup( '../', null, null, SetFileField ) ;
        //
        // The "popup" function can also accept an object as the only argume
        // CKFinder.popup( { basePath : '../', selectActionFunction : SetFil
    }

    // This is a sample function which is called when a file is selected in
    function SetFileField( fileUrl )
    {
        document.getElementById( 'xFilePath' ).value = fileUrl;
        document.getElementById('image').setAttribute('src','..'+fileUrl );
    }
</script>
<p>
    <img src="" width="150" height="150" id="image" style="border: 1px solid #ccc; padding: 2px"/>
    <input id="xFilePath" name="image" type="hidden" size="60" value="" />
    <input type="button" value="Chọn ảnh " onclick="BrowseServer();" />
</p>
```



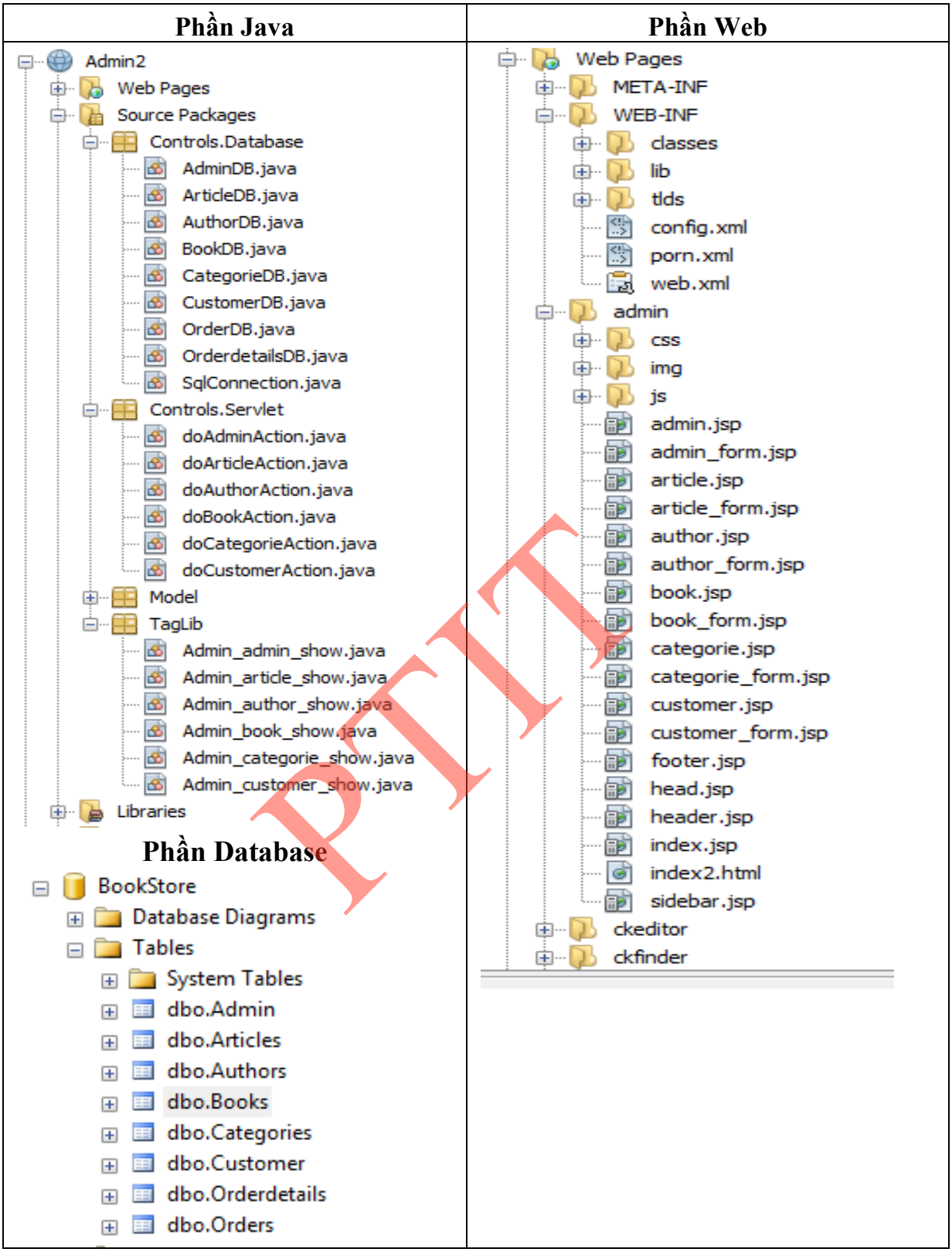
Hình 8.6: Hiển thị dùng CKFinder

3. VÍ DỤ MINH HỌA

Chúng ta tạo module admin với việc xây dựng phần View (trong mô hình MVC) bằng cách tạo thẻ. Dự án tạo ra có dạng như sau:

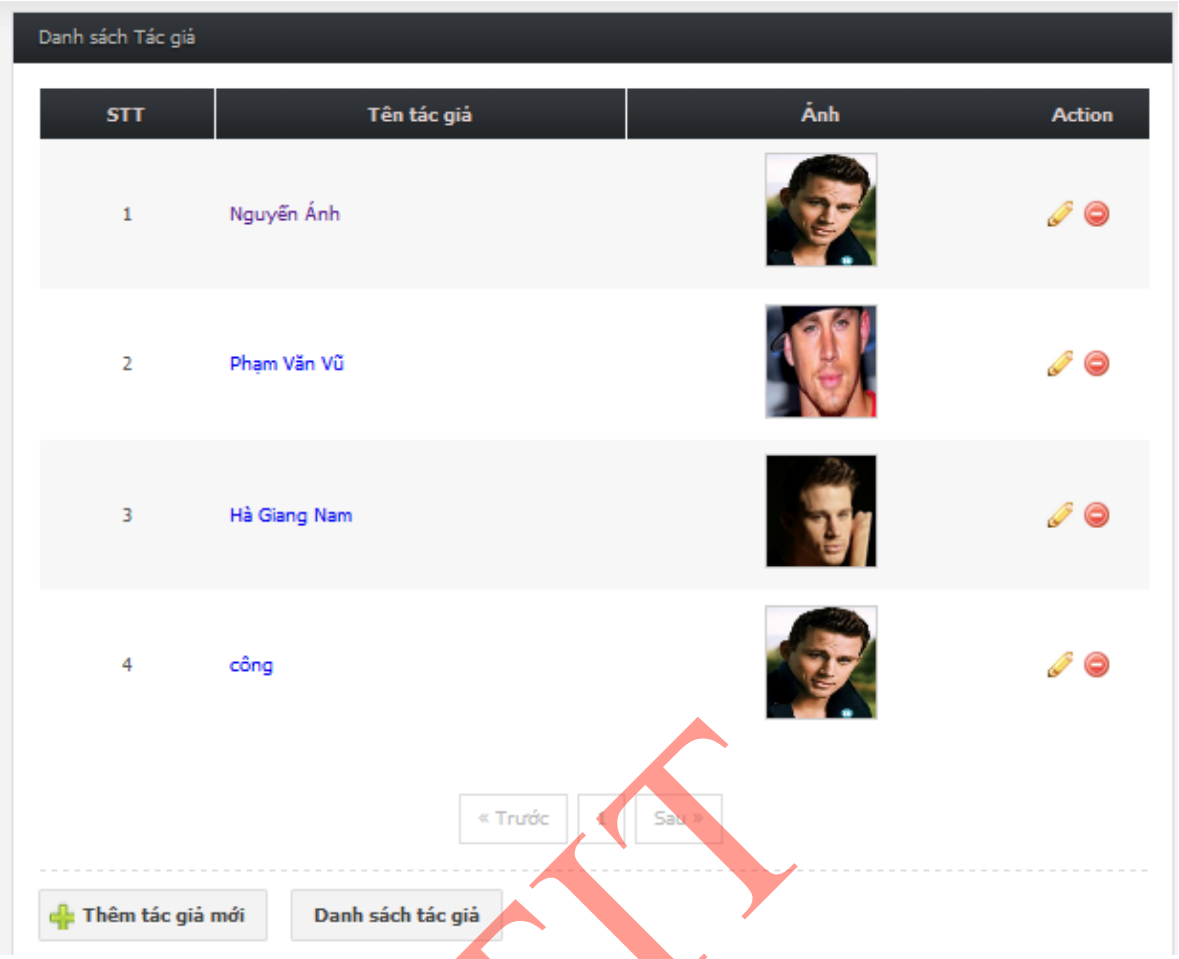
PTIT

Bảng 8.3: Project cho module admin



Chúng ta tạo thẻ hiển thị giao diện sau:

PTIT



Hình 8.7: Giao diện danh sách tác giả để tạo thẻ

Bước 1: Tạo thẻ, đầu tiên tạo thẻ Admin_author_show.java, lớp này chứa thuộc tính page (lật trang).

```
public class Admin_author_show extends SimpleTagSupport {
    private int page;
```

Có phương thức doTag() tạo thẻ:

```
@Override
public void doTag() throws JspException {
    JspWriter out = getJspContext().getOut();
```

Ta có start là con trỏ vào phần tử đầu của trang và end trỏ đến phần tử cuối của trang. Giá trị 10 chỉ mỗi trang có 10 phần tử (ở đây là 10 sản phẩm).

```
AuthorDB authordb = new AuthorDB();
ArrayList<Author> listAuthor = authordb.getAuthorAll();
int start = (page - 1) * 10;
int end = start + 10;
```

Bắt đầu viết vào thẻ:

```
String html = "<table><thead><tr><th scope=\"col\">STT</th><th scope=\"col\">Tên tác giả</th>"
        + "<th>Ảnh</th><th scope=\"col\" style=\"width: 65px;\">Action</th></tr></thead>";
```

Khi đó hiển thị cho phần code trên có dạng:



Hình 8.8: Hiển thị tiêu đề danh sách tác giả

PTIT

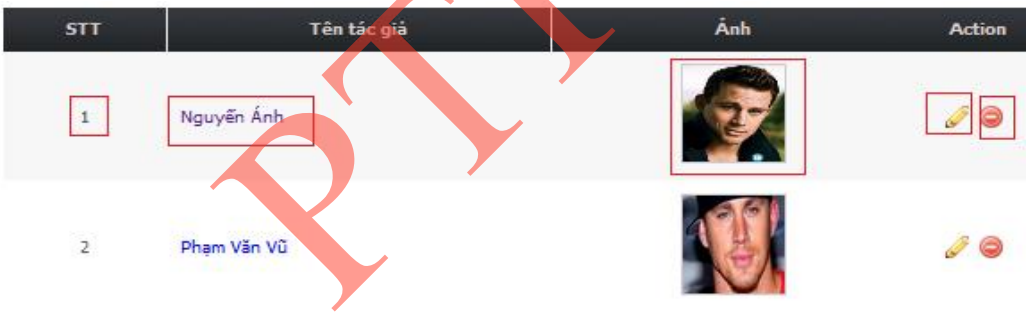
Sau đó đọc các phần tử thuộc trang từ danh sách listAuthor:

```
html += "<tbody>";
for (int i = start; i < end; i++) {
    if (i >= listAuthor.size()) {
        break;
    }
```

Chúng ta viết từng dòng cho danh sách: STT, Tên tác giả, ảnh và action (gồm sửa và xóa):

```
59  Author author = listAuthor.get(i);
60  html += "<tr>";
61  html += "<td class=\"align-center\">" + (i + 1) + "</td>";// STT
62  html += "<td><a href=\"author_form.jsp?id=" + author.getId() + "\">"
63          + author.getName() + "</a></td>";
64  html += "<td class=\"align-center\"><a href=\"author_form.jsp?id="
65          + author.getId() + "\"> <img src=\"http://localhost:8084/Admin2"
66          + author.getImage() + "\" width=\"64\" height=\"64\" style=\"border:1px "
67          + "solid #ccc;\"/></a></td>";
68  html += "<td class=\"align-center\">\n"
69          + "<a href=\"author_form.jsp?id=" + author.getId()
70          + "\" class=\"table-icon edit\" title=\"Edit\"></a>\n"
71          + "<a href=\"../admin/doAuthorAction?action=delete&id="
72          + author.getId() + "\" class=\"table-icon delete\" title=\"Delete\"></a>\n"
73          + "</td>";
74  html += "</tr>";
75  }
76  html += "</tbody></table>";
```

Hiển thị cho phần code trên có dạng:



Hình 8.9: Bảng danh sách tác giả do thẻ tạo ra

Phần phân trang code có dạng:

PTIT

```
int number_of_page = (int) listAuthor.size() / 10 + 1;
html += "<div class=\"entry\">\n"+ "<div class=\"pagination\">\n";
if (page == 1) {
    html += "<span>« Truóc</span>\n";
} else {
    html += "<a href=\"author.jsp?page=" + (page - 1) + "\">« Truóc</a>\n";
}
for (int i = 1; i <= number_of_page; i++) {
    if (i == page) {
        html += "<span>" + i + "</span>\n";
    } else {
        html += "<a href=\"author.jsp?page=" + i + "\">" + i + "</a>\n";
    }             }
if (page == number_of_page) {
    html += "<span>Sau »</span>\n";
} else {
    html += "<a href=\"author.jsp?page="+(page+1)+"\">Sau »</a>\n";
}
html += "</div>";
```

Hiển thị cho code trên (phân trang):



Hình 8.10: Phần lật trang

Thêm 2 nút: Thêm tác giả mới và Danh sách tác giả

```
html +=  "<div class=\"sep\"></div>\n"
        + "<a class=\"button add\" href=\"author_form.jsp\">Thêm tác giả mới</a> "
        + "<a class=\"button\" href=\"author.jsp\">Danh sa´ch ta´c gia˙</a> \n"
        + "</div>";
out.print(html);
```

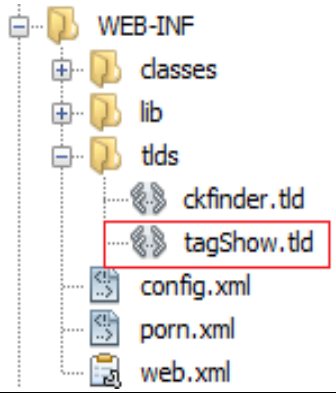
Hiển thị cho code trên:



Hình 8.11: 2 nút dưới danh sách tác giả

Bước 2: Mô tả thẻ (tagShow.ltd)

Bảng 8.4: Mô tả thẻ

Project	**Mô tả trong tệp tagShow.ltd**
	<short-name>admin_show</short-name> <uri>/WEB-INF/tlds/admin_show</uri> <tag> <name>Admin_author_show</name> <tag-class>TagLib.Admin_author_show</tag-class> <body-content>scriptless</body-content> <attribute> <name>page</name> <required>true</required> <rtexprvalue>true</rtexprvalue> </attribute> </tag>

PTIT

Bước 3: dùng thẻ vừa tạo ở trên trong tệp author.jsp

```
<%@taglib uri="/WEB-INF/tlds/tagShow.tld" prefix="tagShow" %>
<head>
    <%@include file="head.jsp" %>
</head>
<body>
    <div class="wrap">
        <div id="header">
            <%@include file="header.jsp" %>
        </div>
        <div id="content">
            <div id="sidebar">
                <%@include file="sidebar.jsp" %>
            </div>
            <div id="main">
                <div class="full_w">
                    <div class="h_title">Danh sách Tác giả</div>
                    <%
                        int page_curent = 1;
                        if (request.getParameter("page") != null) {
                            page_curent = Integer.parseInt(request.getParameter("page"));
                        }
                    %>
                    <tagShow:Admin_author_show page="<%=page_curent%>"/>
                </div>
            </div>
            <div class="clear"></div>
        </div>
        <div id="footer">
            <%@include file="footer.jsp" %>
        </div>
    </div>
</body>
```

PTIT

BÀI CUỐI KỲ (TRANG WEB HOÀN CHỈNH)

Ghép thành 1 trang web hoàn chỉnh gồm đủ cả phía admin và người dùng

CHƯƠNG 9: CÁC NỀN TÀNG HỖ TRỢ PHÁT TRIỂN ỨNG DỤNG WEB TRÊN J2EE

1. KHÁI QUÁT VỀ CÁC NỀN TẢNG HỖ TRỢ PHÁT TRIỂN WEB

1.1 Hệ nền J2EE

Ngày nay, càng có nhiều hơn các nhà phát triển muốn viết các ứng dụng theo kiểu giao dịch cho các doanh nghiệp, do đó nó yêu cầu về mặt tốc độ, bảo mật, cũng như sự đáng tin cậy của công nghệ phía máy chủ. Nếu bạn đang làm việc trong lĩnh vực này, chắc hẳn các bạn cũng biết rằng sự chuyển động nhanh cùng các yêu cầu của thế giới về thương mại điện tử và công nghệ thông tin. Điều đó làm cho các ứng dụng doanh nghiệp phải được thiết kế, xây dựng và xử lý sao cho chi phí là thấp nhất, tốc độ nhanh nhất và ít tốn nguồn tài nguyên hơn bao giờ hết.

Để giảm chi phí cùng theo dõi một cách nhanh nhất việc thiết kế và phát triển các ứng dụng thì hệ nền Java™2, Java2 Enterprise Edition (J2EE™) đã cung cấp một phương pháp tiếp cận dựa trên các thành phần để thiết kế, phát triển, gom lại và triển khai các ứng dụng doanh nghiệp.

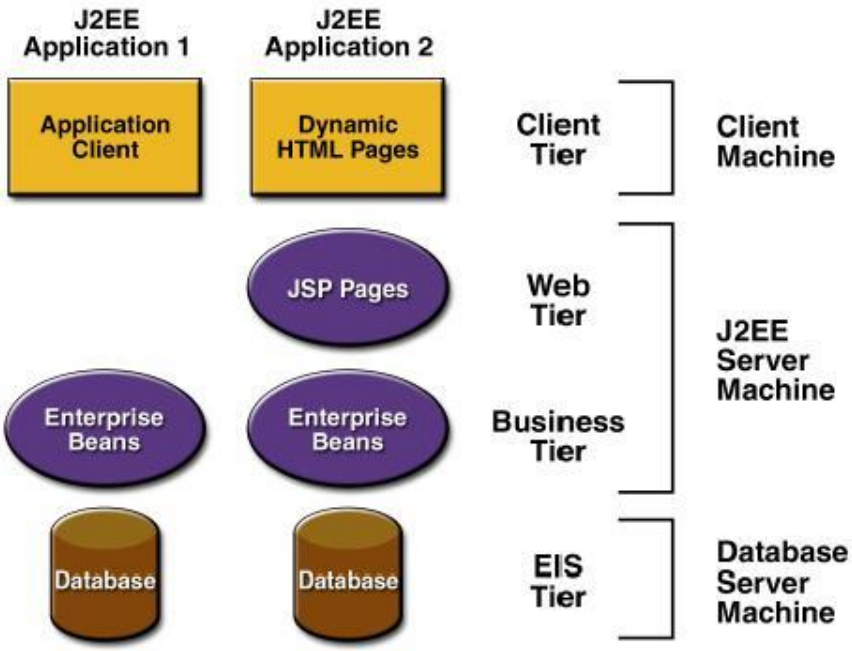
Hệ nền J2EE cung cấp một mô hình ứng dụng theo kiểu đa tầng, tái sử dụng các thành phần, bảo mật thống nhất, linh hoạt trong việc kiểm soát các giao dịch và các dịch vụ web được hỗ trợ thông qua trao đổi dữ liệu tích hợp trên XML (Extensible Markup Language) – theo các tiêu chuẩn mở và các giao thức.

Không chỉ giúp các bạn cung cấp các giải pháp kinh doanh của mình ra thị trường một cách nhanh nhất, mà còn làm cho các giải pháp về các thành phần cơ bản độc lập hệ nền J2EE không phụ thuộc vào các sản phẩm, cũng như các ứng dụng lập trình giao diện của bất cứ nhà cung cấp nào. Các nhà cung cấp và khách hàng có thể tự do chọn lựa các sản phẩm và các thành phần đáp ứng tốt nhất các yêu cầu kinh doanh và công nghệ của họ.

1.2 Các ứng dụng phân tán đa tầng

Hệ nền J2EE sử dụng một mô hình ứng dụng theo kiểu đa tầng cho các ứng dụng doanh nghiệp. Về mặt logic ứng dụng được chia theo các thành phần theo chức năng, và các thành phần ứng dụng khác nhau tạo nên một ứng dụng J2EE được cài đặt trên các máy khác nhau tùy thuộc vào các tầng trong môi trường phân tầng của J2EE nơi mà các thành phần ứng dụng thuộc về.

PTIT



Hình 9.1: Trình bày 2 ứng dụng phân tầng J2EE được phân chia

Các thành phần của ứng dụng J2EE được trình bày trong hình 9.1 bao gồm:

- Các thành phần tầng khách hành chạy trên máy khách hàng
- Các thành phần tầng web chạy trên máy chủ J2EE
- Các thành phần nghiệp vụ kinh doanh chạy trên máy chủ J2EE
- Phần mềm hệ thống thông tin doanh nghiệp(EIS) chạy trên EIS Server

Mặc dù 1 ứng dụng J2EE có thể bao gồm 3 hay 4 tầng như trong hình 9.1 nhưng các ứng dụng phân tầng J2EE thường được coi như là 3 tầng bởi vì chúng được phân chia theo địa điểm : máy khách hàng, máy chủ J2EE, và cơ sở dữ liệu (database). Ứng dụng 3 tầng này chạy bằng cách mở rộng tiêu chuẩn mô hình 2 tầng máy khách hàng và máy chủ bằng cách đặt 1 máy chủ ứng dụng đa luồng giữa máy khách hàng và các lưu trữ phía sau.

1.3 Hai framework hỗ trợ phát triển web

Một framework web thực sự chỉ là một thư viện giúp cho việc phát triển các ứng dụng web một cách dễ dàng hơn. Struts hay Spring MVC là sự lựa chọn tốt. ORM là viết tắt của Object to Relational Mapping, nó được dựa trên JDBC và người dùng để lưu trữ và truy vấn dữ liệu. Nhưng không giống JDBC, ORM hoạt động dựa vào khuôn khổ các đối tượng và cung cấp nhiều cách để lưu trữ và lấy các đối tượng (ở đây nó cũng sử dụng JDBC bên dưới). ORM framework làm giảm rất nhiều mã không cần thiết, cải thiện hiệu suất và giúp nhà phát triển thực hiện công việc dễ dàng hơn. Sự lựa chọn tốt nhất ở đây là Hibernate.

PTIT

Struts Framework là một framwork mã nguồn mở, nó dùng để viết các ứng dụng java, đặc biệt là hay dùng để thiết kế web. Framework này khá khó dùng và ít tính năng hơn so với Spring.

Spring Framwork là một framework tốt cho các dự án lớn, có nhiều người tham gia. Nó sử dụng mô hình MVC giống Struts và có hỗ trợ nhiều thư viện để có thể kiểm tra các dữ liệu đầu vào, các hàm kiểm tra bảo mật, mã hóa, cấu trúc và cấu hình của framework này tốt hơn Struts. Với các dự án lớn thì chúng ta nên sử dụng Spring, còn với các dự án nhỏ hơn thì có thể sử dụng Struts.

2. SPRING

2.1 Giới thiệu

- Spring Framework, hay ngắn hơn là *Spring*, là một framework mã nguồn mở được viết bằng Java. Nó được xem như là một giải pháp kiến trúc tốt nhất của Java EE hiện nay.
- Theo thiết kế, bộ khung hình này giải phóng lập trình viên dùng Java, cho phép họ nhiều quyền tự do hơn.
- Đồng thời cung cấp một giải pháp tiện lợi, đầy đủ dẫn chứng bằng tài liệu, dễ dàng sử dụng, phù hợp với những thực hành thông dụng trong công nghệ phần mềm.

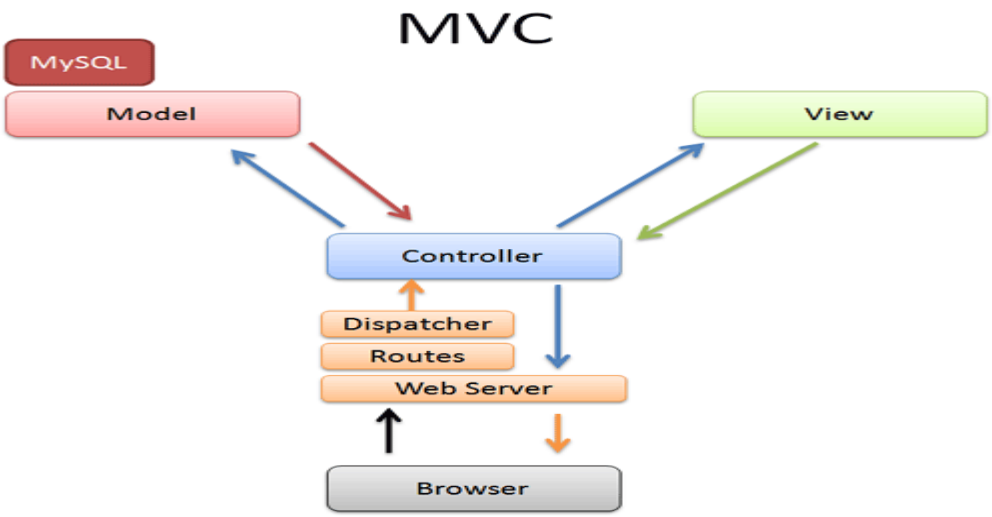
Lý do sử dụng.

- Tất cả các framework đã được tích hợp rất tốt vào Spring.
- Hoạt động rất tốt khi áp dụng theo kiến trúc MVC.
- Sử dụng cơ chế plug-in.
- Kết hợp rất tốt với các O/R (object-relational) Mapping frameworks như là Hibernate.
- Dễ kiểm thử (testing) ứng dụng.
- Ít phức tạp hơn so với các framework khác.
- Cộng đồng người sử dụng rất nhiều, nhiều sách mới được xuất bản.

2.2 Spring Web MVC

PTIT

2.2.1 Mô hình



Hình 9.2: Mô hình MVC

Model (mô hình) gồm các lớp java có nhiệm vụ:

- Biểu diễn dữ liệu (data) và cho phép truy cập tới để lấy (get) và thiết lập (set) dữ liệu trong java (JAVABEAN), Thường thì phần tầng (layer) này mô phỏng 1 cách đầy đủ đối tượng từ thế giới thực.
- Nhận các yêu cầu từ khung nhìn
- Thi hành các yêu cầu đó (tính toán, kết nối CSDL …)
- Trả về các giá trị tính toán theo yêu cầu của phần Controller

View (khung nhìn):

- Bao gồm các mã tương tự như JSP,_HTML, CSS, XML, Javascript, JSON… để hiển thị giao diện người dùng, các dữ liệu trả về từ Model thông qua Controller…

Controller (Bộ điều khiển):

- Đồng bộ hoá giữa Khung nhìn và Mô hình. Tức là với một trang JSP này thì sẽ tương ứng với lớp java nào để xử lý nó và ngược lại, kết quả sẽ trả về trang jsp nào. Nó đóng vai trò điều tiết giữa View và Model.
- Như vậy, chúng ta có thể tách biệt được các mã java ra khỏi mã html. Do vậy, nó đã giải quyết được các khó khăn đã nêu ra trong Mô hình 1. Người thiết kế giao diện và người lập trình java có thể mang tính chất độc lập tương đối.
- Việc gỡ rối (debug) hay bảo trì sẽ dễ dàng hơn, việc thay đổi các theme của trang web cũng dễ dàng hơn …

Spring MVC:

- Framework Spring như thùng chứa nhẹ và hỗ trợ nhiều framework và các thư viện. Nó cho phép các nhà phát triển trộn và kết hợp nhiều framework khi phát

PTIT

triển và triển khai các ứng dụng. Các Spring MVC có những lợi thế từ Spring framework vì nó cung cấp một lần tốt nhất cho framework đó, và hiệu quả cho sự phát triển của các ứng dụng web.

- Các MVC Spring là một framework phát triển web dựa trên các mẫu thiết kế MVC (Model View Controller). Các tính năng của framework Spring MVC là Pluggable công nghệ View và Injection dịch vụ vào điều khiển.

2.2.2 Các khái niệm liên quan

Inversion of Control Container (IoC container):

- Hai package: org.springframework.beans và org.springframework.context cung cấp IoC container cho Spring framework. Giao tiếp BeanFactory cung cấp kỹ thuật cấu hình nâng cao, có khả năng quản lý các đối tượng.
- Giao tiếp ApplicationContext kế thừa từ BeanFactory và thêm một vài chức năng khác như tích hợp với đặc tính Spring AOP, truyền sự kiện, application context như WebApplicationContext sử dụng trong ứng dụng web.
- Hiệu chỉnh các thành phần (components) của chương trình và quản lý vòng đời (lifecycle) của các đối tượng Java.

Bean:

- Các đối tượng trong ứng dụng được quản lý bởi IoC container được gọi là các bean. Một bean thì đơn giản là một đối tượng được khởi tạo, phân phát và được quản lý bởi IoC container. Sự phụ thuộc giữa chúng được phản ánh trong configuration metadata được sử dụng bởi container.
- Giao tiếp org.springframework.beans.factory.BeanFactory là IoC container chịu trách nhiệm chứa, khởi tạo, quản lý và móc nối sự phụ thuộc giữa các bean.
- Một cài đặt (kế thừa) của BeanFactory là XmlBeanFactory - nhận XML configuration metadata và sử dụng nó để cấu hình cho ứng dụng.
- Configuration metadata có định dạng XML (hoặc properties), được sử dụng để viết các *bean definitions* cho các bean mà bạn muốn IoC container quản lý.
- Ở mức cơ bản nhất thì cấu hình IoC container phải chứa ít nhất một bean mà container đó phải quản lý. Khi sử dụng XML-based configuration metadata, các bean này được cấu hình như một hoặc nhiều element bên trong element. Các bean này tương ứng với các đối tượng được tạo trong ứng dụng.

Dependency Injection (DI):

- Kiểm soát ứng dụng, liên kết các đối tượng lại với nhau.
- Tất cả thông tin ứng dụng được cấu hình trong một file xml do dó dễ dàng thay đổi, mở rộng ứng dụng như là một kiến trúc plug-in.

PTIT

- Là các đối tượng định nghĩa sự phụ thuộc của chúng thông qua tham số của phương thức khởi tạo (constructor) hoặc các thuộc tính được thiết lập trên thể hiện (instance) sau khi nó được khởi tạo hoặc trả về từ phương thức factory. Sau đó là công việc của container, nó sẽ đảo ngược sự phụ thuộc đó khi nó tạo bean. Do đó nó có tên là Inversion of Control (IoC).
- Có hai biến thể chính của DI là: Setter Injection và Constructor Injection.

Setter Injection, cho đoạn code sau:

```
<beans>
  <bean name="person" class="examples.spring.Person">
     <property name="email">
          <value>my@email.address</value>
     </property>
  </bean>
</beans>
```

Đoạn code trên tương ứng với đoạn code:

```
Person person = new Person();
person.setEmail("my@email.address");
```

Khởi tạo đối tượng Person và gán giá trị email cho nó. Được nhận ra bởi phương thức setter trên bean sau khi triệu gọi phương thức khởi tạo không tham số hoặc phương thức static factory không tham số để khởi tạo bean.

Constuctor injection:

Được nhận ra bằng cách triệu gọi một phương thức khởi tạo với một số tham số. Một biến thể khác thay thế cho constructor là sử dụng phương thức static factory để trả về thể hiện của đối tượng.

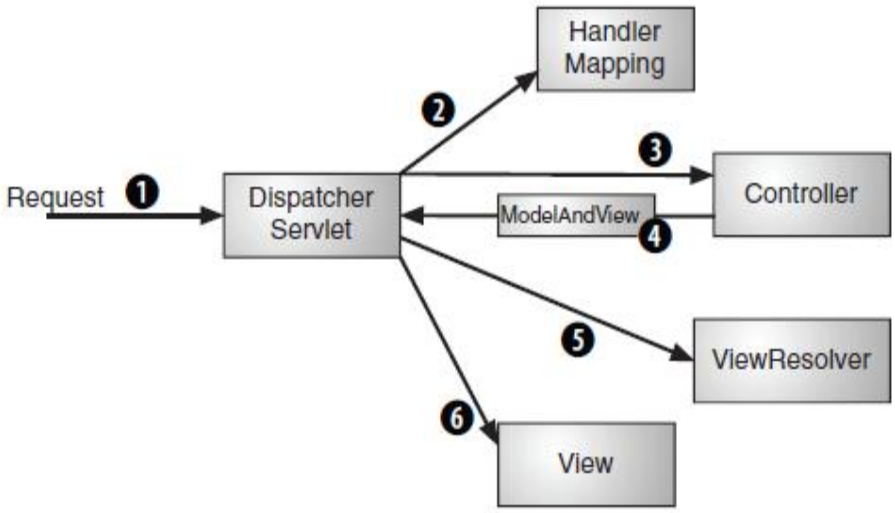
```
<beans>
  <bean name="fileDataProcessor"
class="examples.spring.DataProcessor" singleton="true">
  <constructor-arg>
       <ref bean="fileDataReader"/>
  </constructor-arg>
  </bean>
  <bean name="fileDataReader"
class="examples.spring.FileDataReader" singleton="true">
  <constructor-arg>
       <value>/data/file1.data</value>
  </constructor-arg>
  </bean>
</beans>
```

Đoạn code trên tương đương với:

```
FileDataReader fileDataReader = new
        FileDataReader("/data/file1.data";
DataProcessor fileDataProcessor = new
         DataProcessor(fileDataReader;
```

PTIT

2.3 Cơ chế xử lý Request-Respone



Hình 9.3: vòng đời của Spring MVC

Cũng giống như các java-base MVC FrameWork khác Spring MVC cũng phải requests thông qua một front controller servlet. Một bộ front controller servlet đại diện duy nhất chịu trách nhiệm về yêu cầu các thành phần khác của một ứng dụng để thực hiện việc xử lý thực tế. Trong trường hợp của Spring MVC, DispatcherServlet là bộ điều khiển phía trước. Một Request được gửi bởi DispatcherServlet đến điều khiển (được chọn thông qua một bản đồ xử lý). Một khi điều khiển kết thúc, yêu cầu sau đó được gửi để xem (đó là lựa chọn thông qua ViewResolver) để làm cho đầu ra.

Giải thích sơ đồ luồng dữ liệu:

- 1.Request được gửi đến DispatcherServlet .
- 2.DispatcherServlet gửi yêu cầu đến Handler Mapping (Một bản đồ cấu hình URL) để xác định controller nào sẻ xử lý yêu cầu này.
- 3.DispatcherServlet gửi yêu cầu đến Controller sau khi biết được Controller nào sẻ xử lý yêu cầu. Nếu yêu cầu đó cần truy xuất cơ sở dữ liệu thì Controller sẽ ủy nhiệm cho một business logic hay nhiều hơn một service Objects (MODEL) để lấy thông tin và gửi dữ liệu về cho Controller, lúc này Controller đóng gói mô hình dữ liệu và tên của một view sẽ được tải lên thành đối tượng ModelAndView.
- 4.Gói ModelAndView được gửi trả về DispatcherServlet.
- 5.DispatcherServlet gửi gói ModelAndView cho ViewResolver để tìm xem trang web (JSP) nào sẽ được tải lên.
- DispatcherServlet tải trang web đó lên cùng với dữ liệu của nó.

2.4 Tạo ứng dụng login với spring

PTIT

Ta có bảng User như sau:

Column Name	Datatype	PK	NN	UQ	BIN	UN	ZF	AI	Default
id	INT(11)	✓	✓					✓	
username	VARCHAR(45)		✓	✓					
password	VARCHAR(45)		✓						
fullName	VARCHAR(45)								NULL
idCardNumber	VARCHAR(45)								NULL
idCardType	VARCHAR(45)								NULL
address	VARCHAR(250)								NULL
description	VARCHAR(500)								
adress	VARCHAR(255)								NULL

Hình 9.4: Bảng User

Tạo model User.java như sau:

```
public classUser {
privateInteger id;
privateString username;
privateString password;
privateString fullName;
privateString idCardNumber;
privateString idCardType;
privateString address;
privateString description;
publicInteger getId() {
    return id; }
public voidsetId(Integer id) {
    this.id= id;
    }
//getter và setter các thuộc tính còn lại
```

PTIT

Khai báo Beans.xml như sau:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd ">
<!-- Initialization for data source -->
<bean id="dataSource"
class="org.springframework.jdbc.datasource.DriverManagerDataSource">
<property name="driverClassName" value="com.mysql.jdbc.Driver"/>
<property name="url"
value="jdbc:mysql://localhost:3306/hotelmanagement"/>
<property name="username" value="root"/>
<property name="password" value="12345678"/>
</bean>
<!-- Definition for userJDBCTemplate bean -->
<bean id="userJDBCTemplate" class="control.UserJDBCTemplate">
<property name="dataSource"  ref="dataSource" />
</bean> </beans>
```

Tạo lớp UserJDBCTemplate.java như sau:

```
packagecontrol;
importjava.util.List;
importjavax.sql.DataSource;
importmodel.User;
importorg.springframework.jdbc.core.JdbcTemplate;
public classUserJDBCTemplate {
     privateDataSource dataSource;
     privateJdbcTemplate jdbcTemplateObject;
public voidsetDataSource(DataSource dataSource) {
     this.dataSource= dataSource;
     this.jdbcTemplateObject= newJdbcTemplate(dataSource);
}
public booleancheckLogin(User user) {
     String SQL = "select * from tblUser where username = ?
       and password = ?";
     List<User> result = jdbcTemplateObject.query(SQL,
        newObject[]{user.getUsername(),user.getPassword()},
        newUserMapper());
     if(result.size()>0)
        return true;
     return false;
} }
```

Tạo lớp UserMapper.java như sau:

```
packagecontrol;
importjava.sql.ResultSet;
importjava.sql.SQLException;
importorg.springframework.jdbc.core.RowMapper;
importmodel.User;
public classUserMapper implementsRowMapper<User> {
publicUser mapRow(ResultSet rs, introwNum) throwsSQLException {
      User user = newUser();
      user.setId(rs.getInt("id"));
      user.setUsername(rs.getString("username"));
      user.setPassword(rs.getString("password"));
      user.setFullName(rs.getString("fullName"));
      user.setIdCardNumber(rs.getString("idCardNumber"));
      user.setIdCardType(rs.getString("idCardType"));
      user.setAddress(rs.getString("address"));
      user.setDescription(rs.getString("description"));
      return user;
}}
```

Tạo lớp xử lý LoginControl.java như sau:

```
packagecontrol;
importmodel.User;
importorg.springframework.context.ApplicationContext;
importorg.springframework.context.support.ClassPathXmlApplicationCon
text;
importorg.springframework.stereotype.Controller;
```

PTIT

```
importorg.springframework.web.bind.annotation.ModelAttribute;
importorg.springframework.web.bind.annotation.RequestMapping;
importorg.springframework.web.bind.annotation.RequestMethod;
importorg.springframework.web.servlet.ModelAndView;
importorg.springframework.ui.ModelMap;
@Controller
public classLoginControl{
    privateApplicationContext context= null;
    privateUserJDBCTemplate userJDBCTemplate= null;
publicLoginControl(){
    context= newClassPathXmlApplicationContext("Beans.xml");
    userJDBCTemplate=
(UserJDBCTemplate)context.getBean("userJDBCTemplate");
}
@RequestMapping(value = "/login", method = RequestMethod.GET)
publicModelAndView userLogin() {
     return newModelAndView("login", "command", newUser());
}
@RequestMapping(value = "/loginCheck", method = RequestMethod.POST)
publicString checkUser(@ModelAttribute("SpringWeb")User user,
ModelMap model) {
     model.addAttribute("username", user.getUsername());
     if(userJDBCTemplate.checkLogin(user)){
         return "loginsuccess";
}
     return "loginerror";}
```

Khai báo web.xml như sau:

```
<web-app id="WebApp_ID" version="2.4"
xmlns="http://java.sun.com/xml/ns/j2ee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
<display-name>Spring MVC Web Application</display-name>
<servlet>
<servlet-name>spring-login</servlet-name>
<servlet-class> org.springframework.web.servlet.DispatcherServlet
</servlet-class>
<init-param>
<param-name>contextConfigLocation</param-name>
<param-value>/WEB-INF/spring-login-servlet.xml</param-value>
</init-param>
<load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
<servlet-name>spring-login</servlet-name>
<url-pattern>/</url-pattern>
</servlet-mapping>
</web-app>
```

Khai báo spring-login-servlet.xml như sau:

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

PTIT

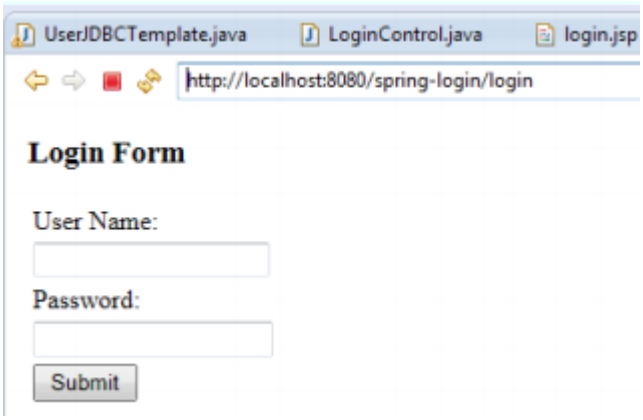
```
  xmlns:context="http://www.springframework.org/schema/context"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:mvc="http://www.springframework.org/schema/mvc"
xsi:schemaLocation="
  http://www.springframework.org/schema/beans
  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
  http://www.springframework.org/schema/context
  http://www.springframework.org/schema/context/spring-context-
3.0.xsd
  http://www.springframework.org/schema/mvc
  http://www.springframework.org/schema/mvc/spring-mvc-3.0.xsd">
<mvc:annotation-driven />
  <context:component-scan base-package="control" />
  <bean id="viewResolver"
class="org.springframework.web.servlet.view.InternalResourceViewReso
lver">
   <property name="prefix" value="/" />
   <property name="suffix" value=".jsp" />
  </bean>
</beans>
```

Tạo trang login.jsp như sau:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<%@ taglib prefix="form"
uri="http://www.springframework.org/tags/form"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html> <head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-
8859-1">
<title>Spring Login Example</title> </head>
<body>
<h3>Login Form</h3>
<form:form method="POST" action="/spring-login/loginCheck">
<table>
  <tr><td>User Name:</td></tr>
  <tr><td><form:input path="username" /></td></tr>
  <tr><td>Password:</td></tr>
  <tr><td><form:password path="password" /></td></tr>
  <tr><td><input type="submit" value="Submit" /></td></tr>
</table>
</form:form>
</body> </html>
```

Hiển thị trang login.jsp:

PTIT



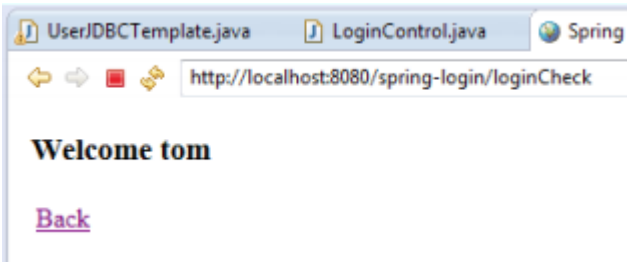
Hình 9.5: Trang login

Trang loginsuccess.jsp:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<%@ taglib prefix="core" uri="http://java.sun.com/jsp/jstl/core"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html> <head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-
8859-1">
<title>Spring Login Example</title> </head>
<body>
<h3>Welcome <core:out value="${username}" /></h3>
<table>
   <tr>
   <td><a href="login">Back</a></td>
   </tr>
</table>
</body> </html>
```

PTIT

Hiện thị trang loginsuccess.jsp như sau:



Hình 9.6: Trang đăng ký thành công

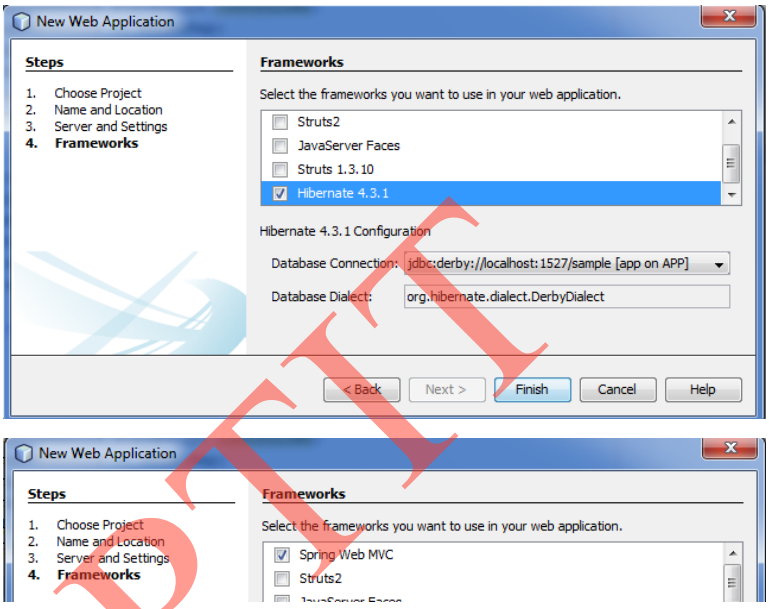
Tạo trang logineror.jsp như sau:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html> <head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-
8859-1">
```

```
<title>Spring Login Example</title></head><body>
<h3>Login Error !!! Click below to login again</h3>
<table>
   <tr><td><a href="login">Retry</a></td></tr>
</table>
</body></html>
```

2.5 Kết hợp Spring Framework với Hibernate Framework

Hibernate là một dịch vụ lưu trữ và truy vấn dữ liệu quan hệ mạnh mẽ và nhanh chóng. Vậy chúng ta có thể kết hợp Spring Framework với Hibernate Framework. Khi xây dựng ứng dụng web thì ta có thể tích vào cả 2 tùy chọn là Spring Framework và Hibernate Framework.



Hình 9.7 Dự án với Spring Framework và Hibernate Framework

Tạo tập tin **hibernate.cfg.xml** để cấu hình Hibernate với các tham số cần thiết về thông tin cơ sở dữ liệu như tên đăng nhập, mật khẩu, tên database…

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate
Configuration DTD 3.0//EN" "http://hibernate.sourceforge.net/hibernate-
configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect </property>
        <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver </property>
        <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/db_demo?createD
atabaseIfNotExist=true</property>
        <property name="hibernate.connection.useUnicode">true</property>
        <property name="hibernate.connection.characterEncoding">UTF-
8</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">root</property>
```

PTIT

```
        <property
name="hibernate.current_session_context_class">thread</property>
        <property name="hibernate.hbm2ddl.auto">update</property>
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.c3p0.min_size">5</property>
        <property name="hibernate.c3p0.max_size">20</property>
        <property name="hibernate.c3p0.timeout">300</property>
        <property name="hibernate.c3p0.max_statements">50</property>
        <property name="hibernate.c3p0.idle_test_period">3000</property>
        <mapping class="kenhlaptrinh.net.model.TaiKhoan"/>
    </session-factory>
</hibernate-configuration>
```

Sau khi **hibernate.cfg.xml** bạn tạo lớp **HibernateUtil.java** như sau:

```
package util;
import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.SessionFactory;
public class HibernateUtil {
    private static final SessionFactory sessionFactory;
    static {
        try {
            sessionFactory = new
AnnotationConfiguration().configure("/kenhlaptrinh/net/util/hibernat
e.cfg.xml").buildSessionFactory();
        } catch (Throwable ex) {
            throw new ExceptionInInitializerError(ex);
        }
    }
    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

Chúng ta tạo lớp **SanPham.java** bao gồm các thuộc tính cơ bản như: mã sản phẩm, tên sản phẩm, giá bán, hình ảnh… bằng cấu hình Hibernate Annotations như sau đây:

```
package model;
import java.io.Serializable;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;

@Entity(name = "sanpham")
public class SanPham implements Serializable {
    @Id
    @GeneratedValue
    private long maSanPham;
    private String tenSanPham;
    private String hinhAnh;
    private double giaBan;
    private String moTa;
```

PTIT

```
    // setter & getter….
}
```

Trong đó:

@Entity: chú thích xác định lớp này như là một thực thể

@Table(name = "tên bảng"): chỉ định entity này ánh xạ xuống table nào trong database

@id: xác định thuộc tính làm khóa chính

@GeneratedValue: xác định khoá chính tự động tăng

Sau đó, ta sẽ viết lớp DAO để kết nối với cơ sở dữ liệu như các phương thức cơ bản thêm, sửa, xóa, hiển thị dữ liệu…

lớp **SanPhamDAO.java**

```
package dao;
import java.util.ArrayList;
import model.SanPham;
public interface SanPhamDAO {
    public boolean insert(SanPham sp);
    public boolean update(SanPham sp);
    public boolean delete(SanPham sp);
    public boolean findByID(long id);
    public ArrayList<SanPham> getAll();
}
```

lớp **SanPhamDAOImpl.java**

```
package dao;
import java.util.ArrayList;
import model.SanPham;
import org.springframework.stereotype.Repository;
@Repository
public class SanPhamDAOImpl implements SanPhamDAO {
    @Override
    public boolean insert(SanPham sp) {
        // code
        return false;
    }
    @Override
    public boolean update(SanPham sp) {
        // code
        return false;
    }
    @Override
    public boolean delete(SanPham sp) {
        // code
        return false;
    }
    @Override
```

PTIT

```
    public SanPham findByID(long id) {
        // code
        return null;
    }
    @Override
    public ArrayList<SanPham> getAll() {
        // code
        return null;
    }
}
```

lớp **SanPhamService.java**

```
package service;
import java.util.ArrayList;
import model.SanPham;
public interface SanPhamService {
    public boolean insert(SanPham sp);
    public boolean update(SanPham sp);
    public boolean delete(SanPham sp);
    public SanPham findByID(long id);
    public ArrayList<SanPham> getAll();
}
```

lớp **SanPhamServiceImpl.java**

```
package service;
import java.util.ArrayList;
import model.SanPham;
import org.springframework.stereotype.Service;
@Service
public class SanPhamServiceImpl implements SanPhamService {
    @Override
    public boolean insert(SanPham sp) {
        // code
        return false;
    }
    @Override
    public boolean update(SanPham sp) {
        // code
        return false;
    }
    @Override
    public boolean delete(SanPham sp) {
        // code
        return false;
    }
    @Override
    public SanPham findByID(long id) {
        // code
        return null;
    }
    @Override
    public ArrayList<SanPham> getAll() {
```

PTIT

```
        // code
        return null;
    }
}
```

2.6 Ví dụ thêm, sửa, xóa (CRUD) trong Spring Framework

Khi xây dựng ứng dụng web thì chúng ta tích vào cả 2 tùy chọn là Spring Framework và Hibernate Framework. Sau đó tạo tập tin **hibernate.cfg.xml** để cấu hình Hibernate với các tham số cần thiết về thông tin cơ sở dữ liệu như tên đăng nhập, mật khẩu, tên database…

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate
Configuration DTD 3.0//EN" "http://hibernate.sourceforge.net/hibernate-
configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.dialect">org.hibernate.dialect.
sqlserverdialect</property>
        <property name="hibernate.connection.driver_class">
com.sun.sql.jdbc.sqlserver.SQLServerDriver </property>
        <property name="hibernate.connection.url"> jdbc:sqlserver:
//localhost:1433/Trading?createDatabaseIfNotExist=true</property>
        <property name="hibernate.connection.useUnicode">true</property>
        <property name="hibernate.connection.characterEncoding">UTF-
8</property>
        <property name="hibernate.connection.username">sa</property>
        <property name="hibernate.connection.password">123456</property>
        <property
name="hibernate.current_session_context_class">thread</property>
        <property name="hibernate.hbm2ddl.auto">update</property>
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.c3p0.min_size">5</property>
        <property name="hibernate.c3p0.max_size">20</property>
        <property name="hibernate.c3p0.timeout">300</property>
        <property name="hibernate.c3p0.max_statements">50</property>
        <property name="hibernate.c3p0.idle_test_period">3000</property>
        <mapping class="model.SanPham"/>
    </session-factory>
</hibernate-configuration>
```

Sau khi tạo tập tin **hibernate.cfg.xml** bạn tạo lớp **HibernateUtil.java** như sau:

```
package util;
import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.SessionFactory;
public class HibernateUtil {
    private static final SessionFactory sessionFactory;
    static {
        try {
            sessionFactory = new
AnnotationConfiguration().configure("/util/hibernate.cfg.xml").build
SessionFactory();
```

PTIT

```
        } catch (Throwable ex) {
            throw new ExceptionInInitializerError(ex);
        }
    }
    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

Tạo lớp **SanPham.java** bao gồm các thuộc tính cơ bản như mã sản phẩm, tên sản phẩm, giá bán, hình ảnh… bằng cấu hình Hibernate Annotations như sau đây:

```
package model;
import java.io.Serializable;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
@Entity(name = "sanpham")
public class SanPham implements Serializable {
    @Id
    @GeneratedValue
    private long maSanPham;
    private String tenSanPham;
    private String hinhAnh;
    private double giaBan;
    private String moTaNgan;
    // setter & getter…
}
```

Lớp DAO:

```
package dao;
import java.util.ArrayList;
import model.SanPham;
import util.HibernateUtil;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.Transaction;
import org.springframework.stereotype.Repository;
@Repository
public class SanPhamDAOImpl implements SanPhamDAO {
    @Override
    public boolean insert(SanPham sp) {
        Session session =
HibernateUtil.getSessionFactory().openSession();
        Transaction transaction = null;
        try {
            transaction = session.beginTransaction();
            session.save(sp);
            transaction.commit();
            return true;
        } catch (Exception ex) {
            if (transaction != null) {
                transaction.rollback();
```

PTIT

```
            }
            ex.printStackTrace();
        } finally {
            session.flush();
            session.close();
        }
        return false;
    }
    @Override
    public boolean update(SanPham sp) {
        Session session =
HibernateUtil.getSessionFactory().openSession();
        Transaction transaction = null;
        try {
            transaction = session.beginTransaction();
            session.update(sp);
            transaction.commit();
            return true;
        } catch (Exception ex) {
            if (transaction != null) {
                transaction.rollback();
            }
            ex.printStackTrace();
        } finally {
            session.flush();
            session.close();
        }
        return false;
    }
    @Override
    public boolean delete(SanPham sp) {
        Session session =
HibernateUtil.getSessionFactory().openSession();
        Transaction transaction = null;
        try {
            transaction = session.beginTransaction();
            session.delete(sp);
            transaction.commit();
            return true;
        } catch (Exception ex) {
            if (transaction != null) {
                transaction.rollback();
            }
            ex.printStackTrace();
        } finally {
            session.flush();
            session.close();
        }
        return false;
    }
    @Override
    public SanPham findByID(long id) {
```

PTIT

```
        Session session =
HibernateUtil.getSessionFactory().openSession();
        Transaction transaction = null;
        try {
            transaction = session.beginTransaction();
            SanPham sp = (SanPham) session.get(SanPham.class, id);
            transaction.commit();
            return sp;
        } catch (Exception ex) {
            if (transaction != null) {
                transaction.rollback();
            }
            ex.printStackTrace();
        } finally {
            session.flush();
            session.close();
        }
        return null;
    }
    @Override
    public ArrayList<SanPham> getAll() {
        Session session =
HibernateUtil.getSessionFactory().openSession();
        Transaction transaction = null;
        try {
            transaction = session.beginTransaction();
            Query query = session.createQuery("from san_pham");
            ArrayList<SanPham> list = (ArrayList<SanPham>)
query.list();
            transaction.commit();
            return list;
        } catch (Exception ex) {
            if (transaction != null) {
                transaction.rollback();
            }
            ex.printStackTrace();
        } finally {
            session.flush();
            session.close();
        }
        return null;
    }
}
```

Lớp Service:

```
package service;
import java.util.ArrayList;
import dao.SanPhamDAO;
import model.SanPham;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
```

PTIT

```
@Service
public class SanPhamServiceImpl implements SanPhamService {
    @Autowired
    SanPhamDAO sanPhamDAO;
    @Override
    public boolean insert(SanPham sp) {
        return sanPhamDAO.insert(sp);
    }
    @Override
    public boolean update(SanPham sp) {
        return sanPhamDAO.update(sp);
    }
    @Override
    public boolean delete(SanPham sp) {
        return sanPhamDAO.delete(sp);
    }
    @Override
    public SanPham findByID(long id) {
        return sanPhamDAO.findByID(id);
    }
    @Override
    public ArrayList<SanPham> getAll() {
        return sanPhamDAO.getAll();
    }
}
```

Viết lớp controller để xử lý

```
package controller;
import model.SanPham;
import service.SanPhamService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.ModelMap;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
@Controller
public class TrangChuController {
    @Autowired
    SanPhamService sanPhamService;
    // trang chủ
    @RequestMapping(value = "Home.html", method = RequestMethod.GET)
    public String viewIndex(ModelMap mm) {
        mm.put("listSanPham", sanPhamService.getAll());
        return "pages/Home";
    }
    // tạo mới dữ liệu
    @RequestMapping(value = "them-du-lieu.html", method =
RequestMethod.GET)
    public String insert(ModelMap mm) {
        mm.put("sanPham", new SanPham());
```

PTIT

```
        return "pages/san_pham";
    }
    // sửa dữ liệu
    @RequestMapping(value = "sua-du-lieu/{id}.html", method =
RequestMethod.GET)
    public String update(ModelMap mm, @PathVariable(value = "id")
long id) {
        mm.put("sanPham", sanPhamService.findByID(id));
        return "pages/san_pham";
    }
    // lưu dữ liệu
    @RequestMapping(value = "luu-du-lieu.html", method =
RequestMethod.POST)
    public String save(ModelMap mm, @ModelAttribute(value =
"sanPham") SanPham sanPham) {
        sanPhamService.insert(sanPham);
        mm.put("listSanPham", sanPhamService.getAll());
        return "pages/Home";
    }
}
```

Sau đó, ta sẽ tạo giao diện hiển thị phần thông tin sản phẩm. Ví dụ tạo 2 trang với một trang nhằm hiển thị danh sách toàn bộ sản phẩm và trang còn lại để người dùng thêm, cập nhật sản phẩm…

```
<div style="clear: both; width: 80%; margin:10px auto;"><a
href="them-du-lieu.html" style="float: right;">Thêm mới</a></div>
<div style="clear: both;"></div>
<table>
     <thead>
            <tr>
                 <th>ID</th>
                 <th>Hình ảnh</th>
                 <th style="width: 200px;">Tên sản phẩm</th>
                 <th>Giá bán</th>
                 <th>Mô tả</th>
                 <th style="width: 80px;">Tùy chọn</th>
           </tr>
     </thead>
     <tbody>
          <c:forEach var="item" items="${listSanPham}">
          <tr>
               <td>${item.maSanPham}</td>
               <td><img width="200px" src="${item.hinhAnh}"> </td>
               <td>${item.tenSanPham}</td>
               <td>${item.giaBan}</td>
               <td>${item.moTaNgan}</td>
               <td><a href="sua-du-
lieu/${item.maSanPham}.html">Sửa</a> | <a href="">Xóa</a></td>
          </tr>
          </c:forEach>
     </tbody>
```

PTIT

```
</table>
```

CSS cho giao diện:

```
<style>
      table {
            width: 80%;
            border-collapse: collapse;
            margin:10px auto;
      }
      tr:nth-of-type(odd) {
           background: #eee;
      }
      th {
           background: #3498db;
           color: white;
           font-weight: bold;
      }
      td, th {
           padding: 10px;
           border: 1px solid #ccc;
           text-align: left;
           font-size: 18px;
      }
</style>
```

```
<form:form action='luu-du-lieu.html' method="POST"
commandName="sanPham">
     <table>
           <tr>
                <th style="width: 80px;">ID</th>
                <th>Hình ảnh</th>
                <th>Tên sản phẩm</th>
                <th style="width: 50px;">Giá bán</th>
                <th>Mô tả ngắn</th>
           </tr>
           <tr>
                <td></td>
                <td>
                     <form:input path="tenSanPham" placeholder="Nhập
tên sản phẩm..." value="${sanPham.tenSanPham}"/>
                </td>
                <td>
                     <form:input path="hinhAnh" placeholder="Nhập
url hình ảnh..."/>
                </td>
                <td>
                     <form:input path="giaBan" placeholder="Nhập giá
bán..."/>
                </td>
                <td>
                     <form:input path="moTa" placeholder="Nhập mô tả
sản phẩm..."/>
                </td>
```

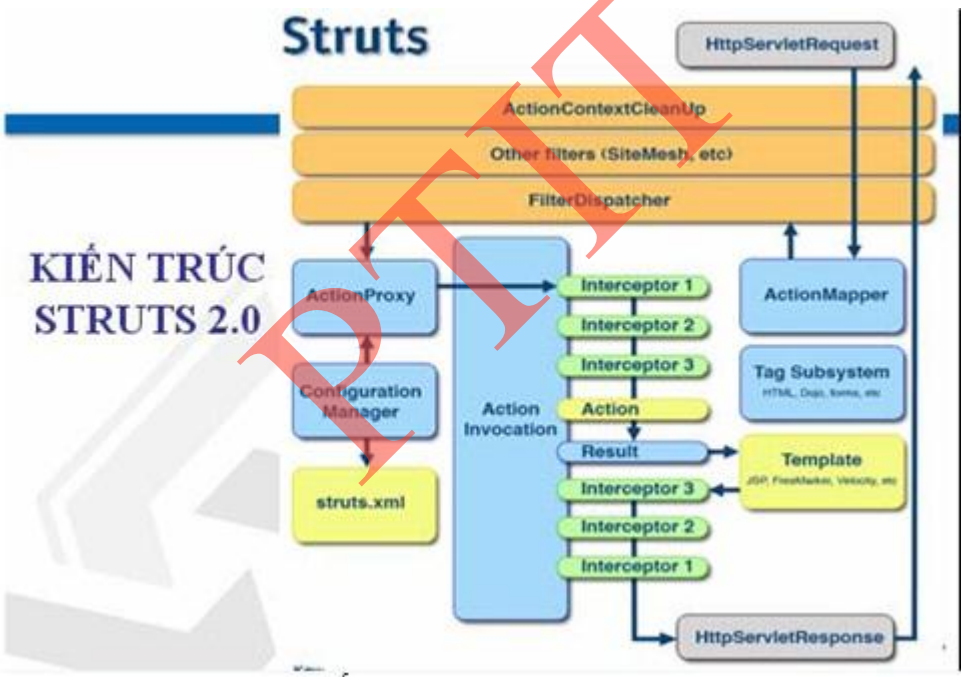
PTIT

```
            </tr>
            <tr>
                <form:hidden path="maSanPham" placeholder="Nhập mã
sản phẩm..." value="${sanPham.maSanPham}"/>
                <td colspan="6"><input type="submit" value="Lưu dữ
liệu" style="float: right;"></td>
            </tr>
     </table>
</form:form>
```

3. STRUTS

3.1 Khái niệm

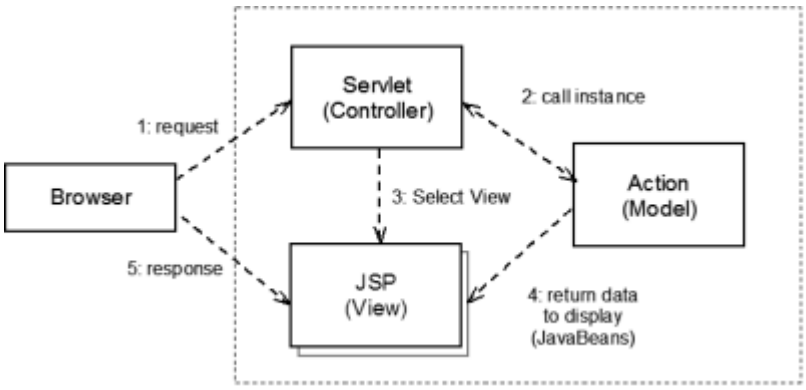
Struts là một framework phục vụ việc phát triển các ứng dụng Web trên Java. Struts cung cấp một framework thống nhất để triển khai (deploy) các ứng dụng Servlet và JSP sử dụng kiến trúc MVC. Sử dụng mẫu thiết kế Model-View-Controller (MVC), Struts giải quyết rất nhiều các vấn đề liên quan đến các ứng dụng Web hướng business đòi hỏi hiệu năng cao sử dụng Java servlet và JSP.



Hình 9.8: Kiến trúc của Struts 2.0

Struts cơ bản định hình lại cách các Web programmer nghĩ về và cấu trúc một ứng dụng Web. Struts là một tập thư viện các thẻ JSP tùy chọn (Custom JSP Tag): Struts cung cấp các thư viện thẻ tùy chọn cho việc thể hiện các thuộc tính của bean, quản lý các HTML forms, lặp lại các kiểu cấu trúc dữ liệu, và đưa ra các HTML có điều kiện.

PTIT



Hình 9.9: Kiến trúc Strust 2

Mô tả tiến trình mà hầu hết các ứng dụng struts tuân theo 5 bước cơ bản sau:
1. Một request được gửi đến từ view

2. ActionServlet sẽ tiếp nhận request này, phân tích, kiểm tra. Sau đó chỉ định cho Action tương ứng thực thi yêu cầu, tính toán những tác vụ cần thiết. ActionServlet đóng vai trò là **Controller**

3. Action sẽ thao tác và xử lí trên Model của ứng dụng

4. Mỗi khi Action hoàn thành việc thao tác và xử lí, nó trả quyền điều khiển về cho ActionServlet kèm theo một key gắn kèm với kết quả trả về. ActionServlet sẽ dựa vào key này mà quyết định xem các kết quả trả về sẽ được hiển thị như thế nào.
5. ActionServlet trả lời bằng cách gửi lại một request cho view là một liên kết đến kết quả trả về của Action thông qua key trên. Sau đó, view làm nốt công việc trình bày kết quả.

PTIT

3.2 Cấu trúc của Strust

Struts Model Components:

Model là một thành phần được cho là quan trọng nhất trong các ứng dụng MVC. Model bao gồm các business entities và một tập các qui tắc để quản lí việc tổ chức và thao tác dữ liệu. Struts không cung cấp các Model Component chuyên dụng, tuy nhiên chúng ta có thể sử dụng lại các Model của các ứng dụng khác hoặc tự xây dựng các model của riêng mình.

Struts View Components:

Mục đích của thanh phần Struts View này cũng giống y như một thành phần view trong các ứng dụng theo mô hình MVC: chịu trách nhiệm trình bày thông tin được cung cấp bởi Model. Struts sử dụng JSP để thiết kế thành phần View. Ngoài ra, để hỗ trợ và mở rộng khả năng của View, chúng ta cũng có thêm thư viện Taglib, sử dụng HTML, JS… cho mục đích trình bày thông tin.

Struts Controller Components:

Struts cung cấp hai thành phần rất quan trọng đó là ActionServlet và Action để điều khiển và quản lí mọi yêu cầu của người dùng cũng như việc thao tác với dữ liệu ActionServlet chịu trách nhiệm nhận và xử lí các request từ phía người dùng, chỉ định Action thực thi tương ứng với từng yêu cầu cụ thể. Action chịu trách nhiệm thao tác với Model, nó kết hợp rất chặt chẽ với ActionServlet. Cả hai thành phần này đóng vai trò làm Controller trong Struts.

Struts taglib: Cung cấp một tập các tag library cho việc phát triển ứng dụng, bao gồm cả các taglib hỗ trợ thiết kế HTML và JSP taglib. Struts Config File cấu hình của ứng dụng Struts.

3.3 Tạo Login với Strust 2

Chúng ta tạo Project Login có tệp cấu hình web.xml như sau:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:web="http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
     xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
     id="WebApp_ID" version="2.5">
<display-name>Login</display-name>
  <filter>
     <filter-name>struts2</filter-name>
     <filter-class>
     org.apache.struts2.dispatcher.FilterDispatcher
  </filter-class>
  </filter>
  <filter-mapping>
     <filter-name>struts2</filter-name>
     <url-pattern>/*</url-pattern>
  </filter-mapping>
  <welcome-file-list>
     <welcome-file>Login.jsp</welcome-file>
  </welcome-file-list>
</web-app>
```

Action class: chúng ta cần một class Action để làm nhiệm vụ kiểm tra xác thực username và password của user. Class LoginAction.java ta có đoạn mã sau:

```
package controller;
public class LoginAction {
      private String username;
      private String password;
      public String execute() {
        if (this.username.equals("admin") &&
            this.password.equals("admin123")) {
        return "success";
      } else {
```

PTIT

```
        return "error"; }
}
     public String getUsername() {
        return username; }
     public void setUsername(String username) {
        this.username = username; }
     public String getPassword() {
        return password; }
     public void setPassword(String password) {
        this.password = password;
}}
```

ResourceBundle

- ResourceBundle rất hữu dụng trong việc chứa các nội dung tĩnh trong một file. Hầu hết các ứng dụng đều định nghĩa ResourceBundle trong file *.properties, dùng để chứa nội dung các message, ví dụ trong trường hợp này là "Username hoặc Password không đúng".
- Chúng ta sẽ tạo và định nghĩa file ApplicationResources.properties.
- Trước tiên, chúng ta tạo một source folder là resources: Click phải Project -> New -> Sources Folder -> đặt tên là resources -> Ok.
- Tiếp theo, click phải resources-> New -> Other ->General -> File ->Next -> đặt tên là ApplicationResources.properties -> Finish.
- Nội dung ApplicationResources.properties là như sau:

```
label.username= Username
label.password= Password
label.login= Login
```

JSP

Chúng ta sẽ tạo 2 file JSP để hiển thị cho user. Login.jsp sẽ là trang mặc định của ứng dụng, gồm một form login đơn giản với 2 trường là username và password. Khi đăng nhập thành công, user sẽ được tải qua trang Welcome.jsp với lời chào mừng.

Tệp Login.jsp như sau:

```
<%@ page contentType="text/html; charset=UTF-8"%>
<%@ taglib prefix="s" uri="/struts-tags"%>
<html> <head>
<title>Struts 2 - Login Application</title> </head>
<body>
<h2>Struts 2 - Login Application</h2>
     <s:actionerror />
     <s:form action="login.action" method="post">
     <s:textfield name="username" key="label.username" size="20" />
     <s:password name="password" key="label.password" size="20" />
     <s:submit method="execute" key="label.login" align="center" />
</s:form>
</body> </html>
```

PTIT

Tệp Welcome.jsp như sau:

```
<%@ page contentType="text/html; charset=UTF-8"%>
<%@ taglib prefix="s" uri="/struts-tags"%>
<html> <head>
<title>Welcome</title> </head>
<body>
     <h2>Hello, <s:property value="username" />...!</h2>
</body> </html>
```

Chú ý: ta sử dụng tag <s:> trong struts2 để hiển thị textbox và label. Struts2 cung cấp một thư viện thẻ khá mạnh mẽ và phong phú để hiển thị giao diện.

Định nghĩa struts.xml

Struts2 sẽ đọc cấu hình và định nghĩa các lớp từ struts.xml. Tạo file struts.xml trong thư mục resources : Click phải resources chọn New ->Other -> XML -> XML file -> Đặt tên là struts.xml -> Finish. File struts.xml có nội dung như sau:

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE struts PUBLIC
"-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
"http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>
    <constant name="struts.enable.DynamicMethodInvocation"
       value="false" />
    <constant name="struts.devMode" value="false" />
    <constant name="struts.custom.i18n.resources"
       value="ApplicationResources" />
    <package name="default" extends="struts-default" namespace="/">
       <action name="login" class="controller.LoginAction">
          <result name="success">Welcome.jsp</result>
          <result name="error">Login.jsp</result>
       </action>
    </package>
</struts>
```

PTIT

Chú ý:

- Trong file trên, chúng ta đã định nghĩa LoginAction. Có 2 kết quả được là "success" và "error", dựa vào kết quả trả về của hàm execute() trong LoginAction.
- Chúng ta cũng khai báo một hằng số có tên là struts.custom.i18n.resources. Hằng số này có liên kết một file resource bundle mà chúng ta đã tạo bước 4.
- Lớp LoginAction có chứa phương thức execute(), đây là phương thức mặc định khi gọi lớp này.

Nâng cao:

- Trong Project trên, vẫn còn nhược điểm, khi chúng ta đăng nhập thất bại, giao diện không có gì thay đổi để nhắc nhở người dùng đăng nhập sai.

- Để thêm chức năng này, chúng ta cần thêm một message báo lỗi trong file ResourceBundle. Lúc này, nội dung file ApplicationResources.properties có nội dung như sau:

```
label.username= Username
label.password= Password
label.login= Login
error.login= Invalid Username/Password. Please try again.
```

Ngoài ra, chúng ta cần chỉnh cho LoginAction xử lý xuất hiện message báo lỗi khi đăng nhập thất bại. Lúc này, LoginAction có nội dung như sau:

```
package controller;
import com.opensymphony.xwork2.ActionSupport;
public class LoginAction extends ActionSupport {
private static final long serialVersionUID = 1L;
    private String username;
    private String password;
    public String execute() {
       if (this.username.equals("admin")&&
          this.password.equals("admin123")) {
         return "success";
       } else {
           addActionError(getText("error.login"));
           return "error"; }}
    public String getUsername() {
      return username;}
    public void setUsername(String username) {
      this.username = username;}
    public String getPassword() {
      return password;}
    public void setPassword(String password) {
      this.password = password;
 }}
```

PTIT

TÀI LIỆU THAM KHẢO

1. http://developers.sun.com/product/jdbc/drivers
2. H. M. Deitel, P. J. Deitel, T. R. Nieto, "Internet - World Wide Web How To Program, 3rd Edition", 2005
3. E. R. Harold, "Java Network Programming", 4th edition, O'Reilly, 2006
4. "Giáo trình Java Script", Khoa Toán Tin, Đại học Quốc gia Hà Nội, 2003
5. Nguyễn Hữu Tuấn, "Giáo trình thiết kế Web với HTML", 2004
6. Microsoft Education, "Microsoft Frontpage 2002 Tutorial", phiên bản Tiếng Việt, 2004.
7. Nguyễn Phú Tài, Hữu Thanh Design, "Căn bản về Dreamweaver 4.0", 2004
8. G.Reese. *Database Programming with JDBC and Java*. O'Reilly,2003.
9. H. Bergsten. *Java Server Page*. 3rd edition, O'Reilly,2004.
10. H. Maruyama et. al. *XML and Java: Developing Web Applications.* Addison Wesley, 2nd edition, 2002.